

The University of Western Australia

Honours Dissertation

Evaluating moving target defence against advanced persistent threat attackers modelled on documented intrusion campaigns

Marc Labouchardiere

(23857377)

Supervised by

Dr Jin B. Hong

Word count: 15 729

2026

Abstract

Moving target defence (MTD) has been proposed against advanced persistent threats (APTs), attackers who pursue a specific objective over months or years. MTD continually reconfigures a network so that what an attacker has learned about it goes out of date before it can be used. Yet simulation studies of MTD commonly test it against simplified attackers that follow a fixed sequence of steps. We build an APT attacker model that chooses its next step by objective, from published reports of real intrusion campaigns. It takes the same attack actions as an existing MTD simulator's scripted baseline attacker, in a different order and at different times. We run both attackers on the same networks, under seven MTD mechanisms at six deployment intervals. At every deployment interval, the most effective MTD mechanisms against the APT attacker model change the network's addresses or layout, and those against the baseline attacker change its services. At the default interval, service diversity reduces the number of hosts the baseline attacker compromises by 92 %, but the number the APT attacker model compromises by 36 %. These results make a case for testing MTD against more than one attacker model.

Acknowledgements

Thank you, Jin, for the many ideas I have explored, for keeping me from falling down many rabbit holes (not that it never happened), and for your frank and honest feedback.

Contents

Abstract ii

Acknowledgements iii

1 Introduction 1

2 Background 3

2.1 Moving target defence . . . 3

2.2 MTDSim . . . 3

2.2.1 Network model . . . 4

2.2.2 MTD mechanisms and deployment strategies . . . 5

2.2.3 Baseline attacker . . . 6

3 Literature review 8

3.1 APT attacker behaviour . . . 8

3.1.1 Advanced persistent threat . . . 8

3.1.2 MITRE ATT&CK . . . 9

3.1.3 Attack profiling . . . 9

3.2 MTD evaluation . . . 10

3.2.1 MTD modelling approaches . . . 11

3.2.2 MTD metrics . . . 11

3.2.3 MTD evaluation methods . . . 12

3.3 MTD attacker models . . . 12

3.3.1 Properties of an APT attacker . . . 12

3.3.2 Recent MTD attacker models . . . 13

3.4 Research gap . . . 15

4 APT attacker model 16

4.1 Attack graph from attack flows . . . 16

4.2 Attack profiles by objective . . . 18

4.3 Petri-net formalism . . . 19

4.4 Integration with MTDSim . . . 21

4.4.1 Tactic-to-action mapping . . . 22

4.4.2 Tactic durations . . . 23
4.4.3 Failure matrix . . . 25
4.4.4 Vulnerability memory . . . 25
4.5 Evaluation metrics . . . 26
4.5.1 Attacker behaviour . . . 26
4.5.2 Attack outcome . . . 28
4.5.3 MTD effectiveness . . . 28

5 Evaluation **30**
5.1 Experimental setup . . . 30
5.2 APT attacker model versus baseline attacker . . . 31
5.3 APT attacker model versus MTD . . . 33
5.3.1 Attack actions blocked and time lost per MTD deployment . . . 33
5.3.2 MTD ranking against each attacker . . . 34
5.3.3 MTD effectiveness by attack profile . . . 36
5.4 Ablation studies . . . 36

6 Discussion **39**
6.1 APT attacker model versus baseline attacker . . . 39
6.2 APT attacker model versus MTD . . . 40
6.3 APT attacker model versus recent MTD attacker models . . . 41
6.4 Implications for MTD evaluation . . . 42
6.5 Threats to validity . . . 43

7 Conclusion **47**
7.1 Contributions . . . 47
7.2 Future work . . . 48

AI Use Declaration **49**

A Original honours proposal **50**

B Predicate tests of the MTD attacker models **56**

C Supplementary material for the APT attacker model **57**
C.1 Attack graph with techniques as nodes . . . 57
C.2 Sources for each attack flow's objective . . . 57
C.3 Edges of the attack profiles . . . 59
C.4 Tactic durations and attack action times . . . 60

C.5 Failure matrix rules and distance factor 61
C.6 Sources of the APT attacker model's values 62

D Supplementary material for the MTD evaluation 63
D.1 MTDShield's settings 63
D.2 Values of attack actions blocked and time lost 64
D.3 Values of the MTD ranking 64
D.4 Sensitivity analysis of the tactic durations 67
D.5 Ablation study tables 68
D.5.1 Ablation of the tactic durations 68

Bibliography 71

CHAPTER 1

Introduction

Attackers who break into a network to spy on it go unnoticed for a median of four months [1]. In February 2024, US and Australian cyber agencies reported that the Chinese state-sponsored group Volt Typhoon had held footholds in American critical infrastructure networks for at least five years [2]. The group stayed hidden by signing in with stolen but valid credentials and running the administration tools already installed on each network [2]. Volt Typhoon is an advanced persistent threat (APT), a well-resourced attacker that pursues a specific objective against a specific target over months or years [3].

Before breaking in, Volt Typhoon conducted reconnaissance of its victims' networks [2]. A static network keeps such reconnaissance valid and lets an attacker plan at its leisure [4]. Moving target defence (MTD) changes a network's addresses, layout, software and credentials on a schedule, so that stolen credentials and reconnaissance go out of date even if the attacker is never detected [5], [6]. MTD aims to shift the advantage of time from the attacker to the defender [4], [7].

MTD has been proposed as a defence against APTs [3], yet it is rarely tested against one. Most tests run in simulation, against attacker models far simpler than an APT [5], [8], and how well MTD performs depends on the attacker it faces [9]. This gap motivates the research question of this dissertation:

How does MTD perform against APT attackers?

The question breaks down into three sub-questions:

SQ1 How can the behaviour of APT attackers be recovered from published threat reports?

SQ2 How can the recovered behaviour be executed as an APT attacker model inside an existing MTD simulator?

SQ3 How does MTD perform against the APT attacker model compared with the simulator's baseline attacker?

We recover the order of APT attackers' steps from cyber threat intelligence (CTI), the threat reports that security firms and government agencies publish on attacks they have investigated. Our source is 38 attack flows, graphs of an attack's steps that analysts drew by hand from CTI reports [10]. We aggregate the 38 attack flows into one graph, then split it into attack profiles by objective, such as stealing data. Each attack profile records the steps drawn in the attack flows with that objective, and how often each step led to the next. We execute each attack profile as the APT attacker model in MTDSim, an existing MTD simulator [6]. MTDSim's original scripted attacker is the baseline attacker. It follows a fixed procedure of six attack actions, such as scanning a host's ports and

exploiting its vulnerabilities. The APT attacker model takes the same six attack actions, but its attack profile decides which comes next, and values we declared decide when and what to do after one fails. Both attackers run on the same network. We run both first with no MTD, then under each of MTDSim's seven MTD mechanisms at six deployment intervals. We measure MTD's performance by how much it reduces the number of hosts each attacker compromises. If the same MTD mechanism is most effective against both attackers, an evaluation against the baseline attacker alone identifies it. If they differ, the most effective MTD mechanism depends on the attacker model.

At every deployment interval, the most effective MTD mechanisms against the APT attacker model change the network's addresses or layout, and those against the baseline attacker change its services. At the default interval, service diversity reduces the number of hosts the baseline attacker compromises by 92 %, but the number the APT attacker model compromises by 36 %.

This dissertation makes the following contributions:

- We build an APT attacker model that any study using MTDSim can run in place of the baseline attacker.[1] Of eight properties an attacker model needs to model an APT attacker, the APT attacker model has one in full and four in part; three recent attacker models have at most one in full and two in part.
- We find that, in MTDSim, the most effective MTD mechanism depends on the attacker model, so an evaluation against a single attacker model is not enough to show how MTD performs against other attackers.

Chapter 2 describes MTD and MTDSim. Chapter 3 reviews how APT attackers behave and how MTD is evaluated, then sets out the eight properties and checks which of them three recent attacker models have. Chapter 4 builds attack profiles from the attack flows, runs each attack profile in MTDSim as the APT attacker model, and defines the metrics for each attacker's behaviour and MTD's performance. Chapter 5 compares the two attackers with no MTD and under MTD, then removes parts of the APT attacker model one at a time to see what each changes. Chapter 6 explains the results and their limits. It answers SQ1 and SQ2 by scoring the APT attacker model on the eight properties, and SQ3 by finding which MTD mechanisms are most effective against each attacker and proposing why. It closes on what the results mean for MTD evaluation. Chapter 7 answers the research question and sets out future work. Appendix A is the original proposal. This dissertation addresses only the first of its four aims.

[1] The APT attacker model and the experiment scripts are publicly available at `https://github.com/marc-la/MTDSim`.

CHAPTER 2

Background

The APT attacker model is executed inside MTDSim, the MTD simulator this dissertation inherits, and compared with the simulator's baseline attacker. Section 2.1 defines MTD by three questions: what to move, how to move and when to move. Section 2.2 describes the simulator's three modules: the network, the MTD mechanisms and the baseline attacker.

2.1 Moving target defence

Moving target defence (MTD) is the timely change of the system configurations an attacker relies on [5]. An MTD is designed by answering three questions: "what to move", "how to move" and "when to move" [5].

What to move is a configurable attribute, such as an IP address, the network topology, an operating system or a piece of software. How to move is one of the three operations of the shuffle–diversity–redundancy taxonomy [11]. *Shuffling* rearranges or randomises existing components, such as addresses or links, so that the attacker's reconnaissance no longer holds. *Diversity* replaces a component with a different implementation of the same function, so that an exploit against one no longer applies. *Redundancy* replicates components, so that service survives a compromise [5]. When to move makes an MTD *proactive*, *reactive* or *hybrid* [5]. A proactive MTD moves on a schedule, and a reactive MTD moves on an intrusion alert. A hybrid MTD adapts its schedule to such alerts, up to a maximum time between moves [5].

2.2 MTDSim

MTDSim simulates an attacker and MTD on one network, to evaluate MTD mechanisms singly and in combination [6]. Every attack action and every deployment takes simulated time, counted in seconds [12]. Figure 2.1 draws the simulator's three modules: the attacker, the network and MTD. MTD deploys its mechanisms at intervals, and each deployment reconfigures part of the network and disrupts the attacker. The attacker observes part of the network and compromises hosts in it. Section 2.2.1 describes the network's hosts, services and vulnerabilities, Section 2.2.2 the MTD mechanisms and deployment strategies, and Section 2.2.3 the baseline attacker.

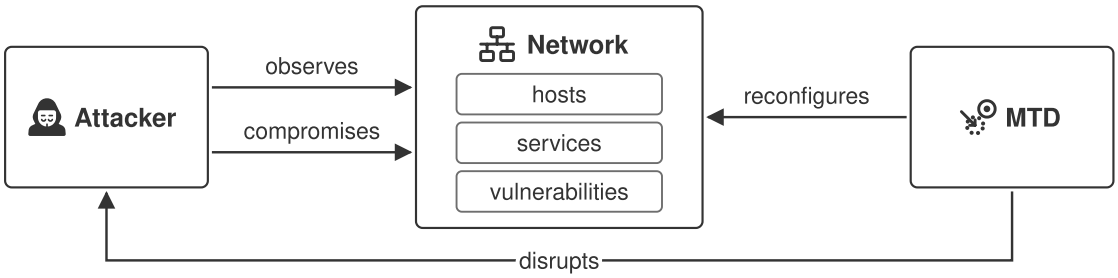


Figure 2.1: The three modules of MTDSim. MTD reconfigures the network and disrupts the attacker. The attacker observes the network and compromises its hosts.

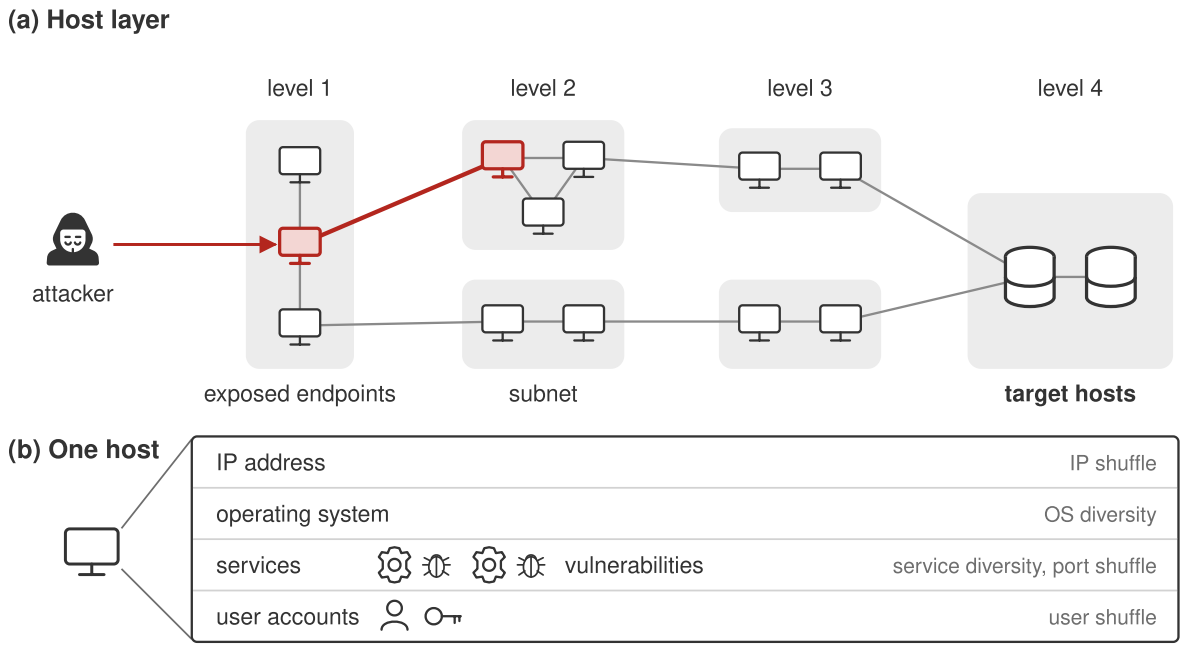


Figure 2.2: The host layer and one host. (a) The hosts in four levels of depth, from the exposed endpoints to the target hosts. Red marks the attacker's route and the hosts it has compromised. (b) The parts of one host, each beside the MTD mechanism that reconfigures it.

2.2.1 Network model

The network in MTDSim is a three-layer hierarchical attack representation model (HARM) [6], [13]. It is a graph of hosts, in which each host runs services and each service carries vulnerabilities (Figure 2.2). The *host layer* is the topology. Its hosts are nodes, and an edge joins two hosts that can reach each other. The *service layer* is the services running on each host.

The hosts sit in levels of depth. The first level is the *exposed endpoints*, the hosts reachable from outside the network. Each deeper level is a set of subnets, small groups of densely linked hosts, joined to the level before it, so reaching a level requires a compromised host in the level before it [6]. By default the network has 50 hosts in four levels of depth and eight subnets, with five exposed endpoints. Five database hosts at the deepest level are the *target hosts* of the targeted attack scenario (Section 2.2.3). Every host other

Table 2.1: The seven MTD mechanisms of MTDSim and what each reconfigures [6].

Mechanism	What it reconfigures
Host layer	
IP shuffle	the IP address of every internal host
Complete topology shuffle	every connection between hosts, regenerated
Host topology shuffle	the positions of internal hosts, swapped in pairs within a level
Service layer	
OS diversity	the operating system of every internal host, and each service the new one cannot run
Service diversity	every service on every internal host, redrawn from those its operating system can run
Port shuffle	the port of every service on every internal host
Credentials	
User shuffle	the user accounts of every internal host, redrawn from the network's pool

than an endpoint is an *internal host*.

Each host has an IP address, user accounts, one operating system and a few services, each running on a port. No MTD mechanism changes the address, software or accounts of the exposed endpoints, although a topology shuffle can change their links. Each vulnerability has an attack complexity from the Common Vulnerability Scoring System (CVSS), which sets the probability that an exploit succeeds, and an impact score for the damage a successful exploit does [6]. A service is compromised when the impact of its exploited vulnerabilities passes a threshold, and a compromised service can compromise its host.

2.2.2 MTD mechanisms and deployment strategies

MTDSim deploys seven MTD mechanisms: five shuffles, two diversities, and no redundancy (Table 2.1). Each reconfigures one part of the network: the host layer, the service layer or the users' credentials. A *deployment* is one run of one mechanism; its duration depends on the mechanism [12].

A *deployment strategy* decides which mechanism is deployed, and when (Table 2.2). The random, alternative and single deployment strategies follow a schedule, so they are proactive [12]. MTDShield is a reinforcement-learning agent [14]. At each interval it reads the network's security metrics and the attacker's current attack action, and deploys one of four mechanisms (complete topology shuffle, IP shuffle, OS diversity or service diversity) or none. If 2 000 s pass without a deployment, one is forced [15]. It adapts its schedule to what it observes within a bound, so it is hybrid. No deployment strategy is reactive.

The *deployment interval* is the time between deployments and is 200 s by default. It is near-periodic. Each interval is the set value plus a random delay of 0.5 s on average [15].

Table 2.2: The three deployment strategies of Zhang [12] and Tay's MTDShield [14].

Deployment strategy	When to move	What it deploys at each interval
random	proactive	a mechanism drawn at random
alternative	proactive	the next mechanism in a fixed rotation
single	proactive	the same mechanism every time
MTDShield	hybrid	the mechanism its agent chooses, or none

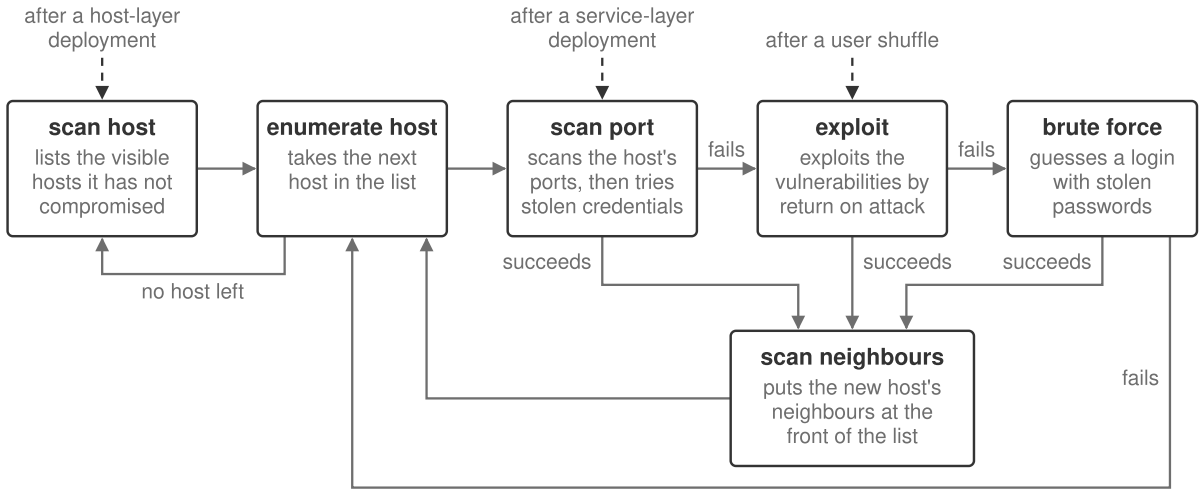


Figure 2.3: The six attack actions of the baseline attacker and the moves between them. A dashed arrow marks the attack action a deployment sends the attacker to.

2.2.3 Baseline attacker

The baseline attacker is the attacker MTDSim was built with [6]. It follows a fixed sequence of six *attack actions*, modelled on the Cyber Kill Chain [6], [16]: it scans for hosts, enumerates one, scans its ports, exploits its vulnerabilities, brute-forces its accounts, and scans the neighbours of each host it compromises (Figure 2.3). The attacker follows one of two attack scenarios (Table 2.3). A run ends when the attacker compromises more than 80 % of the hosts [12], compromises a target host in the targeted scenario, or can discover no new host.

The attacker knows only its *visible subgraph*: the exposed endpoints, the hosts it has compromised and their neighbours. On each host it tries stolen credentials first, then exploits the vulnerabilities in order of *return on attack* [6], and brute force last. Return on attack is an exploit's probability of success times its impact, divided by the time it takes [15]. If all three fail, it moves to the next host on its list. When the list runs out, it scans for hosts again, and the new list holds every host it can reach, including those it has already tried. It gives up on a host after ten attempts, but never on a target host [6]. Scanning for hosts, enumerating a host and scanning a host's neighbours each take 5 s, scanning its ports 25 s and brute force 20 s [15]. An exploit takes 15 s times one minus the vulnerability's attack complexity [15], which lies between 0.4 and 1 [6], so from 0 to 9 s. An exploit of a vulnerability it has exploited before takes half the time [12].

A host-layer deployment disrupts the attacker during any attack action and sends it back to scanning hosts, because the host it was on may have moved. A service-layer

Table 2.3: The two attack scenarios of the baseline attacker and the goal of each [6].

Attack scenario	Goal
General	compromise as many hosts as possible
Targeted	compromise a target host

deployment disrupts it only while it scans ports, exploits or brute-forces, and sends it back to scanning the same host's ports [6], [12]. A user shuffle disrupts only the attacker's credential attacks, and sends it to exploit instead [6]. Each disruption costs the attacker a confusion penalty [12] of about 20 s [15]. A deployment never takes back a compromised host [12] or a stolen credential [15]; the attacker loses only the progress it must repeat, and a topology shuffle also rebuilds its visible subgraph.

The earlier MTDSim studies ran these three modules in the configurations of Table 2.4.

Table 2.4: The configurations the earlier MTDSim studies ran. Times are in seconds; a dash marks a value the study does not state.

Parameter	Brown [6]	Zhang [12]	Ho [17]
Hosts	200	25; 50; 75; 100	150
Levels of depth	5	4	—
Subnets	20	—	—
Exposed endpoints	20	3; 5; 7; 10	—
Attack scenario	general; targeted	general	general
End of run	whole network compromised; target host compromised	80 % of the hosts compromised	time limit of 15 000 s
Deployment interval	uniform on 1–5 s	exponential, mean 50; 100; 150; 200 s	50; 100; 200 s
Runs per configuration	—	100	—

Chapter 3 compares the baseline attacker and the attacker models of recent MTD evaluations with an APT attacker.

CHAPTER 3

Literature review

MTD is evaluated against attacker models, such as MTDSim's baseline attacker (Chapter 2), and the research question asks how it performs against APT attackers. This chapter identifies the research gap between MTD attacker models and APT attackers. Section 3.1 defines the APT attacker and surveys how its behaviour is recovered from CTI (SQ1). Section 3.2 surveys how MTD is evaluated and how attacker models are used in the evaluation (SQ3). Section 3.3 sets out eight properties an MTD attacker model needs to model an APT attacker, and scores three recent MTD attacker models on them (SQ2). Section 3.4 states the research gap and how Chapters 4 and 5 fill it.

3.1 APT attacker behaviour

This section surveys how the behaviour of APT attackers is recovered from CTI. Section 3.1.1 defines the APT attacker. Section 3.1.2 describes MITRE ATT&CK, the knowledge base that records attacker behaviour as techniques. Section 3.1.3 describes attack profiling, which recovers the order of an attack's techniques from CTI.

3.1.1 Advanced persistent threat

An APT attacker is a well-resourced attacker, typically sponsored by a nation-state or organisation, that pursues a specific objective against a specific target [3]. It is *advanced* in its tooling and tradecraft [3]. It is *persistent*. In the definition of the US National Institute of Standards and Technology (NIST), it pursues its objective repeatedly over an extended period and adapts to the defender's resistance [18]. It moves low and slow to avoid detection [3]. Espionage intrusions go undetected for a median of 122 days, against 14 days across all intrusions [1]. And it is a *threat* defined by its objective: data exfiltration, impediment of mission-critical components, or positioning for future operations [3], [18].

This dissertation groups the stages of published lifecycle models of an attack [3], [16], [19] into four: preparation, intrusion, post-intrusion operations and actions on objectives. The models differ in the names and number of their stages but follow a comparable cycle [20]. In preparation, the attacker researches its target and builds its tools; in intrusion, it gains a foothold on a host in the target's network and runs its code there; in post-intrusion operations, it moves through the network undetected; in actions on objectives, it steals data or impedes the victim's systems. Preparation and intrusion are the same for every APT attacker; post-intrusion operations and actions on objectives depend on its objective [3]. An attacker positioning for future operations stays in post-intrusion operations [3].

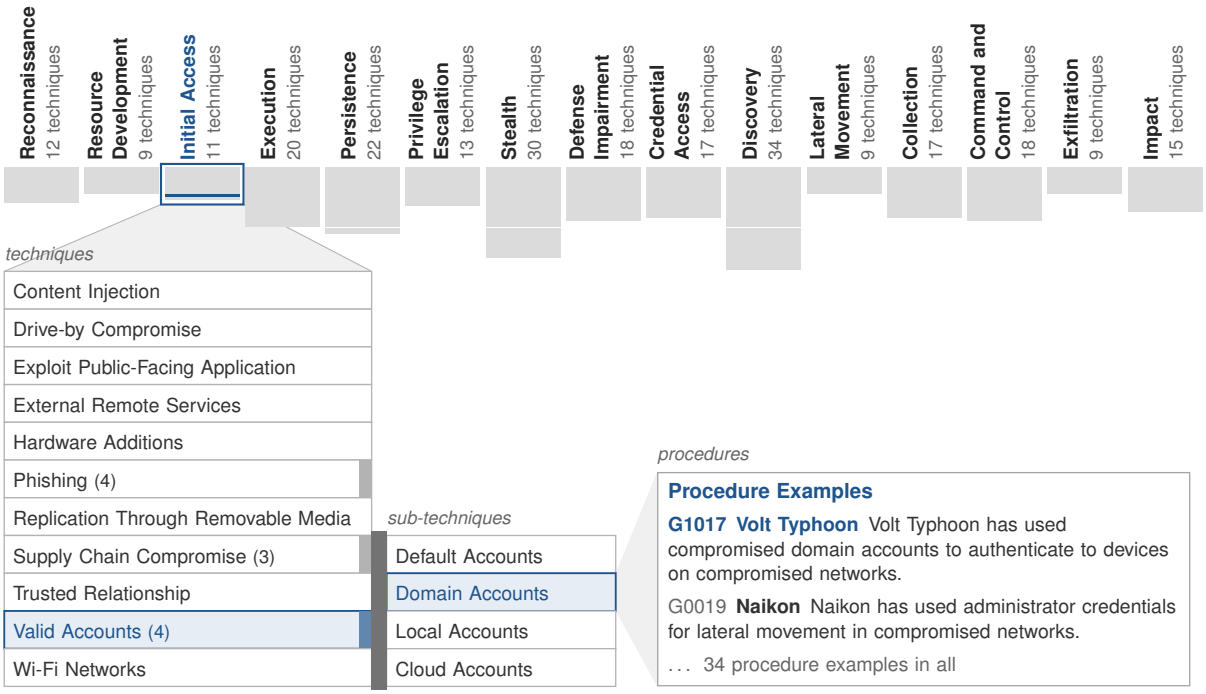


Figure 3.1: The MITRE ATT&CK Enterprise matrix (v19.1), and the technique, sub-technique and Volt Typhoon procedure below its Initial Access tactic [24].

Volt Typhoon, for example, held footholds in US critical infrastructure networks for at least five years, to disrupt their operational technology in a future crisis [2].

3.1.2 MITRE ATT&CK

MITRE ATT&CK is a knowledge base of attacker behaviour derived from real-world observations, developed by the MITRE Corporation in 2013 [21]. It is the common language for attacker behaviour [22], [23]. It holds the attack matrix alongside catalogues of named attacker groups and their campaigns, each group characterised by the techniques observed across its operations [24], [25].

ATT&CK organises behaviour into a hierarchy of tactics, techniques, sub-techniques and procedures [21]. A tactic is what an attacker does to achieve a goal, a technique is how it does it, and a procedure is one recorded instance of a technique [26]. The matrix lays out the top two levels: the 15 tactics as columns, each tactic's techniques beneath it (Figure 3.1). The Initial Access tactic (`TA0001`), for example, is realised through Valid Accounts (`T1078`), whose Domain Accounts sub-technique (`T1078.002`) carries a procedure recording Volt Typhoon's use of compromised domain accounts to authenticate to devices on compromised networks [24].

3.1.3 Attack profiling

Attack profiling recovers the order in which an attacker used ATT&CK techniques to reach its objective [26]. CTI records which techniques an attack used, but rarely their order, and ATT&CK does not record the order either [22]. Attack profiling produces

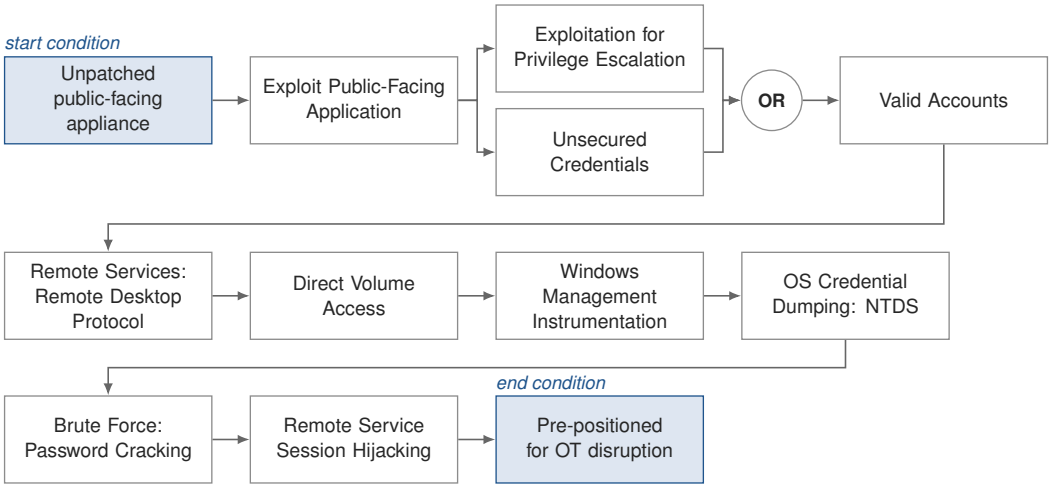


Figure 3.2: An attack flow of the Volt Typhoon campaign [2].

an *attack profile*. An attack profile can be executed as an attacker in a simulation, and Chapter 4 builds its APT attacker model on attack profiles.

Attack profiling is either automated or manual. Automated attack profiling uses language models to recover the order from report text, as ChronoCTI [27] and AttacKG+ [28] do. It scales to many reports, but even identifying which techniques a report describes is not yet reliable. On the 50 most common techniques, the best systems reach 80 % precision at 66 % recall [23]. In manual attack profiling, analysts read a report and draw the order by hand. It is reliable, but covers only the incidents analysts have drawn.

This dissertation uses manual attack profiling in Attack Flow [10]. Attack Flow is a language from MITRE's Center for Threat-Informed Defense (CTID) that records the order of an incident's techniques. An *attack flow* is one incident recorded in Attack Flow, from the condition it starts in to the condition it ends in [10]. CTID publishes a corpus of attack flows that analysts drew from publicly reported incidents [10]. Figure 3.2 shows an attack flow drawn for this dissertation from the Volt Typhoon advisory [2].

Attack profiling recovers the order of an attack's techniques, but not their timing. Attack Flow can record when each technique started and ended, but the attack flows in the corpus almost never do, and ATT&CK dates each campaign only to the month [10], [24]. MTD must therefore decide when to move (Section 2.1) without the attacker's timing. Deploying too often costs service availability, and deploying too rarely gives the attacker time [5].

3.2 MTD evaluation

An MTD evaluation runs MTD against an attacker model and measures how MTD performs. This section surveys the three parts of an MTD evaluation that use the attacker model [5]. Section 3.2.1 covers the MTD modelling approaches, Section 3.2.2 the metrics and Section 3.2.3 the evaluation methods. Section 3.3 then compares MTD attacker models with the APT attacker.

Table 3.1: MTD metrics from the works surveyed and their class in Cho et al. [5].

	Attacker-side	Defender-side
Effectiveness	Attack success probability (ASP) [5], [29], [30], [31]; mean time to compromise (MTTC) [5], [12], [17], [32]; attack path variation, number and exposure [9]; attack surface [33]; attack utility [5]	Defence success probability [5]; attack actions blocked [6]; network compromise ratio (NCR) [12]; risk [17], [34]; mitigation factor [34]; time since last MTD [14], [17]
Efficiency	Attack cost [5], [34]; return on attack (RoA) [34], [35]	Defence cost and system performance overhead [5], [36]; quality of service [5], [36]

3.2.1 MTD modelling approaches

An MTD modelling approach finds an optimal deployment strategy. Cho et al. identify three: game theory, genetic algorithms and machine learning. Game theory dominates by volume [5]. Each represents the attacker differently.

Game theory models the attacker and the defender as rational players, each choosing the strategy that maximises its payoff [5]. Attackers are not necessarily rational. Many aim only to exhaust a system's resources, and a strategy found against a rational attacker may not work against them [5]. Genetic algorithms search over defence configurations for the one with the highest attack resilience and diversity [5]. Their attacker is not an agent: it appears only as the attack resilience the fitness function scores. Machine learning learns a deployment strategy from the observed network state [5]. Its attacker is the attacker model its training environment simulates [14], [17]. MTDShield was trained against the baseline attacker (Section 2.2.3) [14].

The attacker model therefore decides which deployment strategy is optimal. Migrating a service across diverse platforms, for example, defends against persistent attacks but can weaken the defence against fast ones [5]. An APT attacker is persistent (Section 3.1.1), so a deployment strategy found against another attacker may not be optimal against it.

3.2.2 MTD metrics

An MTD metric measures how MTD performs. Cho et al. classify MTD metrics as effectiveness or efficiency metrics, each either attacker-side or defender-side [5]. Table 3.1 gives examples of each. Effectiveness metrics measure how well MTD stops the attacker. Attacker-side effectiveness metrics measure the attacker's progress, such as attack success probability (ASP) and mean time to compromise (MTTC) [5]. ASP is the dominant effectiveness metric [5]. Defender-side effectiveness metrics measure the network's security, such as network compromise ratio (NCR) [12]. Efficiency metrics measure the cost of an attack or of MTD [5]. No standard set of metrics exists [5], [8]. Each evaluation chooses its own, so MTD mechanisms cannot be compared across evaluations [8].

ASP, MTTC and NCR are all computed from what the attacker does, so their values depend on how the attacker is implemented. MTD that is effective against one attacker may therefore not be effective against an APT attacker [9].

3.2.3 MTD evaluation methods

An MTD evaluation method runs MTD against an attacker model. Cho et al. name four: analytical models, simulations, emulations and real testbeds [5]. An analytical model computes the outcome from a probabilistic or graph model of the network, the attacker and MTD. A simulation runs a program that models them over simulated time. An emulation runs real software on a virtual network, and a real testbed runs on physical hardware. Analytical models are the cheapest to evaluate but the most abstract [5]. Emulations and real testbeds are the most realistic but hard to scale to large networks [5].

Simulation is the dominant evaluation method, because it models specific attack behaviours more flexibly than an analytical model [5]. Simulation results do not transfer directly to a real network, which no simulation fully captures [5]. This dissertation uses MTDSim's flexibility to run an APT attacker model through its existing attack actions (Chapter 4).

3.3 MTD attacker models

Two surveys of MTD find that MTD attacker models lack properties of an APT attacker [5], [8]. This section checks whether recent MTD attacker models still lack them. Section 3.3.1 sets out eight properties an MTD attacker model needs to model an APT attacker, and Section 3.3.2 scores three recent MTD attacker models on them.

3.3.1 Properties of an APT attacker

An MTD attacker model needs eight properties to model an APT attacker. The APT definition of Section 3.1.1 gives four: the APT attacker pursues its objective over an extended period (campaign persistence), uses different techniques after the foothold for different objectives (objective-specific behaviour), adapts to the defender's resistance (adaptivity) and moves low and slow to avoid detection (detection avoidance) [3], [18]. The two surveys add the other four: choice of attack strategy, cost–benefit reasoning, learning and deployment-strategy awareness [5], [8]. This dissertation synthesises the eight properties in Table 3.2.

Cho et al. name three limitations of MTD attacker models [5]. First, they lack adaptivity and learning. The attacker follows fixed attack patterns while the defender adapts and learns, although in practice the attacker is often the smarter of the two. Second, they lack choice of attack strategy. Few evaluations let the attacker choose among several attack strategies. Third, they lack cost–benefit reasoning and deployment-strategy awareness. The attacker is seldom modelled as a rational actor, or as one that exploits how the MTD works. Jalowski et al. call the attacker model "the most glaring flaw in the MTD literature" [8, p. 8]. They find the same lack of learning and deployment-strategy awareness. An APT attacker learns the MTD's deployment patterns over time and looks for the logic of the deployment strategy [8]. They add detection avoidance. MTD is typically tested against active scanning, such as Nmap, while an APT attacker uses passive reconnaissance to stay hidden [8].

Table 3.2: The eight properties of an APT attacker and their sources.

	Property	Definition	Source
1	Campaign persistence	pursues one multi-stage campaign toward its objective over an extended period	[3], [18]
2	Objective-specific behaviour	after the foothold, uses different techniques for different objectives, such as exfiltration or impediment	[3]
3	Choice of attack strategy	chooses among several sequences of techniques toward the same objective	[5]
4	Adaptivity	chooses its next technique in response to an MTD disruption	[5], [18]
5	Detection avoidance	moves low and slow and chooses techniques an intrusion detection system (IDS) does not flag, such as passive reconnaissance	[3], [8]
6	Cost–benefit reasoning	chooses each technique by weighing its cost against its benefit	[5]
7	Learning	keeps the patterns it observes in past MTD deployments, and uses them to choose later techniques	[5], [8]
8	Deployment-strategy awareness	predicts and leverages which mechanism the deployment strategy deploys next, and when	[5], [8]

Table 3.3: Three MTD attacker models from 2023 to 2026, scored by this dissertation on the eight properties of Table 3.2. ✓ has the property, P has it in part, ✗ does not have it.

Attacker model	Year	1 Pers.	2 Obj.	3 Strat.	4 Adapt.	5 Detect.	6 Cost	7 Learn.	8 Deploy.
Baseline attacker [6], [12]	2023	✓	✗	P	P	✗	✗	✗	✗
Masud et al. [31]	2025	P	✗	✗	✗	✗	✗	✗	✗
Kim et al. [30]	2026	P	✗	✗	✗	P	✗	✗	✗

3.3.2 Recent MTD attacker models

This section scores three MTD attacker models, published from 2023 to 2026, on the eight properties an MTD attacker model needs to model an APT attacker (Table 3.2). The first is MTDSim's baseline attacker, described in Section 2.2.3 [6], [12]. The other two are the attacker models of two recent MTD evaluations that describe their attacker by the Cyber Kill Chain or ATT&CK, by Masud et al. [31] and Kim et al. [30]. This dissertation chose them because it expects an attacker model described this way to be the most likely to have the properties. This dissertation scores each attacker model on each property from its paper with one of three marks, by predicates taken from each definition in Table 3.2 (Appendix B). A behaviour predicate states what the attacker model does, and a qualifier predicate states a condition on it that can be checked apart. A ✓ means every predicate is true of the attacker model's own behaviour. A P means a behaviour predicate is true, but not every predicate is true of its own behaviour. At least one predicate is false, or true only because its authors assume the behaviour or set it by a fixed rule, such as always re-scanning after a host-layer disruption. A ✗ means no behaviour predicate is true. Table 3.3 shows each attacker model's mark on each property, and the paragraph on each attacker model below gives the evidence from its paper for each mark.

Baseline attacker. The baseline attacker has campaign persistence: it repeats its six attack actions host by host until it reaches its goal, and carries on through every disruption [6]. It has choice of attack strategy in part: it can enter a host with stolen credentials, an exploit or brute force, but a fixed rule sets the order in which it tries them. It has adaptivity in part: a fixed rule sets its response to each kind of disruption, such as always re-scanning after a host-layer disruption [6]. It has no cost–benefit reasoning: it orders the exploits on a host by return on attack (RoA), but that chooses which vulnerability to exploit, not which technique. It has no objective-specific behaviour: after a foothold it takes the same six attack actions in either attack scenario, which changes only the host it attacks next and whether it gives up on it, and none of them exfiltrates or impedes. It has no learning: an exploit of a vulnerability it has exploited before takes half the time [12], but it keeps vulnerabilities, not the patterns of MTD deployments, and the memory changes no choice. It has no detection avoidance: it scans actively and does nothing to avoid an intrusion detection system (IDS). It has no deployment-strategy awareness: nothing in it uses which mechanism deploys next or when it deploys.

Masud et al. Masud et al. use an analytical model to evaluate an MTD that combines shuffle, diversity and redundancy on an Internet-of-Things cloud network [31]. Their attacker model is the set of attack paths through a graph model of that network, from the Internet to a target database. At each deployment interval, the analytical model computes each attack path's success probability, cost and RoA, before and after the MTD deploys. It has campaign persistence in part: each attack path is a multi-stage chain of exploits, but each deployment interval's attack paths are computed afresh, so no campaign spans intervals. It has no choice of attack strategy: it has every attack path to the database, but chooses none. It has no adaptivity: its attack paths are recomputed after each MTD deployment, but no next technique is chosen from them. It has no cost–benefit reasoning: it computes each attack path's cost and RoA, but no choice depends on them. The paper also says its attackers prioritise vulnerabilities by RoA and adapt to new flaws [31, p. 7], but no step of the analytical model does either. It has no objective-specific behaviour: every attack path leads to one objective, the database, and nothing follows once it reaches it. It has no detection avoidance: the analytical model has no IDS for an attack path to avoid. It has no learning: each deployment interval's attack paths are computed afresh, so nothing from earlier MTD deployments is kept. It has no deployment-strategy awareness: nothing in the analytical model uses which mechanism deploys next or when it deploys.

Kim et al. Kim et al. use a real testbed, a software-defined network on physical hardware, to evaluate an MTD that combines virtual IP shuffling, software diversity and redundancy [30]. Their attacker model is an automated attack script that runs four tasks in a fixed order: it scans for web servers, collects the version of each web service it finds, exploits one known vulnerability of the Apache web server, and keeps a reverse shell. A reverse shell is an outbound connection from the compromised server back to the attacker. Each attack is a new run of the script, started at a random time. It has campaign persistence in part: each attack runs all four tasks, but in one short attempt, and the next attack starts again with no knowledge of the network. It has detection avoidance in part: the reverse shell is chosen "in order to evade the detection by a defense system" [30, p. 7], but the testbed runs no IDS, so the evasion is assumed. It has no objective-specific behaviour: after the foothold it keeps its shell and installs tools, but it has one

objective, so no technique differs by objective, and none exfiltrates or impedes. It has no deployment-strategy awareness: the attacker "can determine the remaining available time" before the next MTD deployment [30, p. 10]. Every mechanism deploys at once, so the attacker predicts which mechanism deploys next and when, but no technique leverages the prediction. It has no choice of attack strategy: every attack follows one attack scenario, in the same four tasks. It has no adaptivity: the script does not observe the MTD, and an attack that a deployment disrupts fails. It has no cost–benefit reasoning: no technique's cost is weighed against its benefit. It has no learning: the script observes no MTD deployment, and nothing is kept from one attack to the next.

Finding. The three MTD attacker models still lack the properties of an APT attacker. Only the baseline attacker has one in full, campaign persistence. None of the three has objective-specific behaviour, cost–benefit reasoning, learning or deployment-strategy awareness. Only Kim et al.'s has detection avoidance in part. This dissertation expected the attacker models of Masud et al. and Kim et al. to be the most likely to have the properties. They have none in full.

3.4 Research gap

An MTD evaluation's result depends on its attacker model: the attacker model decides which deployment strategy is optimal (Section 3.2.1) and how effective MTD is (Section 3.2.2). Two surveys find that MTD attacker models lack properties of an APT attacker [5], [8], and the three recent MTD attacker models scored in Section 3.3.2 still lack them. How MTD performs against APT attackers therefore remains unmeasured. This dissertation recovers the behaviour of APT attackers from CTI by attack profiling (Section 3.1.3) and executes it as an APT attacker model inside MTDSim (Chapter 4). It then measures how MTD performs against the APT attacker model compared with the baseline attacker (Chapter 5).

CHAPTER 4

APT attacker model

The MTD evaluations surveyed in Chapter 3 leave how MTD performs against APT attackers unmeasured, because their attacker models lack properties of an APT attacker. This chapter defines an APT attacker model built for three of the eight properties of Section 3.3.1: objective-specific behaviour, choice of attack strategy and adaptivity. It takes the same attack actions as the baseline attacker, but the Petri net of its attack profile chooses its next tactic at runtime. With those attack actions it keeps the baseline attacker's campaign persistence; they also cap what it gains, so it has one property in full and four in part (Section 6.3). It is a proof of concept that tests whether an attacker with more of an APT's properties changes what an MTD evaluation measures. It avoids detection only through its tactic durations, which make it slower than the baseline attacker. Figure 4.1 shows the pipeline that builds the model from 38 attack flows and runs it in MTDSim. Sections 4.1 and 4.2 answer SQ1: they recover the behaviour of APT attackers from CTI as an attack graph and four attack profiles. Sections 4.3 and 4.4 answer SQ2: they execute each attack profile as a Petri net inside MTDSim. Section 4.5 defines the metrics with which Chapter 5 answers SQ3.

4.1 Attack graph from attack flows

The attack graph aggregates the 38 attack flows into one weighted directed graph whose nodes are the 15 tactics of ATT&CK v19.1 [37]. Each technique in an attack flow counts as its tactic, and a technique listed under several tactics counts as the leftmost of them in the ATT&CK matrix. An edge runs from tactic p to tactic q wherever an attack flow steps from a technique of p to a technique of q, and its weight is the number of attack flows that draw it (Figure 4.2). A step between two techniques of the same tactic is not an edge because the time an attacker spends within a tactic is its tactic duration (Section 4.4.2).

We built the attack graph from attack flows because they come from manual attack profiling. An analyst drew the order of every step. We also tried two automatic alternatives. Co-occurrence mining pairs techniques that the same APT group or tool uses; keyword mining pairs a technique with each technique its ATT&CK description names. Neither pairing records which technique came first, so both had to assume the order of the tactics. They recovered 10 % and 14 % of the possible tactic-to-tactic edges, against 50 % for the attack flows. The corpus is not limited to APT attackers. Only 18 of the 38 attack flows are attributed to a group in ATT&CK, criminal groups among them, and the other 20 name no group. One is an adware campaign. We keep all 38, because dropping the 20 would halve an already sparse corpus. The attack graph therefore records attackers reported in CTI, not APT attackers alone (Section 6.5).

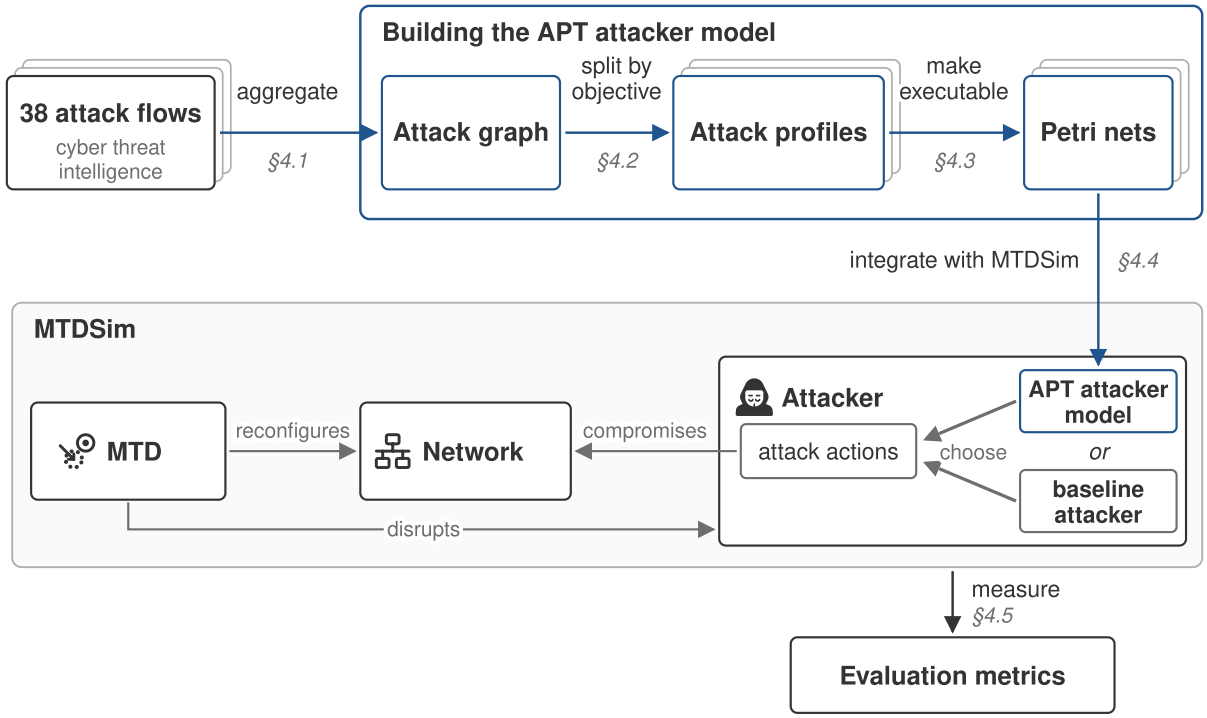


Figure 4.1: The pipeline that builds the APT attacker model from 38 attack flows and runs it in MTDSim. Blue marks what this dissertation builds.

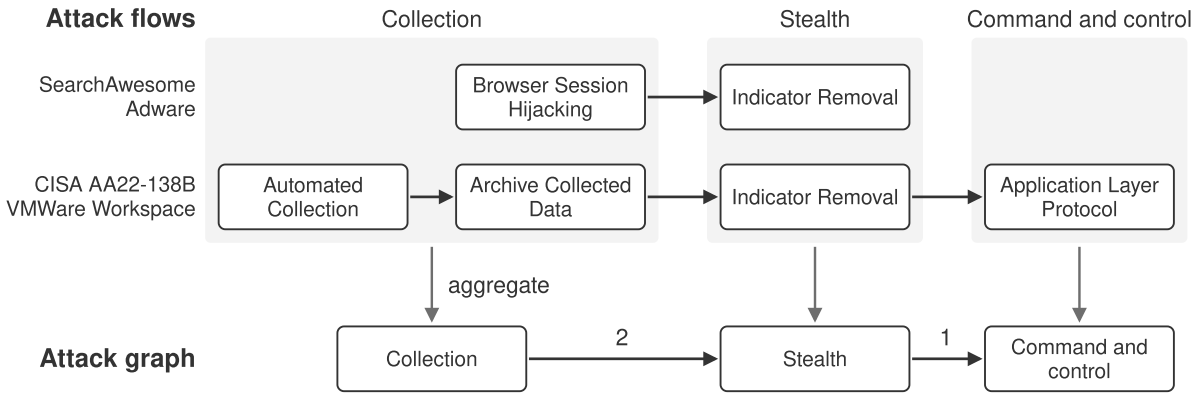


Figure 4.2: Part of two attack flows aggregated into the attack graph. A number on an edge counts the attack flows that draw it.

We chose tactics as the nodes rather than techniques, trading resolution for edges that recur across attack flows. Most edges between techniques (88 %) come from one attack flow alone, against 39 % of the edges between tactics (Appendix C.1). An edge weight needs several attack flows behind it, because Section 4.3 converts the weights into the probability of each next tactic. Tactics also make the tactic-to-action mapping smaller. It maps 15 tactics, rather than the 124 techniques the attack flows use, to MTDSim's six attack actions (Section 4.4).

The attack graph is sparsest before Initial Access. Reconnaissance appears in 10 of the 38 attack flows. Reconnaissance and Resource Development happen largely outside the scope of the defender's controls [24], so the investigations that CTI reports rarely see them. Any CTI corpus shares the gap. An MTD evaluation cannot ignore it, because

shuffling works by making the attacker's reconnaissance out of date. Section 4.3 adds the missing edges as the pre-intrusion overlay.

4.2 Attack profiles by objective

APT attackers with different objectives act differently after the foothold (Section 3.1.1). The attack graph of Section 4.1 is built from all 38 attack flows, so it averages the behaviour of attackers with different objectives. We therefore split it by objective into four attack profiles, c_1 to c_4. Each attack profile is a subgraph of the attack graph, aggregated from its own attack flows only.

Section 3.1.1 gives three objectives. Data exfiltration and impediment each have an action in actions on objectives: the attacker steals data, or it impedes the victim's systems by encrypting, destroying or hijacking them. Positioning for future operations has no action of its own, because the attacker stays in post-intrusion operations. We therefore asked two questions of each attack flow: did the attacker steal data, and did it impede the victim's systems? Their yes-or-no answers give the four attack profiles of Table 4.1. An attack flow that did both belongs to c_3 alone, so no edge weight counts it twice. Six of the seven attack flows of c_3 are ransomware that stole data and then encrypted the systems. The five attack flows of c_4 did neither, so c_4 is a mixed group, not a single objective: three attackers were removed from the network before they acted, one positioned for future operations, and one ran an adware campaign.

Table 4.1: The four attack profiles.

Attack profile	Stole data	Impeded systems	Attack flows	Example
c_1	yes	no	19	SolarWinds
c_2	no	yes	7	NotPetya
c_3	yes	yes	7	Conti Ransomware
c_4	no	no	5	Hancitor DLL

We answered both questions from the CTID description of each attack flow and, where that was not enough, its ATT&CK group page or the threat report it cites. Three attack flows stayed open after these sources, and we decided them ourselves (Appendix C.2). We did not read the answers off the attack flow, because its analyst drew only the steps the threat report describes, and these can stop short of the objective. Only 10 of the 19 attack flows of c_1 reach the Exfiltration tactic, although the sources of all 19 report the theft (Figure 4.3).

Figure 4.3 groups the tactics by the four stages of Section 3.1.1. Before actions on objectives, the attack profiles share their most common tactics: Execution, Stealth, Discovery and Command and Control appear in more than half the attack flows of every attack profile. The attack profiles differ in actions on objectives. No attack flow of c_2 or c_4 reaches Exfiltration, and none of c_4 reaches Impact. Collection, which neither question asks about, appears in 14 of the 19 attack flows of c_1 but in only one each of c_2 and c_3.

Section 4.3 formalises each attack profile as a Petri net, whose attacker chooses its next tactic only among the edges its attack flows drew. Two attack profiles that reach the same tactics can therefore move between them differently (Appendix C.3). Section 5.4

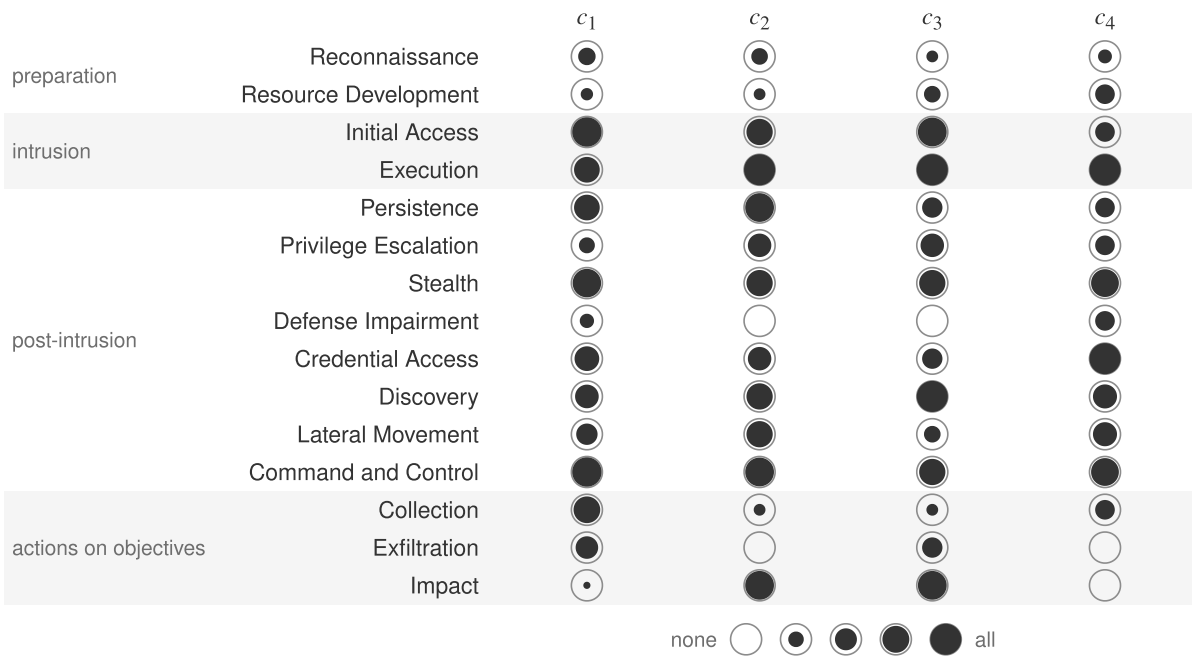


Figure 4.3: The share of the attack flows of each attack profile (Table 4.1) that reach each tactic.

ablates the split. It runs the APT attacker model on the whole attack graph of Section 4.1 and compares the results with those of the four attack profiles.

4.3 Petri-net formalism

Stochastic Petri nets are an established formalism in MTD evaluation [36], [38]. Cai et al. compare MTD mechanisms with the generalised stochastic Petri net (GSPN) [39]. We follow this convention and formalise each attack profile as a GSPN. An attack profile gives only the edges between tactics and their weights. A GSPN adds a token and two kinds of transition. The token marks the tactic the attacker is on. A timed transition models the tactic duration, and an immediate transition models the attacker's choice of the next tactic.

A GSPN is a directed bipartite graph of places and transitions [40]. Places hold tokens. A transition is enabled when each of its input places holds a token. Firing it moves a token from each input place to each output place. A *timed* transition fires after a delay drawn from an exponential distribution; its *rate* is one over the mean delay. An *immediate* transition fires in zero time and takes priority over timed ones. Of several enabled immediate transitions, one fires with probability proportional to its weight. Following Ajmone Marsan et al. [41], a GSPN is the tuple

$$\mathcal{N} = (P, T, I, O, W, M_0), \tag{4.1}$$

where P is the set of places; T the set of transitions, each timed or immediate; I and O the input and output arcs; W the rate of each timed transition and the weight of each immediate one; and M_0 the initial marking, the number of tokens on each place. The full definition adds inhibitor arcs and priority levels, which this dissertation does not use.

Each attack profile c (Section 4.2) gives a Petri net $\mathcal{N}_c$, built as follows. Only its places and weights depend on c. Figure 4.4(a) draws the construction for one tactic p.

Places P: one place p for each tactic of the attack profile and, for each, a decision place $\hat{p}$ at which the next tactic is chosen.

Transitions T: a timed transition τ_p from each tactic p to its decision place $\hat{p}$, whose firing is the attacker carrying out the tactic; and an immediate transition t_{pq} from $\hat{p}$ to q for each edge $p \to q$ of the attack profile with a base weight above zero (Equation 4.2). No transition leads from a tactic back to itself because the tactic duration is the delay of τ_p.

Arcs I, O: those of the chain $p \to \tau_p \to \hat{p} \to t_{pq} \to q$.

Weights W: each timed transition has rate $W(\tau_p) = 1/\mu_p$, so that its delay has mean μ_p, the mean duration of tactic p (Section 4.4.2). Resource Development has a mean duration of zero, so its transition is immediate. Each immediate transition has the *base weight* of its edge $p \to q$, the share of the attack profile's attack flows leaving p that go to q:

$$W(t_{pq}) = \frac{n(p,q)}{\sum_{q'} n(p,q')}, \tag{4.2}$$

where $n(p,q)$ is the number of the attack profile's attack flows that draw $p \to q$. The base weights count each APT group once, through its attack flow with the most steps, so that a group reported several times does not outweigh the rest. They rest on 29 of the 38 attack flows.

Initial marking M_0: one token on Reconnaissance. The single token means the attacker pursues one line of the campaign at a time. Where an attack flow draws parallel branches, the token follows one of them.

In c_3 and c_4, no attack flow leads on from Reconnaissance, so the token could not leave the place it starts on. For these two attack profiles we add the *pre-intrusion overlay*. It consists of three immediate transitions that none of their attack flows draws: from Reconnaissance to Resource Development, from Resource Development to Initial Access, and from Initial Access back to Reconnaissance, for an attacker whose intrusion fails. Their weights are declared. A place with no transition of its own sends all its weight through the overlay. A place with transitions of its own gives the overlay transition one-tenth of its weight, and its own transitions share the other nine-tenths in their proportions.

The *verdict* of each attack action the APT attacker model runs in MTDSim is success or failure (Section 4.4). The base weights describe where an attacker goes after a success, because an attack flow draws only the steps its attacker completed (Section 4.4.3). A GSPN cannot route on the verdict. Its weights are constants, or functions of the marking, and the verdict comes from MTDSim, outside the Petri net.

We therefore extend the GSPN with one element, the failure matrix F. It holds a declared factor between zero and one for each pair of tactics (Section 4.4.3). The Petri net of attack profile c becomes the tuple

$$(P, T, I, O, W, M_0, F), \tag{4.3}$$

whose first six elements are those of $\mathcal{N}_c$. When τ_p fires and its attack action fails, F scales the weights of the immediate transitions out of $\hat{p}$:

$$W(t_{pq} \mid \text{failure}) = \frac{W(t_{pq})\, F(p,q)}{\sum_{q'} W(t_{pq'})\, F(p,q')}. \tag{4.4}$$

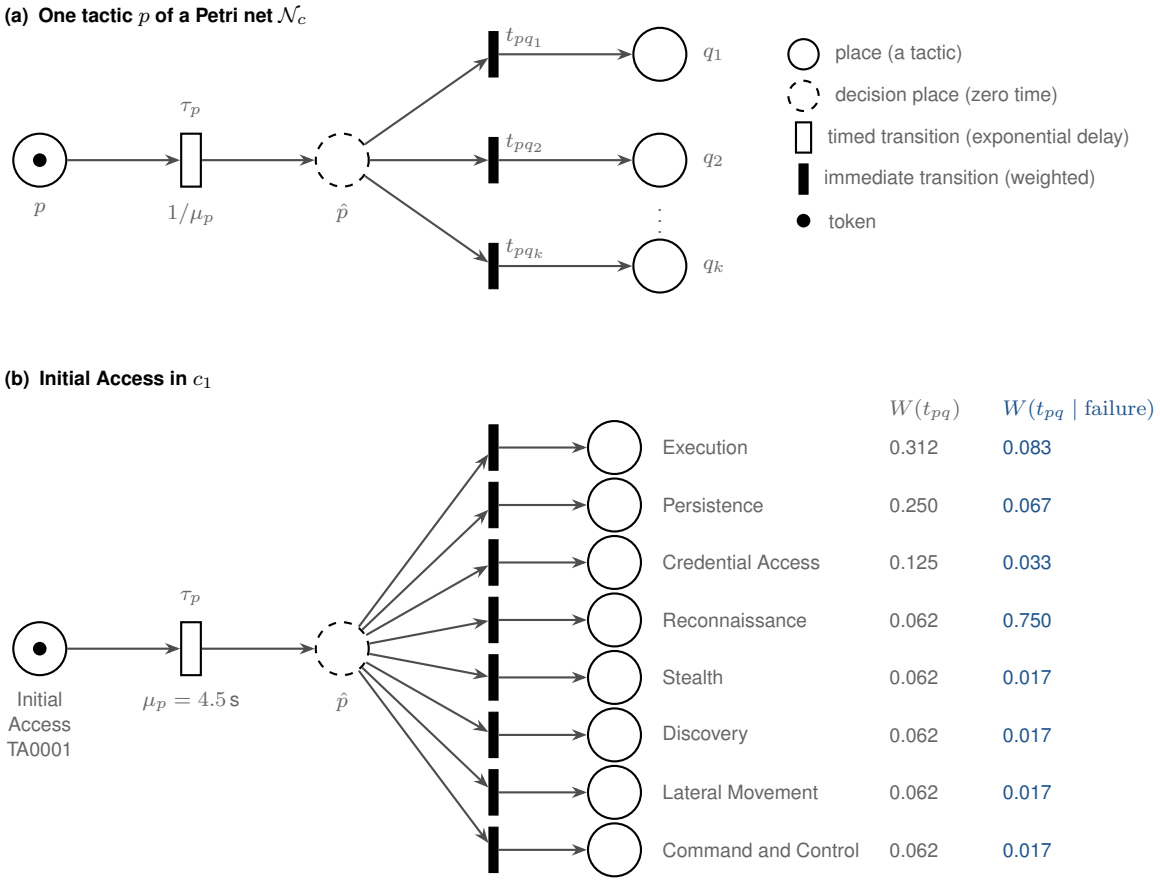


Figure 4.4: One tactic of a Petri net. (a) The construction for one tactic p. (b) Each successor q of Initial Access in c_1 with its base weight $W(t_{pq})$ and, in blue, its weight after a failure $W(t_{pq} \mid \text{failure})$ (Equations 4.2 and 4.4).

After a success, or at a tactic that runs no attack action, the immediate transitions keep their base weights. A factor of zero disables a transition for that firing only; places, transitions and arcs never change. An MTD deployment can also cut a timed transition short; if an attack action was in flight, its verdict is failure (Section 4.4). Figure 4.4(b) gives both sets of weights at Initial Access in c_1.

4.4 Integration with MTDSim

The Petri net of an attack profile chooses the attacker's next tactic, but it cannot carry the tactic out. MTDSim's attack actions carry tactics out on the network, and each returns a verdict. The APT attacker model integrates the two in one loop: the tactic the token reaches runs as one of MTDSim's attack actions, and the attack action's verdict selects the weights of the immediate transitions that take the token to the next tactic.

One iteration of the loop runs as follows, numbered as in Figure 4.5:

1. the token's tactic leaves the Petri net;
2. the tactic-to-action mapping maps the tactic onto one of MTDSim's attack actions;
3. the attack action runs for the tactic duration, the delay of the tactic's timed transition, in place of its own time in MTDSim;

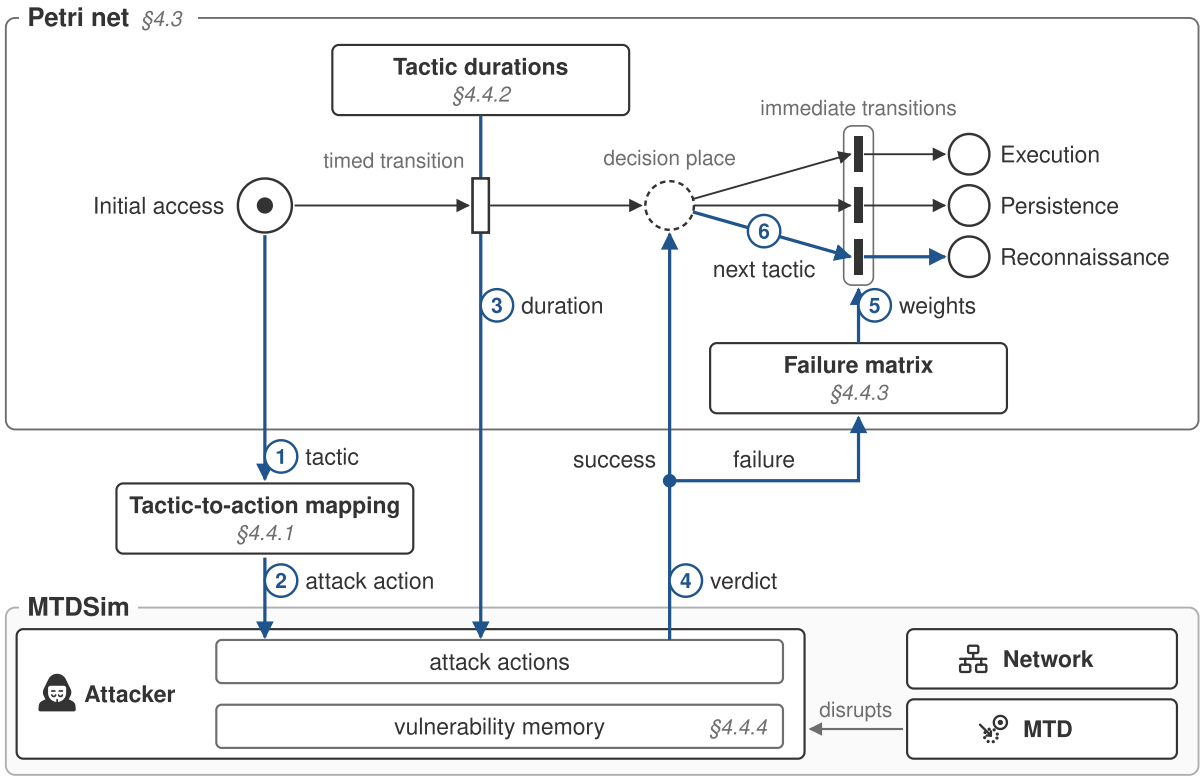


Figure 4.5: The Petri net integrated with MTDSim, drawn at Initial Access in c_1 with three of its eight successors. Blue traces one iteration of the loop, numbered in order.

4. the attack action's verdict, success or failure, returns from MTDSim;
5. after a failure, the failure matrix reweights the immediate transitions (Equation 4.4); after a success, they keep their base weights;
6. the token fires one immediate transition to the next tactic, and the loop repeats.

An attack action fails if its preconditions were missing (for example, an exploit of a vulnerability not yet scanned for), or if an MTD deployment disrupts it (Section 2.2.3).

The attack flows cannot supply three inputs of the loop: the tactic-to-action mapping (Section 4.4.1), the tactic durations (Section 4.4.2) and the failure matrix (Section 4.4.3). We declare each, from MTDSim's constants where they exist and otherwise from the literature or our judgement. The vulnerability memory (Section 4.4.4) extends MTDSim's exploit attack action. Each earlier success on a vulnerability raises the attacker's odds of exploiting it again.

4.4.1 Tactic-to-action mapping

The *tactic-to-action mapping* maps the token's tactic onto one of MTDSim's attack actions (2 in Figure 4.5). Figure 4.6 shows the attack action each of the 15 tactics maps to, if any. We declared each pair from ATT&CK, in which a tactic is what an attacker does to achieve a goal and its techniques are how it does it (Section 3.1.2). A tactic maps to the attack action that carries out one of its techniques. Credential Access, for example, includes guessing passwords by brute force, which is what MTDSim's brute force attack action does. Initial Access, Execution and Privilege Escalation each include a technique

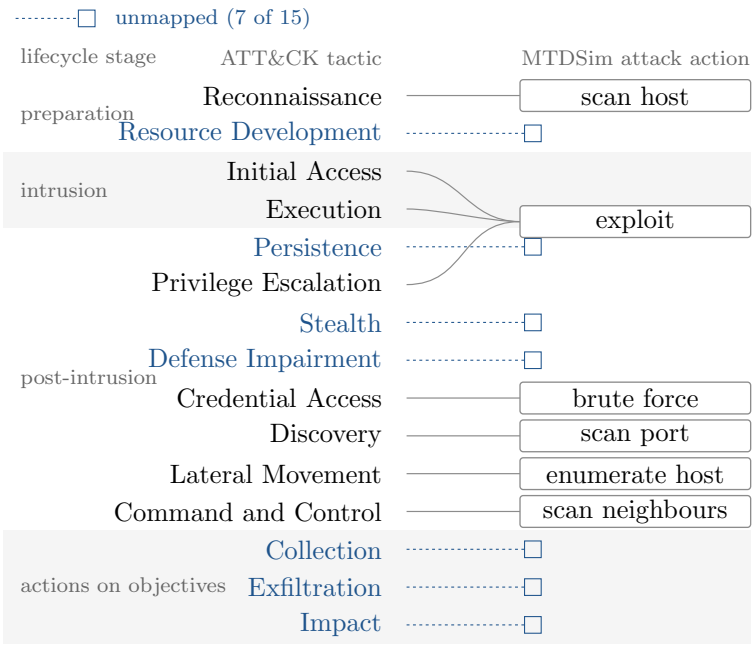


Figure 4.6: The tactic-to-action mapping. The tactics are grouped by the four lifecycle stages of Section 3.1.1.

that exploits a vulnerability, so all three map to exploit. Reconnaissance and Discovery each include a scanning technique, so Reconnaissance maps to scan host and Discovery to scan port. For the remaining two, Lateral Movement and Command and Control, we chose the attack action that serves the tactic's goal. Enumerate host moves the attacker to its next host, which is the goal of Lateral Movement, and Brown describes scan neighbours as Command and Control revealing connected hosts [6].

Seven tactics are *unmapped*, because no attack action in MTDSim carries them out. MTDSim's attacker stops at compromising hosts, so Collection, Exfiltration and Impact have nothing to act on. Nothing in MTDSim watches the attacker, so Stealth has nothing to hide from. The attacker can neither see nor change the MTD schedule, so Defense Impairment has nothing to impair. A compromised host stays compromised, so Persistence has no foothold to keep. Resource Development happens off the target's network, which MTDSim does not model. These gaps set the ceiling of this dissertation. The APT attacker model can only be as good as the attack actions it adopts.

An unmapped tactic runs no attack action, so it cannot fail. An MTD deployment during it still cuts its duration short and costs the attacker the confusion penalty, and a host-layer deployment also costs it the host it was on (Section 2.2.3), but no deployment changes which tactic comes next.

4.4.2 Tactic durations

The *tactic durations* set how long each tactic takes (3 in Figure 4.5). They set how often the APT attacker model acts. The longer a tactic runs, the more likely an MTD deployment disrupts it. A tactic's duration is random each time it runs, because attack behaviour is stochastic [42], [43], [44], and an exponential delay, because the timed transitions of a GSPN fire after exponential delays (Section 4.3). An exponential delay has

one parameter, its mean, so each tactic needs one value, its *mean duration* μ_p.

MTDSim times only its attack actions, and seven tactics are unmapped (Section 4.4.1), so each tactic has a mean duration of its own. Table 4.2 gives each mean duration and its source: MTDSim for six tactics, and our judgement for the other nine.

Table 4.2: The mean duration of each tactic, and its source.

Tactics	Source	Mean duration μ_p (s)
Initial Access, Privilege Escalation, Credential Access, Lateral Movement	Mean time of MTDSim's exploit attack action	4.5
Reconnaissance, Discovery	Total time of MTDSim's scans for hosts, ports and neighbours ($5 + 25 + 5$ s)	35.0
Persistence, Stealth, Command and Control	Judgement	45.0
Execution, Defense Impairment	Judgement	22.5
Collection, Exfiltration, Impact	Judgement	36.0
Resource Development	Judgement	0.0

Six tactics do the same kind of work as one of MTDSim's attack actions, so each takes that attack action's time, even where the tactic maps to another (Section 2.2.3). Initial Access, Privilege Escalation, Credential Access and Lateral Movement each act briefly on one host, as the exploit attack action does, so each takes the mean time of the exploit attack action, 4.5 s. Reconnaissance and Discovery survey a network, as MTDSim's scans for hosts, ports and neighbours do together, so each takes the total time of the three scans, 35 s.

No source we found times the other nine tactics, since threat reports time whole intrusions. We set their mean durations by judgement, from what each tactic does in an attack, as other models that time attack steps do where data are missing [32], [44]. Chapter 5 compares the APT attacker model with the baseline attacker under the same MTD deployments, so we kept the nine on the scale of the baseline attacker's attack actions (Section 2.2.3). We also kept each under the 200 s default deployment interval (Section 2.2.2), or a deployment would disrupt most runs of the tactic.

Persistence, Stealth and Command and Control keep a foothold, stay hidden and stay in contact throughout an attack [24], so we set them longest, at 45 s. Collection, Exfiltration and Impact move or destroy data, which we judged to take a little longer than surveying a network, so we set them at 36 s. Execution and Defense Impairment are single acts, so we set them at half the 45 s, 22.5 s, still longer than an exploit because the attacker slows them to stay hidden. Resource Development happens before the attack reaches the target's network, so it takes no time. Because these values are our judgement, Appendix D.4 tests whether NCR (Section 4.5) depends on them. After the values were fixed, it moved each mean duration in turn to both ends of a band, from half to double, and from a quarter to four times for the 45 s. Only the 45 s changes NCR much, most likely through Command and Control. Command and Control is the only tactic that maps to scan neighbours, the attack action that finds the hosts next to those the attacker has compromised (Section 4.4.1). The slower the attacker finds new hosts, the fewer it reaches before the run ends, so Chapter 5's NCR rests most on this one value.

4.4.3 Failure matrix

The *failure matrix* reweights the immediate transitions after an attack action fails (5 in Figure 4.5). It holds one factor between zero and one for each pair of tactics, the tactic whose attack action failed and a tactic that can come next, and Equation 4.4 multiplies each base weight by its factor. The attack flows cannot supply it. The threat reports behind them describe incidents that succeeded and were worth writing about, so the attack flows draw only the steps their attackers completed. The base weights therefore say where an attacker goes after a success, and nothing about where it goes after a failure.

We set the failure matrix from three assumptions about what an attacker does after a failure. First, an attacker without a foothold (Section 3.1.1) is unlikely to go on to a tactic that needs one, so after a failed Initial Access it is likely to go back to preparation to try again. Hutchins et al. describe an APT campaign that tried again the next day, with new recipients, after its intrusion attempt was detected and mitigated [16]. Second, an attacker that fails after it has a foothold keeps the foothold, so we judged it more likely to try another tactic of the same lifecycle stage than to move to another stage. Third, an attacker seldom skips a lifecycle stage, as in the lifecycle models of Section 3.1.1.

We wrote the assumptions as rules instead of setting the 210 factors one by one. Nine rules give one value to each kind of move the assumptions tell apart, such as a move to another tactic of the same lifecycle stage after a failure with a foothold. The third assumption is also written as a distance factor, which scales a move down by each lifecycle stage it skips. The rules and the distance factor set all 210 factors (Appendix C.5), and their values are our judgement. Equation 4.4 rescales the weights out of a tactic to sum to one, so a factor counts only against the other factors of the same tactic. After a failed Initial Access in c_1, for example, the weight on Reconnaissance rises from 0.062 to 0.750 (Figure 4.4(b)), so the attacker most likely goes back to Reconnaissance to try again. Without the failure matrix, it would most likely carry on into the network as though it had broken in.

The failure matrix gives the APT attacker model adaptivity, property 4 of Table 3.2: it chooses its next tactic in response to a failure, and reads only the verdict, so a failure caused by an MTD deployment routes like any other. The failure matrix can reweight only the next tactics the attack flows give. In c_2, no move leads from Initial Access back to preparation, so a failed Initial Access still goes on into the network. Where the attack flows give a tactic no next tactic, there is nothing to reweight, and the token retraces the edge it travelled. Section 5.4 sets every factor to one and finds that the failure matrix changes where the attacker goes after a failure, but not what it achieves.

4.4.4 Vulnerability memory

The *vulnerability memory* extends MTDSim's exploit attack action so that the APT attacker model is more likely to exploit a vulnerability it has exploited before. The baseline attacker already remembers the vulnerabilities it has exploited. Exploiting one again takes it half the time (Section 2.2.3). In the APT attacker model, the tactic duration replaces the exploit's time (3 in Figure 4.5), so the baseline attacker's memory saves it no time. The vulnerability memory acts on the chance of success instead. It is not learning (property 7 of Table 3.2): it keeps vulnerabilities, not the patterns of MTD deployments, and it changes the odds of an exploit, not which technique comes next.

Each earlier success on a vulnerability, on any host, triples the attacker's odds of exploiting it again (the odds its attack complexity sets, Section 2.2.1). The factor of three is our judgement. The memory lasts the whole run, through every MTD deployment. The *diversity pool* is the services each operating system can run. Hosts that run the same service carry the same vulnerabilities, so we expect the memory to matter more when the diversity pool is small. Section 5.4 tests this expectation, and finds that the memory raises the share of exploits that succeed, most with a small diversity pool, but leaves what the attacker achieves unchanged.

4.5 Evaluation metrics

SQ3 asks how MTD performs against the APT attacker model compared with the baseline attacker. We adopt the attack outcome and MTD effectiveness metrics of MTD evaluations (Section 3.2.2). None of them measures how an attacker behaves, so none shows whether an attacker model has the properties it needs to model an APT attacker (Table 3.2). We add attacker behaviour metrics for three of those properties: objective-specific behaviour, choice of attack strategy and detection avoidance. We also add one MTD effectiveness metric for a fourth property, adaptivity: time lost per MTD deployment. Table 4.3 gives the metrics and their sources.

Table 4.3: The evaluation metrics and their sources.

Metric	Source
Attacker behaviour (Section 4.5.1)	
Relative tactic occurrence	[26]
Distinct openings	this dissertation, after [9]
Attack rate	[45]
Attack confidentiality	[46]
Attack outcome (Section 4.5.2)	
Attack success probability (ASP)	[5]
Network compromise ratio (NCR)	[12]
Mean time to compromise (MTTC)	[12], [32]
MTD effectiveness (Section 4.5.3)	
NCR reduction	[34]
Attack actions blocked per MTD deployment	[6]
Time lost per MTD deployment	this dissertation, after [47]

4.5.1 Attacker behaviour

The four attacker behaviour metrics count turns or attack actions. A *turn* is one tactic the token of the Petri net enters (Section 4.3). The baseline attacker has no tactics, so its turn is one attack action, and consecutive repeats of an attack action are one turn. A turn in an unmapped tactic runs no attack action (Section 4.4.1). Every attack action counts except one that fails because its preconditions are missing (Section 4.4), which never reaches the network. The baseline attacker's exploit attack action counts once, however many of a host's vulnerabilities it tries.

Relative tactic occurrence. An attacker with objective-specific behaviour uses different techniques after the foothold for different objectives. The attack profiles of the APT attacker model should therefore divide their turns among the tactics differently, most of all in actions on objectives (Section 4.2). Relative tactic occurrence is the percentage of an attacker's turns taken in each tactic, counting over every run:

$$\text{relative tactic occurrence}(p) = 100 \times \frac{\text{turns in tactic } p}{\text{all turns}}. \tag{4.5}$$

Rodríguez et al. tabulate the same percentage over the events of an attack log, in which one tactic can log many events [26, Table 3]. The attack profiles were built to differ, so the metric tests whether their runs still differ in MTDSim, where a failed attack action reweights the attacker's next tactics (Section 4.4.3).

Distinct openings. An attacker with choice of attack strategy chooses among several sequences of techniques toward the same objective. An attacker with one attack strategy takes the same first turns in every run, and an attacker that chooses does not. A run's *opening* is the sequence of its first k turns. Distinct openings is the number of different openings among an attacker's runs, from 1, when every run has the same opening, to the number of runs:

$$\text{distinct openings}(k) = \text{number of different openings}. \tag{4.6}$$

Hong et al. measure how much the attack paths through a network, sequences of hosts, change when an MTD deployment changes the network [9]. Distinct openings measures how much one attacker's openings vary across its runs.

Attack rate. An attacker with detection avoidance moves low and slow and chooses techniques an IDS does not flag. MTDSim detects no attacker, so the two metrics for detection avoidance measure it by proxy, through how often the attacker acts. Attack rate [45] is the number of attack actions an attacker takes per minute, averaged over its runs:

$$\text{attack rate} = \operatorname*{mean}_{\text{runs}} \left(\frac{\text{attack actions}}{\text{run length in minutes}} \right), \tag{4.7}$$

where a run's length is from its start to the end of its last turn. A higher attack rate is a more aggressive attacker [48].

Attack confidentiality. Attack rate is a mean, so it does not show bursts of attack actions, which an IDS flags. Attack confidentiality [46] is the share of an attacker's attack actions that an IDS does not flag. MTDSim has no IDS, so the default port-scan rule of the Snort IDS stands in for one. It flagged a source that connected to five different addresses within 60 s [49, Sect. 5.2]. Here an attack action is flagged when it is at least the attacker's fifth attack action within a window of 60 s, on any target. Counting over every run,

$$\text{attack confidentiality} = \frac{\text{attack actions not flagged}}{\text{attack actions}}. \tag{4.8}$$

A higher attack confidentiality is an attacker the IDS flags less often.

4.5.2 Attack outcome

The attack scenario sets the attacker's goal (Table 2.3).

Attack success probability (ASP). ASP [5] is the share of runs in which the attacker reaches its goal:

$$\text{ASP} = \frac{\text{runs in which the attacker reaches its goal}}{\text{runs}}. \tag{4.9}$$

Network compromise ratio (NCR). NCR [12] is the share of the hosts in the network that the attacker has compromised when a run ends, averaged over runs:

$$\text{NCR} = \operatorname*{mean}_{\text{runs}} \left(\frac{\text{hosts compromised}}{\text{hosts in the network}} \right). \tag{4.10}$$

Mean time to compromise (MTTC). MTTC [12], [32] is the mean time from the start of a run to the attacker reaching its goal, over the runs in which it does:

$$\text{MTTC} = \operatorname*{mean}_{\text{runs in which the attacker reaches its goal}} (\text{time the attacker reaches its goal}). \tag{4.11}$$

4.5.3 MTD effectiveness

NCR reduction and time lost per MTD deployment compare an attacker's runs under MTD with its runs on the same seeds with no MTD, so each pair of runs starts from the same network.

NCR reduction. NCR reduction is the share of an attacker's NCR that MTD removes:

$$\text{NCR reduction} = 1 - \frac{\text{NCR under MTD}}{\text{NCR with no MTD}}. \tag{4.12}$$

It is 0 when MTD has no effect, 1 when no host is compromised under MTD, and negative when MTD raises NCR. It takes the form of the mitigation factor of Alavizadeh et al. [34, eq. (13)], with NCR in place of annual loss expectancy and no floor at 0.

Attack actions blocked per MTD deployment. Attack actions blocked per MTD deployment is the number of attack actions MTD disrupts [6], divided by the number of MTD deployments before the run ends, counting over every run:

$$\text{attack actions blocked per MTD deployment} = \frac{\text{attack actions disrupted}}{\text{MTD deployments}}. \tag{4.13}$$

A deployment disrupts at most the attack action in progress, so the metric runs from 0, when no deployment disrupts one, to 1, when every deployment does.

Time lost per MTD deployment. Time lost per MTD deployment, the metric for adaptivity, is how much an MTD deployment delays the attacker's next compromise of a host. The *wait* is the time from an MTD deployment to the attacker's next compromise of a host, capped at the next MTD deployment. Time lost is the wait under MTD less the wait from the same moment on the same seed's run with no MTD, averaged over MTD deployments, a difference of restricted means [47, eq. (1)]:

$$\text{time lost per MTD deployment} = \operatorname*{mean}_{\text{MTD deployments}} \big(\text{wait under MTD} - \text{wait with no MTD}\big). \tag{4.14}$$

A deployment at 6 000 s, with the next compromise at 6 900 s under MTD and at 6 400 s with no MTD, costs the attacker 500 s. Time lost is negative when a deployment brings the next compromise forward.

CHAPTER 5

Evaluation

This chapter answers SQ3 by evaluating MTD in MTDSim against the APT attacker model and the baseline attacker. Both attackers are measured with the attacker behaviour, attack outcome and MTD effectiveness metrics of Section 4.5. Section 5.1 sets out the experimental setup: the varied and held settings, runs, statistical analysis and ablation studies. Section 5.2 compares the attacker behaviour and attack outcome of the two attackers with no MTD. Section 5.3 measures the MTD effectiveness of each MTD mechanism and deployment strategy against both attackers. Section 5.4 compares the APT attacker model with and without each of three components: the attack profiles, the failure matrix and the vulnerability memory. Chapter 6 reads the results against the eight properties and what they mean for MTD evaluation.

5.1 Experimental setup

Varied settings. The APT attacker model and the baseline attacker each run with no MTD and under each of the four deployment strategies (Table 2.2). The single deployment strategy runs once for each of the seven MTD mechanisms. MTDShield runs as released (Appendix D.1). The APT attacker model runs on each of its four attack profiles, c_1 to c_4 (Section 4.2). Each deployment strategy runs at six deployment intervals: 50, 100 and 200 s, which the earlier MTDSim studies ran, and 500, 1 000 and 2 000 s, up to ten times their longest (Table 2.4).

Held settings. Table 5.1 gives every held setting and its source. Both attackers follow the targeted attack scenario (Table 2.3), on the default network of Section 2.2.1 with two target hosts in place of five, as in the simulator's released experiment code [15].

Runs. Each configuration of attacker, MTD and deployment interval in Table 5.1 is run on the same 1 000 seeds, the number recommended for randomised algorithms [50]. Each seed generates one network, so the 1 000 runs of every configuration start from the same 1 000 networks. The 408 000 runs of Sections 5.2 and 5.3 took about 29 hours, seven at a time, on a laptop with an Intel Core i7-1360P processor, in Python 3.11 under Ubuntu 22.04. Every metric at every configuration, with its interval, is published with the experiment scripts.

Statistical analysis. Every interval is a 95 % confidence interval: Clopper–Pearson for ASP [51], the normal approximation for a mean, printed with ±, and otherwise a

Table 5.1: The experimental setup. Values without a citation are our choice.

Setting	Value
Varied	
Attacker	the baseline attacker [6]; the APT attacker model on c_1 to c_4
MTD	no MTD; the single deployment strategy, once for each of the seven MTD mechanisms [6], [12]; the random and alternative deployment strategies [12]; MTDShield, as released [14]
Deployment interval	50, 100 and 200 s [12], [17]; 500, 1 000 and 2 000 s; near-periodic (Section 2.2.2) [15]
Held	
Network	50 hosts, 4 levels of depth, 8 subnets, 5 exposed endpoints [15]
Target hosts	2, at the deepest level [15]
Diversity pool	20 services per operating system [15]
Attack scenario	targeted [6]
End of run	a target host compromised [6], or 15 000 s [17]
Runs per configuration	1 000 [50], on seeds 0 to 999
Ablation studies	
Attack profiles	replaced by the attack graph (Section 4.1): every MTD and deployment interval; by 10 random partitions of the attack flows: no MTD, and IP shuffle and OS diversity at 200 and 2 000 s
Failure matrix	removed: no MTD, and IP shuffle and OS diversity at 200 and 2 000 s
Vulnerability memory	removed: no MTD, and service diversity and OS diversity at 200 s, each at diversity pools of 1, 2, 3, 5, 10 and 20 services per operating system

percentile bootstrap over runs [52]. Effect size is Cohen's d with the pooled standard deviation, negligible within ± 0.2 [53], [54]. The Scott–Knott effect size difference test [54] ranks the MTD by NCR reduction for each attacker at each deployment interval. Neighbouring ranks differ significantly and by more than a negligible effect. Spearman's rank correlation, ρ [55], measures how closely the two attackers order the MTD by NCR reduction.

Ablation studies. Table 5.1 gives what replaces each component and the configurations each ablation study runs. IP shuffle and OS diversity are one MTD mechanism of each layer (Table 2.1), and service diversity and OS diversity are the two that draw services from the diversity pool. A component's effect is d on NCR, the APT attacker model with it minus without it. The effect is negligible when its interval lies wholly inside ± 0.2, not negligible when it lies wholly outside, and inconclusive when it crosses ± 0.2 [56]. The tactic durations cannot be removed, so Appendix D.4 varies their values instead.

5.2 APT attacker model versus baseline attacker

This section compares the two attackers with no MTD running.

On objective-specific behaviour, the baseline attacker takes only its six attack actions (Figure 5.1(a)). Every attack profile also spends 26 to 47 % of its turns in unmapped tactics, which run no attack action (Section 4.4.1). The unmapped tactics include Collection, Exfiltration and Impact, the tactics of actions on objectives. The attack profiles divide their turns among the tactics differently, each over the tactics its attack flows reach

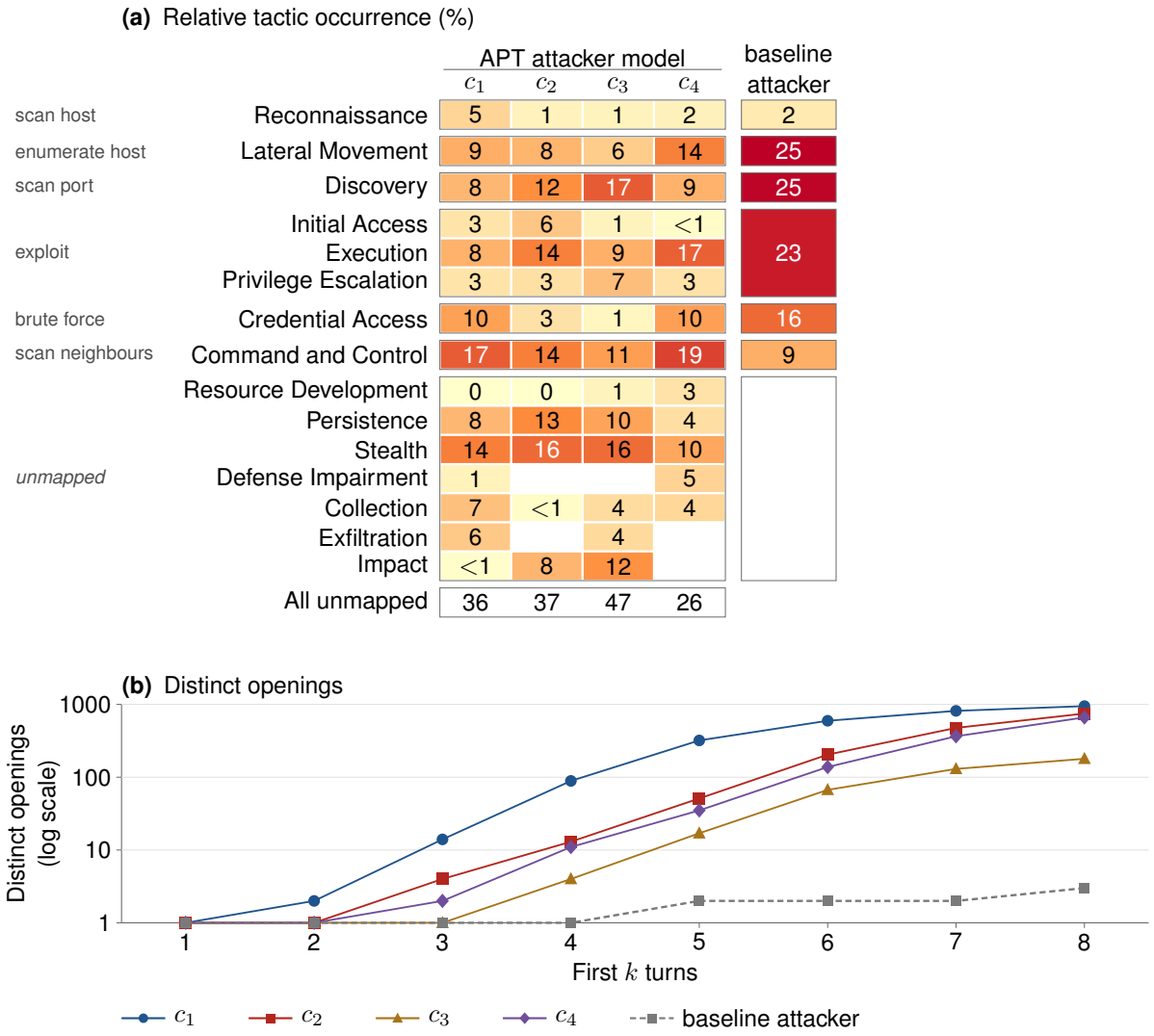


Figure 5.1: Relative tactic occurrence and distinct openings with no MTD running (Section 4.5.1). (a) Left: the attack action each tactic runs (Section 4.4.1). The baseline attacker's column counts attack actions. Blank: a tactic the attack profile's attack flows do not reach (Figure 4.3); 0: a tactic they reach that no run takes. (b) 1: every run has the same opening; 1 000: every run has its own.

Table 5.2: Attack outcome, attack rate and attack confidentiality with no MTD running (Section 4.5). In parentheses: a 95 % interval, or the runs MTTC is over.

Attacker	ASP	NCR	MTTC (s)	Attack rate (per minute)	Attack confidentiality (%)
APT attacker model					
c_1	0.12 (0.10, 0.15)	0.149 ± 0.004	$11\,300 \pm 500$ (124 runs)	1.087 ± 0.005	92.4 (92.2, 92.5)
c_2	0.14 (0.12, 0.16)	0.209 ± 0.005	$10\,500 \pm 500$ (139 runs)	0.888 ± 0.006	96.0 (95.8, 96.1)
c_3	0.06 (0.04, 0.07)	0.172 ± 0.006	$9\,700 \pm 900$ (56 runs)	0.680 ± 0.008	98.1 (98.0, 98.2)
c_4	0.04 (0.03, 0.06)	0.154 ± 0.005	$10\,800 \pm 800$ (42 runs)	1.228 ± 0.009	89.4 (89.3, 89.6)
Baseline attacker	0.66 (0.63, 0.69)	0.497 ± 0.012	$9\,100 \pm 200$ (656 runs)	1.709 ± 0.015	82.2 (81.8, 82.6)

(Figure 4.3).

On choice of attack strategy, the baseline attacker has fewer distinct openings than any attack profile (Figure 5.1(b)). The baseline attacker's fixed procedure gives all its runs one opening over their first four turns (Section 2.2.3). All but two of its 1 000 runs then branch once, into brute force or scan neighbours, before enumerating a host again. The runs of the attack profiles part ways from the second turn (c_1) to the fourth (c_3), and have 179 to 947 distinct openings over eight turns.

On detection avoidance, the baseline attacker has a higher attack rate and a lower attack confidentiality than any attack profile (Table 5.2). The baseline attacker's attack rate is 1.709 attack actions per minute, against 0.680 to 1.228 for the attack profiles, and its attack confidentiality 82.2 %, against 89.4 to 98.1 %. Across the four attack profiles and the baseline attacker, attack confidentiality falls as attack rate rises.

On attack outcome, the baseline attacker has a higher ASP and NCR than any attack profile (Table 5.2). The baseline attacker's ASP is 5 to 16 times that of an attack profile, and its NCR 2.4 to 3.3 times. The baseline attacker's MTTC, 9 100 s, is just shorter than the attack profiles', 9 700 to 11 300 s.

5.3 APT attacker model versus MTD

This section compares the two attackers with each MTD mechanism and deployment strategy running.

5.3.1 Attack actions blocked and time lost per MTD deployment

Each MTD mechanism runs under the single deployment strategy at the longest deployment interval, 2 000 s, because the next MTD deployment caps time lost at the deployment interval (Section 4.5.3).

An MTD deployment blocks the baseline attacker's attack actions more often than the APT attacker model's (Figure 5.2(a)). Attack actions blocked per MTD deployment is 1 for the baseline attacker under every host-layer mechanism, against 0.35 to 0.39 for the APT attacker model. Under every service-layer mechanism, attack actions blocked per MTD deployment is 0.95 against 0.15, and under user shuffle 0.07 against 0.01. The APT attacker model runs no attack action while in an unmapped tactic, so an MTD deployment then blocks none (Section 4.4.1).

On adaptivity, the two attackers lose time to different MTD mechanisms (Figure 5.2(b)). The APT attacker model loses 242 to 324 s per host-layer deployment, and the baseline attacker none, though every host-layer deployment blocks its attack action. The baseline attacker loses 493 s per service-diversity deployment, and the APT attacker model 60 s. Under every other MTD mechanism, the APT attacker model loses 26 s or less. The baseline attacker's time lost is at or below zero under every MTD mechanism but service diversity. Under every host-layer and service-layer mechanism, the first deployment of each run brings the baseline attacker's first compromise 204 to 232 s forward. The first deployment disrupts the baseline attacker while it exploits its first exposed endpoint. Its procedure sends it straight back to that endpoint with the progress it made there (Section 2.2.3). It otherwise returns there only after its list of

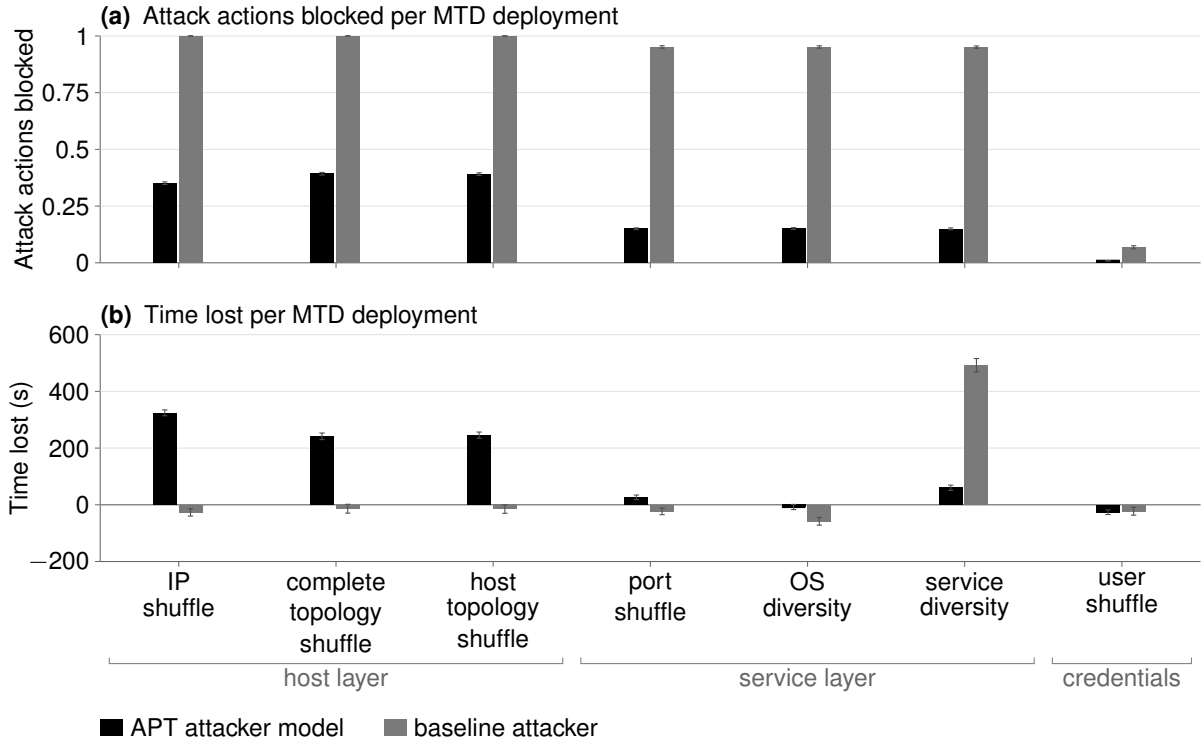


Figure 5.2: Attack actions blocked and time lost per MTD deployment for each MTD mechanism, at a deployment interval of 2 000 s. The APT attacker model is averaged over c_1 to c_4. Table D.2 gives each bar's value with its confidence interval.

hosts runs out. After the first MTD deployment, each later host-layer or port-shuffle deployment costs it 18 to 32 s.

5.3.2 MTD ranking against each attacker

No MTD mechanism or deployment strategy ranks first against both attackers at the default deployment interval, 200 s (Table 5.3). IP shuffle, host topology shuffle and complete topology shuffle share rank 1 against the APT attacker model, and service diversity and port shuffle share rank 1 against the baseline attacker. At every deployment interval, rank 1 against the APT attacker model holds only host-layer mechanisms, and rank 1 against the baseline attacker only service-layer mechanisms (Table D.4). ρ between the two attackers' NCR reductions is 0.01 (−0.19, 0.05) at 200 s, and lies between −0.24 and 0.25 at every deployment interval.

The host-layer mechanisms reduce the APT attacker model's NCR more than the service-layer mechanisms at every deployment interval, and the service-layer mechanisms reduce the baseline attacker's NCR more than the host-layer mechanisms (Figure 5.3(a) and (b)). Up to 200 s, the host-layer mechanisms reduce the APT attacker model's NCR by 0.96 or more on average, and the service-layer mechanisms the baseline attacker's by 0.84 or more. The APT attacker model's host-layer reduction stays above 0.5 up to 1 000 s, and the baseline attacker's service-layer reduction only up to 200 s. User shuffle reduces the APT attacker model's NCR below zero at every deployment interval (Figure 5.3(c)).

Random and alternative reduce the APT attacker model's NCR more than the baseline

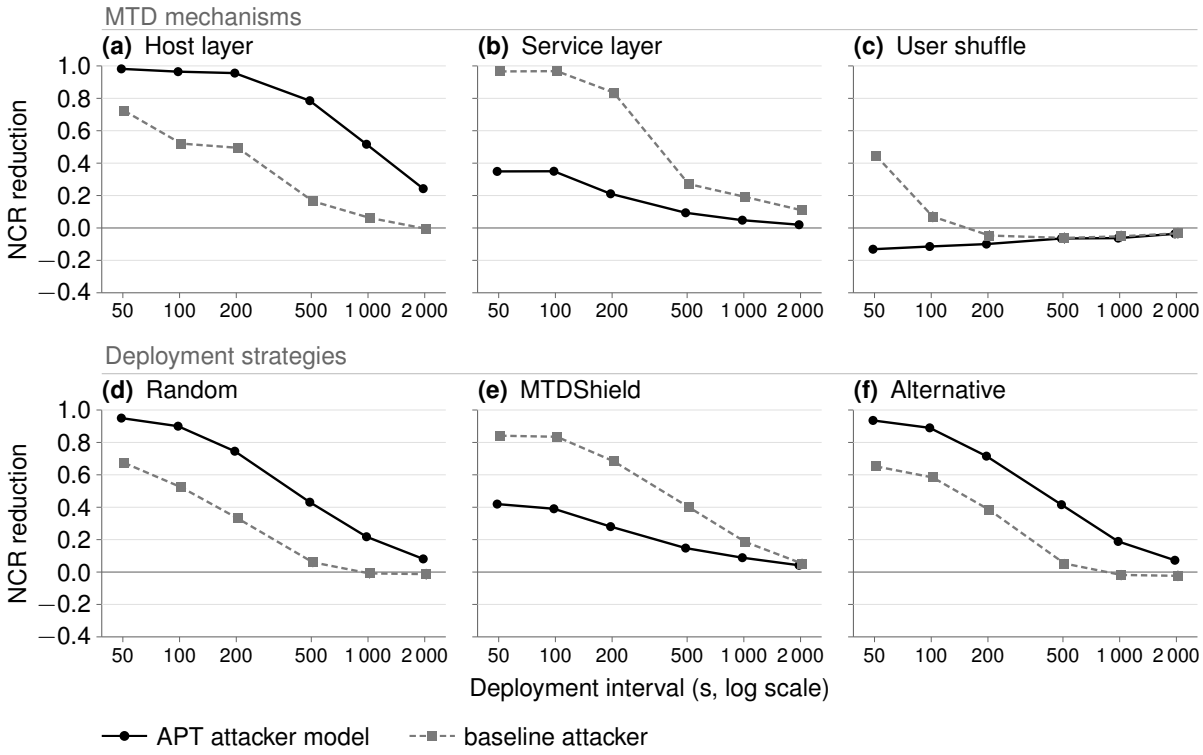


Figure 5.3: NCR reduction against the deployment interval. The APT attacker model is averaged over c_1 to c_4. Host layer and service layer: the mean over the layer's MTD mechanisms. Where no whisker shows, the confidence interval is narrower than the marker. Table D.3 gives each point's value with its confidence interval.

Table 5.3: The MTD mechanisms and deployment strategies ranked by NCR reduction against each attacker, at a deployment interval of 200 s. The APT attacker model is averaged over c_1 to c_4. Bold: rank 1.

	APT attacker model		Baseline attacker	
MTD	Rank	NCR reduction	Rank	NCR reduction
IP shuffle	**1**	0.96	2	0.68
host topology shuffle	**1**	0.95	3	0.38
complete topology shuffle	**1**	0.95	3	0.42
random	2	0.75	4	0.33
alternative	3	0.72	3	0.38
service diversity	4	0.36	**1**	0.92
MTDShield	5	0.28	2	0.68
port shuffle	6	0.20	**1**	0.91
OS diversity	7	0.07	2	0.68
user shuffle	8	−0.10	5	−0.05

attacker's at every deployment interval (Figure 5.3(d) and (f)). MTDShield reduces the baseline attacker's NCR more, at every deployment interval up to 1 000 s (Figure 5.3(e)). At 200 s, MTDShield chooses service diversity in 70 % or more of its decisions against either attacker in 100 runs (Appendix D.1).

5.3.3 MTD effectiveness by attack profile

Under every MTD mechanism and deployment strategy at 50 to 200 s, the four attack profiles' NCR reductions differ from one another less than the baseline attacker's NCR reduction differs from the nearest attack profile's (Figure 5.4). The earlier MTDSim studies ran these deployment intervals. At 200 s, the attack profiles' NCR reductions lie within 0.14 of one another, and the baseline attacker's lies 0.24 or more from the nearest attack profile's under every MTD mechanism and deployment strategy but user shuffle. Under the deployment strategies, the baseline attacker's NCR reduction lies outside the attack profiles' range up to 1 000 s (panels h to j).

Of the four attack profiles, c_3 has the NCR reduction furthest from the baseline attacker's at 500 and 1 000 s, under the host-layer mechanisms and service diversity. Under the host-layer mechanisms, the baseline attacker's NCR reduction is below all four attack profiles', and c_3's is the highest (panels a to c). Under service diversity, the baseline attacker's NCR reduction is above all four, and c_3's is the lowest (panel f).

5.4 Ablation studies

This section compares the APT attacker model with and without each of three components: the attack profiles, the failure matrix and the vulnerability memory.

Attack profiles. Removing the attack profiles runs the APT attacker model on the attack graph (Section 4.1). The APT attacker model compromises more without them, by an effect on NCR that is not negligible (Table 5.4). With no MTD, its NCR is 0.171 with the attack profiles and 0.189 without, a d of -0.28. Under the MTD mechanisms and deployment strategies, the effect is not negligible in 29 of the 60 configurations, negligible in seven and inconclusive in 24 (Table D.8). At every deployment interval, the same three MTD mechanisms reduce NCR the most with the attack profiles and without them: IP shuffle, host topology shuffle and complete topology shuffle.

Ten random partitions split the attack flows into four groups of the attack profiles' sizes (Table 5.1). With no MTD, the APT attacker model compromises more with the attack profiles than with any of the ten, by an effect on NCR that is not negligible, a d of 0.25 to 1.47 (Table D.9). Under IP shuffle and OS diversity, the effect is not negligible against six to ten of the ten in each configuration, in the attack profiles' favour except against two partitions under IP shuffle at 200 s.

Failure matrix. Removing the failure matrix makes the APT attacker model choose its next tactic after a failure by the base weights, as it does after a success (Section 4.4.3). The failure matrix changes where the APT attacker model goes after a failure, and its effect on NCR is negligible (Table 5.4). After a failed Initial Access (the example of Section 4.4.3),

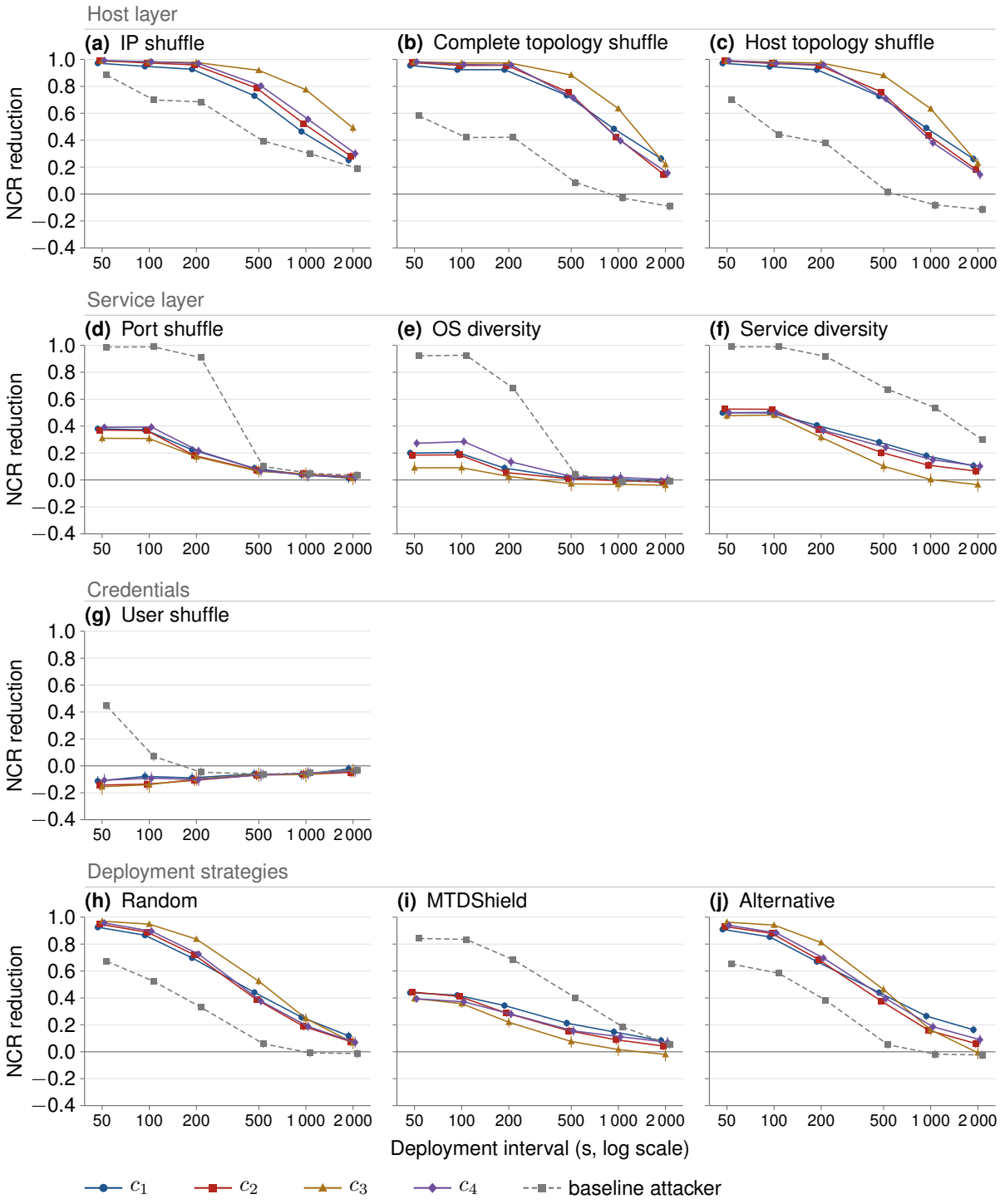


Figure 5.4: NCR reduction against the deployment interval for each attack profile and the baseline attacker. Where no whisker shows, the confidence interval is narrower than the marker. Table D.3 gives each point's value for the baseline attacker and the mean over c_1 to c_4 for the APT attacker model.

Table 5.4: The APT attacker model with and without each component, averaged over c_1 to c_4. d on NCR is with minus without. Parentheses give its 95 % interval. Bold: not negligible (Section 5.1).

	NCR		
MTD	with	without	d
Attack profiles			
no MTD	0.171	0.189	**−0.28 (−0.35, −0.22)**
IP shuffle, 200 s	0.007	0.013	**−0.47 (−0.54, −0.40)**
IP shuffle, 2 000 s	0.114	0.145	**−0.56 (−0.63, −0.49)**
OS diversity, 200 s	0.159	0.174	**−0.26 (−0.33, −0.20)**
OS diversity, 2 000 s	0.174	0.187	−0.21 (−0.27, −0.14)
Failure matrix			
no MTD	0.171	0.167	+0.07 (+0.03, +0.11)
IP shuffle, 200 s	0.007	0.006	+0.13 (+0.06, +0.19)
IP shuffle, 2 000 s	0.114	0.109	+0.12 (+0.07, +0.17)
OS diversity, 200 s	0.159	0.159	−0.01 (−0.05, +0.04)
OS diversity, 2 000 s	0.174	0.170	+0.06 (+0.02, +0.11)
Vulnerability memory			
no MTD	0.171	0.169	+0.04 (+0.02, +0.06)
service diversity, 200 s	0.109	0.106	+0.07 (+0.05, +0.09)
OS diversity, 200 s	0.159	0.156	+0.05 (+0.02, +0.07)

the APT attacker model goes back to Reconnaissance in 0.76 to 0.83 of cases in c_1, c_3 and c_4 with the failure matrix, and in 0.08 to 0.10 without. The failure matrix leaves time lost per MTD deployment unchanged. Under IP shuffle and OS diversity at 2 000 s, the 95 % interval of the difference with and without it includes zero.

Vulnerability memory. Removing the vulnerability memory makes the APT attacker model no more likely to exploit a vulnerability it has exploited before (Section 4.4.4). The vulnerability memory raises the share of exploits that succeed, and its effect on NCR is negligible (Table 5.4). With no MTD, the share of exploits that succeed is 0.73 with the vulnerability memory and 0.69 without it (Table D.10).

Section 4.4.4 expects the vulnerability memory to matter more with a smaller diversity pool. With a diversity pool of one service per operating system, the share of exploits that succeed is 0.92 with it and 0.69 without it, and its effect on NCR is still negligible (Table D.10).

CHAPTER 6

Discussion

This chapter explains the results of Chapter 5 and answers the three sub-questions of Chapter 1; Chapter 7 draws the answers together into the answer to the research question. Every finding is about MTDSim; Section 6.4 draws implications for MTD evaluation from them. Sections 6.1 and 6.2 follow Chapter 5's order. Section 6.1 explains how the two attackers differ with no MTD on the three properties Chapter 5 measures, mostly by the APT attacker model's tactic durations and the order of its tactics. Section 6.2 explains why a different layer's MTD mechanisms reduce each attacker's NCR most, and answers SQ3. Section 6.3 scores the APT attacker model on the eight properties of Section 3.3.1, beside the three MTD attacker models scored there, and answers SQ1 and SQ2. Section 6.4 states what the answers mean for anyone who evaluates MTD, and Section 6.5 sets out the threats to their validity.

6.1 APT attacker model versus baseline attacker

The APT attacker model differs from the baseline attacker with no MTD on objective-specific behaviour, choice of attack strategy and detection avoidance (Section 5.2). In attack outcome, it also reaches a target host in fewer runs and compromises fewer hosts (Table 5.2). We attribute these differences to its tactic durations and the order of its tactics. With the baseline attacker's times in place of its tactic durations, it closes almost two thirds of its gap in ASP, all of it for c_1 and c_2 but almost none for c_3 (Appendix D.5.1), and we attribute the rest to the order of its tactics (T2). Removing the failure matrix or the vulnerability memory changes its NCR negligibly (Section 5.4), so neither alone explains its lower NCR.

On objective-specific behaviour, the attack profiles' runs still differ in MTDSim, most of all in actions on objectives (Figure 5.1(a)). Exfiltration occurs only in c_1 and c_3, Impact almost only in c_2 and c_3, and neither in c_4. Each attack profile follows the order of tactics of its own attack flows (Section 4.2). Collection, Exfiltration and Impact, the tactics of actions on objectives, are unmapped because MTDSim's six attack actions only find and compromise hosts (Section 2.2.3). The APT attacker model therefore steals no data and impedes no system.

On choice of attack strategy, the attack profiles' 1 000 runs have 179 to 947 distinct openings over eight turns, and the baseline attacker's 3 (Figure 5.1(b)). In each run, the Petri net picks the next tactic at random, in proportion to the edge weights (Equations 4.2 and 4.4), whereas the baseline attacker's procedure fixes the order of its attack actions (Figure 2.3). Through the tactic-to-action mapping, the order of the APT attacker model's tactics can run an attack action before its preconditions are met (Section 4.4),

and 15 to 30 % of the attack profiles' attack actions fail this way (Appendix D.5.1).

On detection avoidance, the attack profiles have a lower attack rate than the baseline attacker, 0.68 to 1.23 attack actions per minute against 1.71, and a higher attack confidentiality, 89 to 98 % against 82 % (Table 5.2). The APT attacker model's tactic durations make it slower. With the baseline attacker's times in place of its tactic durations, it takes 2.18 to 4.59 attack actions per minute, more than the baseline attacker, and its attack confidentiality falls to 32 to 56 %, below the baseline attacker's (Appendix D.5.1). MTDSim has no IDS, so the APT attacker model cannot choose techniques an IDS does not flag. Stealth and Defense Impairment, the tactics for avoiding detection, are unmapped, so in MTDSim its detection avoidance comes only from its tactic durations.

We attribute the APT attacker model's three properties and its lower ASP and NCR to the same two causes: its tactic durations and the order of its tactics. ASP and NCR alone would therefore rate it the weaker attacker for what gives it objective-specific behaviour, choice of attack strategy and detection avoidance.

6.2 APT attacker model versus MTD

In answer to SQ3, at every deployment interval the host layer reduces the APT attacker model's NCR more than the service layer does, and the service layer reduces the baseline attacker's NCR more than the host layer does (Figure 5.3). We attribute the difference between the two attackers to the APT attacker model's tactic durations and the order of its tactics (T2).

On adaptivity, the APT attacker model loses time mainly to the host layer, 242 to 324 s per deployment, where the baseline attacker's time lost is at or below zero; the baseline attacker loses time only to service diversity, 493 s per deployment, where the APT attacker model loses 60 s (Figure 5.2(b)). After a host-layer deployment, neither attacker has a host to act on (Section 2.2.3). MTDSim sends the baseline attacker back to scan host, and its procedure runs enumerate host next. For the APT attacker model, only Lateral Movement runs enumerate host (Section 4.4.1), so it has no host until the order of its tactics next brings it to Lateral Movement, and meanwhile the attack actions of its other tactics can fail because their preconditions are missing. The tactic-to-action mapping therefore gives the host layer part of its lead (T2). The APT attacker model's failure matrix leaves its time lost under IP shuffle unchanged (Section 5.4), so the host layer's lead reflects a disruption it does not adapt to. Service diversity redraws every service, and MTDSim sends the baseline attacker back to scan port on the same host, so it exploits the host's new vulnerabilities from the start. The APT attacker model keeps the ports it had found and goes on to the tactic its Petri net picks, which is consistent with its 60 s.

We attribute the service layer's smaller reduction of its NCR to the APT attacker model's tactic durations. A service-layer deployment disrupts an attacker only while it scans ports, exploits or brute-forces (Section 2.2.3), which the baseline attacker does back to back. The APT attacker model's tactic durations keep it mostly in unmapped tactics, which run no attack action (Section 5.3.1), and in Command and Control, which runs scan neighbours. Consistent with this, attack actions blocked per MTD deployment under every service-layer mechanism is 0.15 for the APT attacker model, against 0.95 for the baseline attacker (Figure 5.2(a)). With its unmapped tactics at 0 s, the APT attacker

model spends 45 % of its time in scan neighbours, and service diversity's reduction of its NCR at 200 s falls from 0.36 to 0.29 (Table D.12).

6.3 APT attacker model versus recent MTD attacker models

Table 6.1: The APT attacker model beside the three MTD attacker models of Table 3.3, on the eight properties of Table 3.2: 1 campaign persistence, 2 objective-specific behaviour, 3 choice of attack strategy, 4 adaptivity, 5 detection avoidance, 6 cost–benefit reasoning, 7 learning and 8 deployment-strategy awareness. ✓ has the property, P has it in part, ✗ does not have it. Appendix B derives each mark from its predicates.

Attacker model	Year	1 Pers.	2 Obj.	3 Strat.	4 Adapt.	5 Detect.	6 Cost	7 Learn.	8 Deploy.
Baseline attacker [6], [12]	2023	✓	✗	P	P	✗	✗	✗	✗
Masud et al. [31]	2025	P	✗	✗	✗	✗	✗	✗	✗
Kim et al. [30]	2026	P	✗	✗	✗	P	✗	✗	✗
APT attacker model, this dissertation	2026	✓	P	P	P	P	✗	✗	✗

The APT attacker model has more of the eight properties than any of the three MTD attacker models of Section 3.3.2 (Table 6.1). It is the only attacker model with any objective-specific behaviour, which it has in part. All four attacker models lack cost–benefit reasoning, learning and deployment-strategy awareness.

In answer to SQ1, the attack flows drawn from threat reports give the APT attacker model the order of its tactics for each objective. That order gives it two properties in part, choice of attack strategy and objective-specific behaviour (Table 6.1). Threat reports time only whole intrusions, so the attack flows almost never record how long a step took (Section 4.4.2). Drawn from incidents that succeeded, they never record what the attacker did after a step failed (Section 4.4.3). In MTDSim, MTD disrupts the APT attacker model at these two points: its tactic durations set how likely a deployment is to cut an attack action short and fail it, and its failure matrix sets which tactic it takes after an attack action fails (Section 4.4). The APT attacker model is therefore grounded in threat reports for the order of its tactics, and rests on our judgement where MTD disrupts it.

In answer to SQ2, the APT attacker model executes the order of its tactics inside an existing simulator, MTDSim, as a Petri net that chooses its next tactic at runtime. It maps eight of the 15 tactics onto MTDSim's six attack actions (Section 4.4.1). It shares the baseline attacker's attack actions and MTD deployments, so it keeps the baseline attacker's campaign persistence. Any difference between the two attackers comes from what the APT attacker model adds: the order of its tactics and four parts we declared (Section 4.4). Two of those parts, the tactic durations and the failure matrix, fill what the threat reports do not record. The failure matrix makes the APT attacker model's response to failure a fixed rule of ours, so adaptivity is in part (Table 6.1). The Petri net's draw of its next tactic, at random in proportion to the edge weights, is also a fixed rule of ours, so choice of attack strategy is in part. The same attack actions cap the APT attacker model, because the other seven tactics are unmapped. Three of them are the

tactics of actions on objectives, so objective-specific behaviour is in part. MTDSim has no IDS, so the APT attacker model avoids detection only through its tactic durations. Detection avoidance is therefore in part. The APT attacker model is therefore comparable with the baseline attacker, and limited to what MTDSim's attack actions do: find and compromise hosts, with no actions on objectives.

6.4 Implications for MTD evaluation

In MTDSim, at every deployment interval, the most effective MTD mechanism differs between the two attackers (Table D.4). Chapter 1 set out this outcome. Which MTD mechanism an MTD evaluation finds most effective can therefore depend on its attacker model. An evaluation against a single attacker model is not enough to show how MTD performs against other attackers. This section says what that means for each of the three parts of an MTD evaluation that use the attacker model (Section 3.2): MTD modelling approaches, metrics and evaluation methods.

On MTD modelling approaches, MTDShield reduces the baseline attacker's NCR more than the random deployment strategy does, but the APT attacker model's less, at every deployment interval. Section 3.2.1 argued that the attacker model decides which deployment strategy is optimal [5], and that MTDShield's attacker is the baseline attacker its training environment simulates. At the default deployment interval of 200 s, MTDShield reduces the baseline attacker's NCR by 68 % and the APT attacker model's by 28 %; the random deployment strategy over the same four MTD mechanisms reduces them by 56 % and 80 % (Figure 5.3(e); Table D.3). At 200 s, MTDShield chooses service diversity in most of its decisions against either attacker (Section 5.3.2), and service diversity is the only MTD mechanism the baseline attacker loses time to (Section 6.2). Its choices are consistent with a policy that learned to deploy the MTD mechanism that disrupts its training attacker most (T3). Tay tested MTDShield in its training environment, against the baseline attacker, and compared it with no MTD [14]. Like a test on training data, a test in the training environment cannot separate a deployment strategy that generalises to other attackers from one fitted to its training attacker. An MTD evaluation should therefore test a learned deployment strategy against an attacker model other than the one it was trained against. The random deployment strategy over the same MTD mechanisms shows what the learning adds.

On effectiveness metrics, the two attackers rank the MTD mechanisms and deployment strategies differently, at every deployment interval. Section 3.2.2 found that MTD mechanisms cannot be compared across evaluations because each evaluation chooses its own metrics [8], and that metric values depend on how the attacker is implemented. At the default deployment interval of 200 s, the two attackers' rankings on NCR reduction are uncorrelated ($\rho = 0.01$; Section 5.3.2). Service diversity, for example, reduces the baseline attacker's NCR by 92 % and the APT attacker model's by 36 %, on the same networks (Table 5.3). Ranking on ASP reduction orders the two layers as NCR reduction does, for each attacker (Appendix D.3). A shared set of metrics may not be enough to compare MTD mechanisms across evaluations whose attacker models differ. An MTD evaluation should therefore report its metrics against its own attacker model and against one that other evaluations also use.

On efficiency metrics, which MTDSim does not measure (Section 6.5), an MTD eval-

uation may still depend on its attacker model. An MTD evaluation weighs the cost of MTD, such as system performance overhead and quality of service (Table 3.1), against the effectiveness of MTD. For a given MTD mechanism, the cost grows with how often it deploys, and how often it must deploy for a given NCR reduction depends on the attacker model. Service diversity, for example, reduces NCR by about half when it deploys every 1 000 s against the baseline attacker, but only when it deploys every 100 s or more often against the APT attacker model (Table D.3). The same MTD mechanism can therefore be efficient against one attacker model and not against another.

On evaluation methods, a simulator's attack actions limit how far a new attacker model can differ from the attacker the simulator was built with. Section 3.2.3 found that simulation, the dominant evaluation method, models specific attack behaviours flexibly but that its results do not transfer directly to a real network [5]. The APT attacker model shares the baseline attacker's six attack actions, so it is comparable with the baseline attacker and limited to what those attack actions do: find and compromise hosts, with no actions on objectives (Section 6.3). We therefore attribute the two attackers' different results mostly to the APT attacker model's tactic durations and the order of its tactics (Section 6.1). With MTD, we attribute to these two differences which layer's MTD mechanisms reduce each attacker's NCR most (Section 6.2). A new attacker model in an existing simulator can therefore change how MTD is measured to perform, even when it differs in little more than when and in what order it runs the simulator's attack actions.

6.5 Threats to validity

We limited the scope of this dissertation by design. The APT attacker model chooses among the six inherited attack actions, none of them modelling actions on objectives, and pursues one line of the campaign at a time (Section 4.3). Every result is simulated, in the targeted attack scenario only, on a network of 50 hosts with a time limit of 15 000 s. We evaluate MTDSim's existing MTD mechanisms, with no efficiency metric and no IDS. Section 7.2 proposes work on the attack actions and intrusion detection. We group the threats into four types of validity [57]: internal, construct, conclusion and external.

Internal validity.

T1 A defect in the code is a plausible alternative explanation for the results in this chapter. MTDSim's code was inherited from prior studies [6], [12], [14], [17], and all of ours was written with AI assistance (AI Use Declaration). To mitigate this threat, we checked the code by its behaviour, with an event tracer, unit tests and fixed seeds that reproduce every run exactly. We repaired the defects we found in MTDSim, calling a behaviour a defect only where it departs from what the prior studies describe, so that its design choices are implemented as intended. Because of these repairs, we make no quantitative comparison with their results. The code is public for review (Chapter 1). The threat remains for behaviour no check exercised and for code we only partially reviewed.

T2 Other components of the APT attacker model may confound our explanation of its results (Sections 6.1 and 6.2), which credits tactic durations and the order of its tactics. To mitigate this threat, we ablated the attack profiles, the failure matrix

and the vulnerability memory one at a time, which rules out each as a cause of the host layer's lead (Section 5.4). A second ablation, with no MTD, replaced its tactic durations with the baseline attacker's times and closed almost two thirds of the gap in ASP between the two attackers (Appendix D.5.1). Under MTD, the host layer still leads with its unmapped tactics at 0 s (Table D.12). We never varied the tactic-to-action mapping, so we cannot separate its share from that of the order of its tactics.

T3 How we ran MTDShield may confound our explanation of its results against the APT attacker model (Section 6.4), which credits its training against only the baseline attacker. We ran MTDShield as released, outside its training settings (Table D.1), and assumed that it detects nothing during an unmapped tactic. To mitigate this threat, we kept the experimental setup identical for both attackers, and against the baseline attacker MTDShield still reduces NCR more than the random deployment strategy does at every deployment interval. Only the APT attacker model has unmapped tactics. At 200 s MTDShield chose to deploy nothing in 29 % of its decisions against it and 23 % against the baseline attacker, in 100 runs (Appendix D.1). We cannot rule out this assumption as a cause of part of the random deployment strategy's lead over MTDShield against the APT attacker model.

Construct validity.

T4 NCR reduction, our main metric, only partially measures how MTD performs. It counts the hosts an attacker compromises, whether or not it reaches a target host in the targeted attack scenario. With no MTD, 66 % of the baseline attacker's runs end at their first target host compromised, and 91 % of the APT attacker model's at the time limit. A run's NCR only rises while the run lasts, because a deployment never takes back a compromised host (Section 2.2.3). NCR reduction can therefore credit an MTD mechanism that helps the baseline attacker reach a target host, because more of its runs then end early, with fewer hosts compromised. It does so under IP shuffle at 500 to 2 000 s, and under complete topology shuffle and OS diversity at 500 s, where the baseline attacker's NCR reduction is positive and its ASP reduction negative (Tables D.3 and D.5). To mitigate this threat, we also computed ASP reduction. For both attackers at every deployment interval, it orders the two layers as NCR reduction does (Table D.5). The threat remains for rank 1 against the APT attacker model up to 200 s, where ASP reduction ties every host-layer mechanism at 1 and only NCR reduction orders them.

T5 The APT attacker model only partially represents the likelihood, timing and response to failure of an APT attacker's steps, because attack flows record only their order. We took each step's likelihood as the share of attack flows that draw it, and declared timing and response to failure by judgement. To mitigate this threat, we declared these values before any result and tested them one at a time. Timing changes NCR most, but the rankings hold across its band (Table D.7). Without the failure matrix, which sets the response to failure, NCR changes negligibly (Section 5.4). We did not test uniform likelihoods or the values together, so we cannot rule out that they change the rankings.

T6 Section 6.3's comparison only partially measures how closely an attacker model represents an APT attacker. We synthesised the eight properties from the APT definition and two surveys, and scored each attacker model from a face-value reading of its papers. To mitigate this threat, we tied each mark to a predicate (Appendix B) and cross-checked our marks against language models that blindly scored all four attacker models. We did not validate the property set, which is coarse and likely incomplete. Nor did we read the attacker models' code, which may behave differently from their papers, so the threat remains. The comparison covers only four attacker models. The three MTD attacker models we chose (Section 3.3.2) include none from game theory, the dominant MTD modelling approach [5]. That MTD attacker models in general lack properties of an APT attacker is the finding of the two surveys [5], [8].

Conclusion validity.

T7 The confidence intervals of NCR reduction treat each run as independent. Our design violates this assumption, because at each seed the four attack profiles run on the same network. To mitigate this threat, we also resampled whole seeds, keeping each network's runs together, and these wider intervals still keep the host layer and the service layer apart at every deployment interval against both attackers (Appendix D.3). The ablations are weaker evidence. They compare many cells without correction, so some may cross Cohen's threshold for a negligible effect (± 0.2) by chance. The ablation of the tactic durations was also designed post hoc (Appendix D.5.1). We therefore treat the ablations as exploratory.

External validity.

T8 The finding for SQ3 may not generalise beyond MTDSim's disruption rules, which model how an MTD deployment disrupts the attacker (Section 2.2.3). We kept these rules to their original intent, which keeps the two attackers comparable, but they were written for the baseline attacker and cannot be validated against an APT campaign (T10). In MTDSim, an IP shuffle costs the attacker the host it was on, but a real APT may keep that host through Command and Control. In MTDSim, no deployment takes back a compromised host, but in a real network, replacing a service may evict an attacker that compromised its host through that service. Each would narrow the host layer's lead over the service layer.

T9 The 38 attack flows are an unrepresentative sample of APT attackers, so the finding for SQ3 may not generalise beyond the attack flows. They record only successful incidents worth writing about (Section 4.4.3), to the detail their threat reports give, and not all by APT groups (Section 4.1). Three of the four attack profiles are drawn from only 5 to 7 attack flows (Table 4.1). Our use amplifies these biases, as the APT attacker model treats every step equally, even one its analyst drew with confidence 0 (Appendix C.3). We accepted this sample because no other source we found records the order of an attack's steps (Section 4.1). No analysis within the corpus mitigates this threat, because every part of the APT attacker model draws on the same attack flows. We make no claim about APT behaviour the attack flows do not record.

T10 We cannot validate the APT attacker model or its findings against real APT campaigns that involve MTD, because to our knowledge no data set records such a campaign. To mitigate this threat, we explored the model's behaviour through the sensitivity analysis (Appendix D.4) and the ablations, and compared it with the baseline attacker, as is standard when the real system cannot be observed [58]. However, these analyses cannot take the findings beyond the simulation.

CHAPTER 7

Conclusion

This dissertation asked how MTD performs against APT attackers. In MTDSim, MTD performs differently against the APT attacker model than against the baseline attacker. At every deployment interval, the host layer reduces the APT attacker model's NCR most, and the service layer reduces the baseline attacker's NCR most. Which MTD mechanism an MTD evaluation finds most effective can therefore depend on its attacker model. An evaluation against a single attacker model is not enough to show how MTD performs against other attackers.

7.1 Contributions

The first contribution answers SQ1 and SQ2. It is an APT attacker model that any study using MTDSim can run in place of the baseline attacker. Every earlier MTDSim study evaluated MTD solely against the baseline attacker. Threat reports record the order of an attack's tactics, but not how long each took or what followed a failure. The APT attacker model's timing and response to failure, where MTD disrupts it, therefore rest on our judgement. In MTDSim, the APT attacker model uses a Petri net to choose its next tactic at runtime, and maps eight of the 15 tactics onto the baseline attacker's six attack actions, none of which models actions on objectives. Of the four attacker models in Table 6.1, it is the only one with any objective-specific behaviour.

The second contribution answers SQ3. In MTDSim, the most effective MTD mechanism depends on the attacker model. We attribute this dependence to the APT attacker model's tactic durations and the order of its tactics. Our ablation of the tactic durations ran under MTD only with the unmapped tactics at 0 s, and we never varied the tactic-to-action mapping (T2). The finding may not generalise beyond MTDSim's disruption rules or the 38 attack flows (T8, T9). Even so, an attacker model that differs in little more than when and in what order it runs a simulator's attack actions can change how MTD is measured to perform. A learned deployment strategy should therefore be tested against an attacker model other than the one it was trained against. MTDShield was trained solely against the baseline attacker. At every deployment interval, it outperforms the random deployment strategy against that attacker, and underperforms it against the APT attacker model.

7.2 Future work

The directions below are in order of priority. The attack actions are the highest priority. We kept them unchanged by design, and they limit every result.

- *Attack actions rebuilt for modelling APT tradecraft.* The APT attacker model shares the baseline attacker's attack actions, so it stops at compromising hosts (Section 6.3). This dissertation leaves open which attack actions an MTD simulator needs to model APT tradecraft. Such attack actions could be built from the ATT&CK techniques that CTI records, each with what it acts on, such as data for actions on objectives, and its own rule for how an MTD deployment disrupts it (T8). The APT attacker model could then carry out actions on objectives, the part of objective-specific behaviour it lacks (Table 6.1).

- *Automated attack profiling.* Three of the four attack profiles are drawn from only 5 to 7 attack flows, and not all by APT groups (T9). Automated attack profiling (Section 3.1.3) could recover the order of techniques from reports on APT campaigns chosen for each attack profile's objective, for analysts to check. Each attack profile could then be drawn from more incidents, all by APT groups, which would test whether the finding for SQ3 generalises beyond the 38 attack flows.

- *Intrusion detection and MTD.* MTDSim has no IDS, so the APT attacker model avoids detection only through its tactic durations (Section 6.3). This dissertation leaves open how intrusion detection would change which MTD mechanism is most effective. The rule that flags attack actions for attack confidentiality (Section 4.5.1) could instead raise an alert each time an attacker trips it during the run. The APT attacker model could then be tested on choosing techniques an IDS does not flag, the part of detection avoidance it lacks (Table 6.1). This question could be answered by comparing the MTD mechanisms with an IDS in place, under both proactive and reactive MTD (Section 2.1).

- *Testing learned deployment strategies against other attacker models.* Our results raise the question of whether a learned deployment strategy can do better than the random deployment strategy against an attacker model it was not trained against (Section 6.4). MTDShield could be retrained against the APT attacker model and tested against both attackers beside the random deployment strategy. The result would show whether a learned deployment strategy must be trained against an attacker model close to the attackers it will face.

AI Use Declaration

I used Claude (Anthropic), through its chat interface and Claude Code, from March to October 2026, at whichever model was current: Claude Opus 4.6 to Opus 5.5, Sonnet 5, and Fable 5 and 5.1. I used no other generative AI tool.

Ideas. The research questions, the APT attacker model and the experiments came from me or from weekly meetings with my supervisors. I used Claude to develop ideas and test them against the literature and the code. I showed the results at each meeting, and my supervisors' feedback set the next step.

Literature. Claude helped me find papers and extract their content. I triaged papers by abstract and read in full those my argument rests on.

Code. Claude wrote all the code for this dissertation: the APT attacker model, the experiment scripts, the unit tests, and the scripts that produce every figure and every table of results. I specified what each part should do and checked it by its behaviour, with an event tracer, unit tests and fixed seeds that reproduce every run (T1).

Writing. Chapters 2 to 4 began as my own dictated drafts, which Claude helped me edit and cut. For the abstract and Chapters 1 and 5 to 7, Claude proposed sentences, and I chose, rewrote and critiqued them over many rounds.

The direction and every decision were mine or my supervisors'. Where Claude proposed an idea, I decided whether to keep it. I have read, checked and revised every sentence, and I take full responsibility for the content. Any errors that remain are mine.

Appendix A Original honours proposal, as submitted in March 2026. After submission, work on the defender was ruled out of scope, so this dissertation addresses only the first of the proposal's four aims, "a more sophisticated adversarial module", and runs MTDShield as released, with no IDS.

Research Proposal

Marc Labouchardiere (23857377)

13 March 2026

Title: Towards Reactive AI-Driven Moving Target Defence: Parameter Analysis and Adversarial Modelling in a Network Simulator

Supervisor: Dr Jin B. Hong

Degree: Bachelor of Advanced Computer Science (Honours)

1 Introduction

1.1 Moving Target Defence

Moving Target Defence (MTD) is the deliberate creation of asymmetric uncertainty that redirects the cost advantage from the attacker to the defender [1]. The fundamental design principle for developing MTD techniques lies in the decisions for the following three questions: *what to move, how to move, and when to move* [2]. Critically, there is a distinction between proactive MTD (reconfiguration at fixed intervals) and reactive MTD (triggered by specific events such as IDS alerts). This results in a fundamental trade-off. If the interval time is too large, the attacker has sufficient time to complete an attack before the next move. If the interval time is too small, the MTD triggers even when there is no attack on the system, wasting defence resources and degrading performance [3]. This is the core motivation for a reactive MTD that uses IDS feedback to make informed reconfiguration decisions rather than moving blindly.

Zhang (2023) [4] extended the foundational *MTDSim* simulator [5] into *MTDSimTime*, a Discrete-Event Simulation framework that integrates the time domain into the evaluation of combined MTD strategies. Zhang's evaluation demonstrates that network-shuffle-based techniques (IP Shuffle, Complete Topology Shuffle) consistently outperform diversity-based techniques by over 130% on average. However, the work has several limitations that constrain the real-world scalability of its conclusions. First, the attacker model is notably shallow: only one of two originally available attack scenarios was implemented, and the adversary follows a fixed action sequence with no adaptive intelligence, representing the lowest common denominator of attacker sophistication. Second, the MTD operations module was reduced from seven techniques to four, and evaluation was limited to single and pairwise combinations only, leaving open the question of whether further combinations yield further gains.

Tay (2024) [6] extended the *MTDSimTime* framework by producing *MTDShield*, a reinforcement learning module that uses Double Deep Q-Network (DDQN) to automate the selection and timing of MTD operations based on network security metrics. The system processes both static features and temporal metrics through separate neural network

branches before fusing them to produce a set of five Q-values representing possible MTD actions. This work is a meaningful step towards reactive MTD deployment, moving beyond fixed-interval scheduling by basing defensive actions on the observed network state. However, *MTDShield* has notable limitations. Most critically, some metrics used for decision making (such as Host Compromise Ratio or Mean Time to Compromise) are aggregate measures that assume the defender has complete and accurate knowledge of the network state. Further investigation of the input parameters driving MTD decision-making is needed to make the defence truly reactive.

Both works are constrained by aspects of the underlying simulation that remain underexplored. The attacker model lacks adaptive behaviour, the MTD action space remains artificially reduced from its original scope, and the input parameters driving the RL agent's decisions have not been systematically evaluated for their individual contribution to defensive outcomes. These elements require systematic investigation and strengthening as a precursor for further work.

1.2 Intrusion Detection in Moving Target Defence

MTD does not exist in isolation. An Intrusion Detection System (IDS) provides the observation channel through which defenders perceive attacker activity, while MTD provides the action space through which defenders alter the attack surface. The relationship is mutually reinforcing: MTD forces attackers into anomalous behaviours that IDS can more readily detect, while IDS provides the situational awareness needed for intelligent, event-driven reconfiguration rather than costly blind randomisation [7]. Previous works on this simulator [4] characterised IDS and firewalls as static legacy mechanisms that MTD supersedes. This proposal adopts a more nuanced framing, treating IDS as an architectural complement to MTD. A consequence of this integration is that the defender never observes the true state of the network. The defender sees only noisy IDS alerts, subject to missed detections, false alarms, and variable latency [8]. Network defence under these conditions is fundamentally a problem of partial observability, formalised in the literature as a Partially Observable Markov Decision Process (POMDP) [9]. Current work on this simulator does not enforce any such constraint: the RL agent learns from network security metrics directly.

Tay [6] undertook a preliminary investigation into this gap through a sensitivity study that varied the proportion of attacker actions visible to the model during training, simulating the effect of imperfect detection. The study found that model performance degrades below 70% detection visibility, with performance becoming uncorrelated with detection rate at lower sensitivities. However, this approach models IDS as a uniform random masking of attacker actions — a single scalar parameter applied identically across all attack types. Replacing this simplified model with a realistic IDS observation layer is a key stretch goal of this work.

1.3 Optimisation of MTD Operations

Beyond the observation model, prior work has also optimised the reinforcement learning agent's configuration. Tay [6] evaluated DDQN hyperparameters and conducted ablation studies on the *MTDShield* architecture, finding that removing either the static or temporal input module degraded performance by only 6–8%. Dunstall (2025) [10] extended this through multi-objective optimisation using NSGA-II, introducing cost metrics such as downtime ratio and MTD action ratio alongside security metrics. However, both efforts

inherit the full-observability assumptions, constrained action space, and simplified attacker model of the underlying simulator. This project includes systematic parameter exploration but treats it as a component of strengthening the simulation framework rather than an end in itself.

2 Aim

This project aims to strengthen the *MTDShield* simulation framework [4, 6, 10] and systematically evaluate how the AI-MTD module's input parameters, observation configurations, and adversarial conditions affect its defensive performance. Prior work evaluated the RL agent under fixed configurations and a simplified attacker; this project addresses those constraints. Specifically, this project will:

- **Develop a more sophisticated adversarial module** that exercises the AI-MTD agent beyond the capabilities of the existing baseline attacker, enabling non-trivial evaluation of defensive strategies.
- **Systematically explore AI-MTD input parameters and configurations** to identify which parameter regimes most meaningfully affect defensive outcomes.
- **Analyse and characterise the relationship between configuration choices and performance**, offering empirical insight into not only which settings improve outcomes but why.
- **(Stretch) Design and integrate an IDS observation module** that replaces the agent's current full-observability assumption with a realistic, partial-observation interface, and evaluate agent performance under degraded observation conditions.

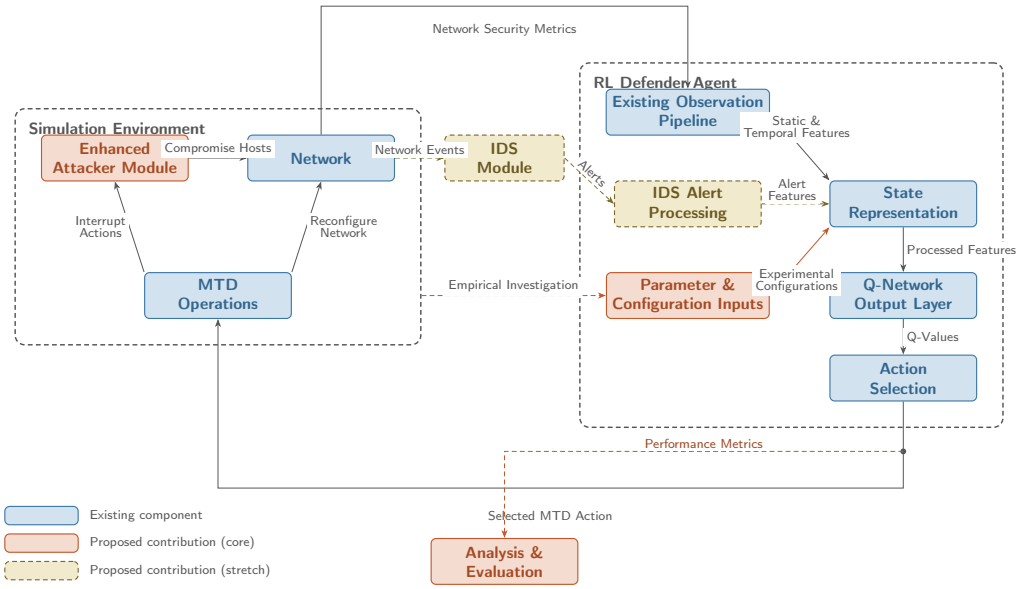


Figure 1: Overview of proposed contributions to the simulation architecture.

3 Method

This section outlines the approach for achieving each aim identified above, broken into sequential and overlapping phases. A consolidated timeline is presented in Section 3.5.

3.1 Codebase Comprehension and Baseline Establishment

A prerequisite to any experimental contribution is a thorough understanding of the existing *MTDShield* codebase and its antecedents [4, 6]. This phase, conducted in parallel with the literature review (March–April 2026), involves auditing the simulator architecture, reproducing key results from prior work to establish baseline performance figures, and documenting the codebase to support future extension. Baseline reproduction serves as the completion criterion: the phase is complete when prior results have been independently verified within an acceptable margin.

3.2 Adversarial Model Enhancement

The existing attacker follows a fixed action sequence with no adaptive behaviour, and only one of two originally available attack objectives was implemented [4]. This phase strengthens the adversarial module by first reintegrating the second attack objective, then introducing more sophisticated attack behaviours informed by mapping the simulator's action space to the MITRE ATT&CK framework to identify coverage gaps. New attack scenarios will implement multi-stage kill chains combining reconnaissance, lateral movement, and privilege escalation, calibrated so that the current AI-MTD policy exhibits measurable degradation relative to the baselines from Section 3.1. This phase is estimated at approximately two months (May–June 2026).

3.3 Parameter Exploration and Evaluation

With an enhanced attacker in place, this phase systematically varies the AI-MTD module's inputs and configurations across three categories: observation features, action-space configuration, and training hyperparameters (reward shaping, discount factor, exploration schedule). For each category, individual parameters will be varied against a held baseline (ablation studies), followed by interaction effects between promising combinations. Where parameter changes produce significant performance differences, further analysis will seek to explain *why* — for example, examining which observation features correlate most strongly with effective action selection. This phase runs concurrently with the later stages of adversarial enhancement (late May–late July 2026).

3.4 IDS Integration (Stretch Goal)

This phase is contingent on satisfactory progress in the preceding phases and is scoped as a proof of concept. The objective is to replace the RL agent's current full-observability assumption with an IDS module that intercepts simulated attacker events and produces alerts with configurable detection probability, false positive rate, and detection latency. The agent's observation space would be rewired to receive these alerts rather than ground-truth network state, and retrained under partial-observability conditions. The parameter exploration from Section 3.3 would then be extended to evaluate performance as IDS

fidelity is varied, building on Tay's preliminary sensitivity analysis [6]. If reached, this phase would occupy approximately August–September 2026.

3.5 Timeline

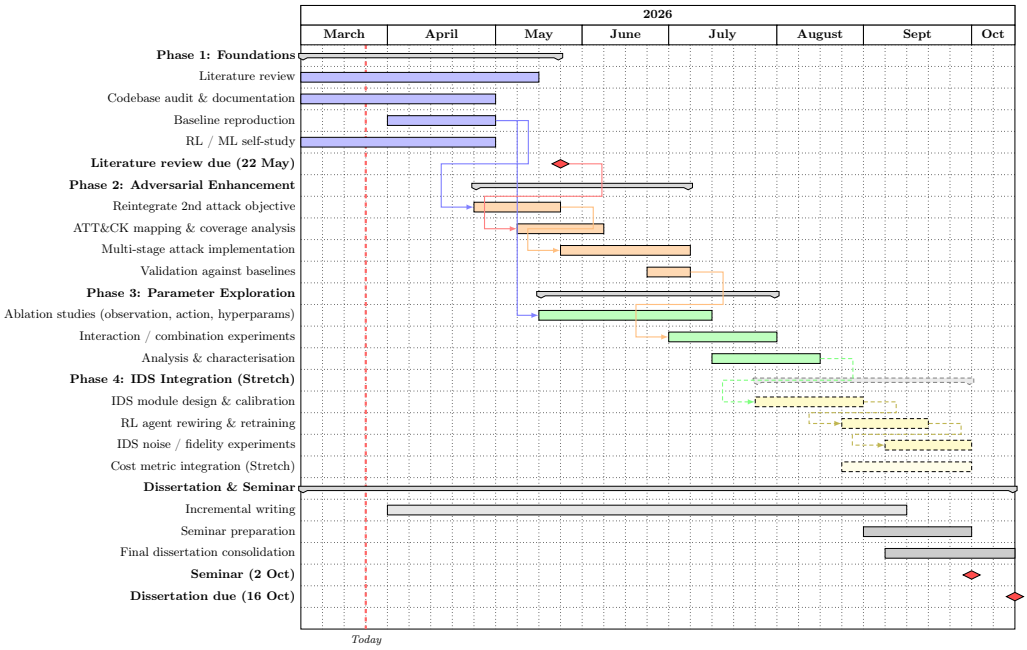


Figure 2: Project timeline. Dashed elements indicate stretch goals.

4 Project Requirements

The project will be conducted in Python 3, expanding on the *MTDShield* [6] framework and its dependencies. All prior codebases are publicly available on GitHub. Training of reinforcement learning models will be performed on commodity hardware; no specialised GPU infrastructure is anticipated. The simulator generates synthetic network data and does not involve human subjects, personal data, or external datasets; no ethics approval is required. Writing and code production will be assisted through the use of Claude Opus 4.6. No IP or access restrictions have been identified.

References

[1] S. Jajodia, A. K. Ghosh, V. Swarup, C. Wang, and X. S. Wang, Eds., *Moving Target Defense: Creating Asymmetric Uncertainty for Cyber Threats*, ser. Advances in Information Security, vol. 54. New York, NY, USA: Springer, 2011.

[2] G.-l. Cai, B.-s. Wang, W. Hu, and T.-z. Wang, "Moving target defense: State of the art and characteristics," *Frontiers of Information Technology & Electronic Engineering*, vol. 17, no. 11, pp. 1122–1153, Nov. 2016.

[3] S. Sengupta, A. Chowdhary, A. Sabur, A. Alshamrani, D. Huang, and S. Kambhampati, "A survey of moving target defenses for network security," *IEEE Communications Surveys & Tutorials*, vol. 22, no. 3, pp. 1909–1941, Jul. 2020.

[4] W. Zhang, "Evaluating multiple moving target defence in the time domain," Master's thesis, School of Engineering, The University of Western Australia, 2023.

[5] A. Brown, "MTDSim," GitHub repository, School of Engineering, The University of Western Australia, 2021. [Online]. Available: `https://github.com/Ccamm/MTDSim`

[6] J. K. Tay, "Using artificial intelligence to automate the deployment of moving target defence operations," Master's thesis, School of Engineering, The University of Western Australia, 2024.

[7] J.-H. Cho, D. P. Sharma, H. Alavizadeh, S. Yoon, N. Ben-Asher, T. J. Moore, D. S. Kim, H. Lim, and F. F. Nelson, "Toward proactive, adaptive defense: A survey on moving target defense," *IEEE Communications Surveys & Tutorials*, vol. 22, no. 1, pp. 709–745, 2020.

[8] A. Khraisat, I. Gondal, P. Vamplew, and J. Kamruzzaman, "Survey of intrusion detection systems: Techniques, datasets and challenges," *Cybersecurity*, vol. 2, no. 1, art. 20, 2019.

[9] A. S. M. McAbee, M. Tummala, and J. C. McEachen, "The use of partially observable Markov decision processes to optimally implement moving target defense," in *Proc. 54th Hawaii Int. Conf. Syst. Sci. (HICSS)*, 2021, pp. 1–10.

[10] K. Dunstall, "Multi-objective optimisation of artificial intelligence automated deployment of moving target defence," Honours thesis, School of Engineering, The University of Western Australia, 2025.

APPENDIX B

Predicate tests of the MTD attacker models

Table B.1 gives the predicates behind each mark of Tables 3.3 and 6.1. Its cells judge the APT attacker model from Chapter 4 and the three MTD attacker models from their papers. The predicates are Table 3.2's definitions; rows 1 and 6 split off a qualifier, row 5 splits at its "and", and rows 1 and 5 add words to make "over an extended period" and "low and slow" testable.

Distinct openings (Equation 4.6) backs the APT attacker model's mark for choice of attack strategy (Section 6.1). Counted in attack actions, as for the baseline attacker, the attack profiles have 190 to 844 distinct openings against its 3.

Table B.1: Each MTD attacker model tested on each predicate of Table 3.2. A cell is *yes* when the predicate is true of the attacker model's own behaviour, *rule* when only a fixed rule its authors set makes it true, *assumed* when the attacker model takes the step but only its authors assume it, and *no* otherwise. Each property's row gives its mark: ✓ when every cell is yes, ✗ when every behaviour predicate is no, and P otherwise.

Predicate	Baseline attacker	Masud et al.	Kim et al.	APT attacker model
1 Campaign persistence	✓	P	P	✓
behaviour: pursues one multi-stage campaign toward its objective	yes	yes	yes	yes
qualifier: over an extended period, until it reaches its objective, however many MTD deployments that takes	yes	no	no	yes
2 Objective-specific behaviour	✗	✗	✗	P
behaviour: after the foothold, uses different techniques for different objectives	no	no	no	assumed
3 Choice of attack strategy	P	✗	✗	P
behaviour: chooses among several sequences of techniques toward the same objective	rule	no	no	rule
4 Adaptivity	P	✗	✗	P
behaviour: chooses its next technique in response to an MTD disruption	rule	no	no	rule
5 Detection avoidance	✗	✗	P	P
behaviour: moves low and slow, slower than the baseline attacker	no	no	no	rule
behaviour: chooses techniques an IDS does not flag	no	no	assumed	no
6 Cost–benefit reasoning	✗	✗	✗	✗
behaviour: chooses techniques by weighing their cost against their benefit	no	no	no	no
qualifier: for each technique	no	no	no	no
7 Learning	✗	✗	✗	✗
behaviour: keeps the patterns it observes in past MTD deployments, and uses them to choose later techniques	no	no	no	no
8 Deployment-strategy awareness	✗	✗	✗	✗
behaviour: predicts and leverages which mechanism the deployment strategy deploys next, and when	no	no	no	no

APPENDIX C

Supplementary material for the APT attacker model

Each section of this appendix gives the data behind one part of the APT attacker model, in Chapter 4's order.

C.1 Attack graph with techniques as nodes

Section 4.1 chose tactics as the nodes of the attack graph because 419 of the 478 edges between techniques (88 %) are drawn by only one attack flow, against 47 of the 122 edges between tactics (39 %). Figure C.1 gives 55 of the 59 edges between techniques that two or more attack flows draw.

C.2 Sources for each attack flow's objective

Table C.1 gives the sources that answered the two questions of Section 4.2 for each attack flow. We decided the three attack flows these sources left open, each from its threat report. CISA AA22-138B VMWare Workspace (Alt) draws a script that the advisory says steals sensitive data, so its attacker stole data [59]. The attacker of Mac Malware Steals Crypto stole credentials and hijacked the victims' machines to mine cryptocurrency, so it stole data and impeded systems [60]. SearchAwesome Adware injects adverts but, by its report, takes no user data and impedes no system, so its attacker did neither [61]. The base weights count NotPetya and WhisperGate as one group, Sandworm (Section 4.3), although ATT&CK attributes WhisperGate to Ember Bear [24].

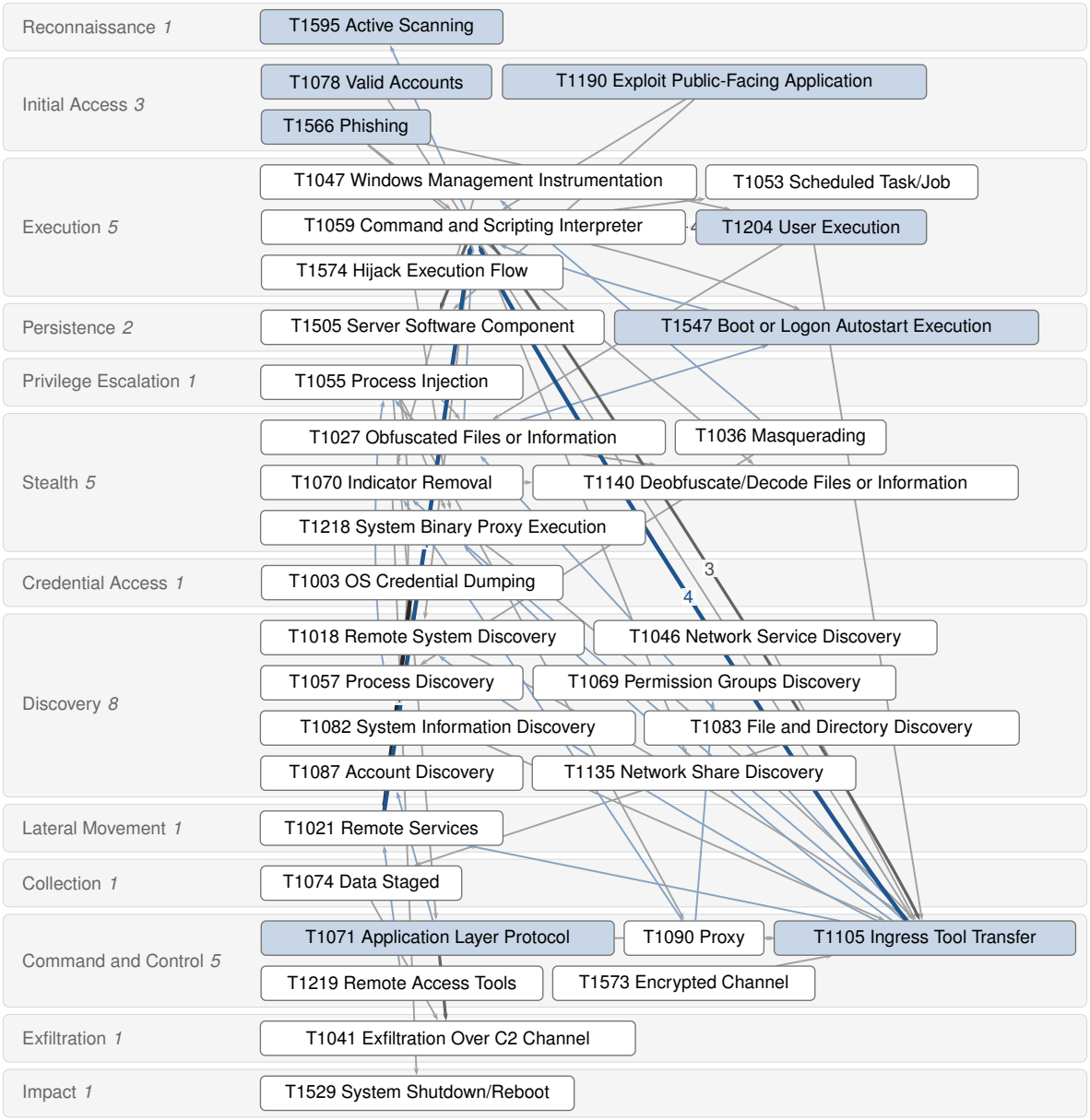


Figure C.1: The attack graph with techniques as nodes, limited to the edges drawn by two or more of the 38 attack flows and the connected groups of three or more techniques those edges form. The figure keeps 35 of its 124 techniques and 55 of its 478 edges. Techniques are grouped by tactic, in ATT&CK's order, and labelled by identifier. Each group gives its tactic and, in italic, how many of its techniques are drawn. Darker, thicker edges are drawn by more attack flows, and a number on an edge counts them where more than two did. Blue edges run back to an earlier tactic. A filled box is a technique some attack flow starts at.

Table C.1: The sources that answered the two questions of Section 4.2 for each attack flow, grouped by attack profile (Table 4.1). We read the CTID description of every attack flow [10] and the analyst's notes on its steps where needed. The columns give what else we read, and a blank row needed nothing else. *ATT&CK group* is the group page read [37], and *Threat report* the report the attack flow cites.

Attack flow	ATT&CK group	Threat report	Attack flow	ATT&CK group	Threat report
c_1 *(19 attack flows)*			c_2 *(7 attack flows)*		
CISA AA22-138B VMWare Workspace (Alt)		[59]	CISA Iranian APT		
CISA AA22-138B VMWare Workspace (TA1)		[59]	Maastricht University Ransomware		
Cobalt Kitty Campaign		[62]	NotPetya	G0034	
Equifax Breach		[63]	Shamoon		[64]
FIN13 Case 1	G1016		Sony Malware	G0032	
FIN13 Case 2	G1016		Tesla Kubernetes Breach		
Ivanti Vulnerabilities		[65]	WhisperGate		
JP Morgan Breach			c_3 *(7 attack flows)*		
Marriott Breach			Black Basta Ransomware		[66]
MITRE NERVE		[67]	Conti CISA Alert	G0102	
Muddy Water	G0069	[68]	Conti PWC	G0102	
OceanLotus	G0050		Conti Ransomware	G0102	
SolarWinds	G0016		Mac Malware Steals Crypto		[60]
SWIFT Heist	G0082		Ragnar Locker		
Target Breach			REvil	G0115	
ToolShell Vulnerability in Sharepoint		[69]	c_4 *(5 attack flows)*		
Turla - Carbon Emulation Plan	G0010		CISA AA22-138B VMWare Workspace (TA2)		[59]
Turla - Snake Emulation Plan	G0010		DFIR - BumbleBee Round 2		[70]
Uber Breach	G1004	[71]	Gootloader		[72]
			Hancitor DLL		[73]
			SearchAwesome Adware		[61]

C.3 Edges of the attack profiles

Each attack profile's Petri net takes only the edges its own attack flows drew (Section 4.2); Figure C.2 gives them. c_1's one edge into Impact is Muddy Water's step Data Encrypted for Impact, which its analyst drew with confidence 0; its threat report describes no encryption in this campaign [68].

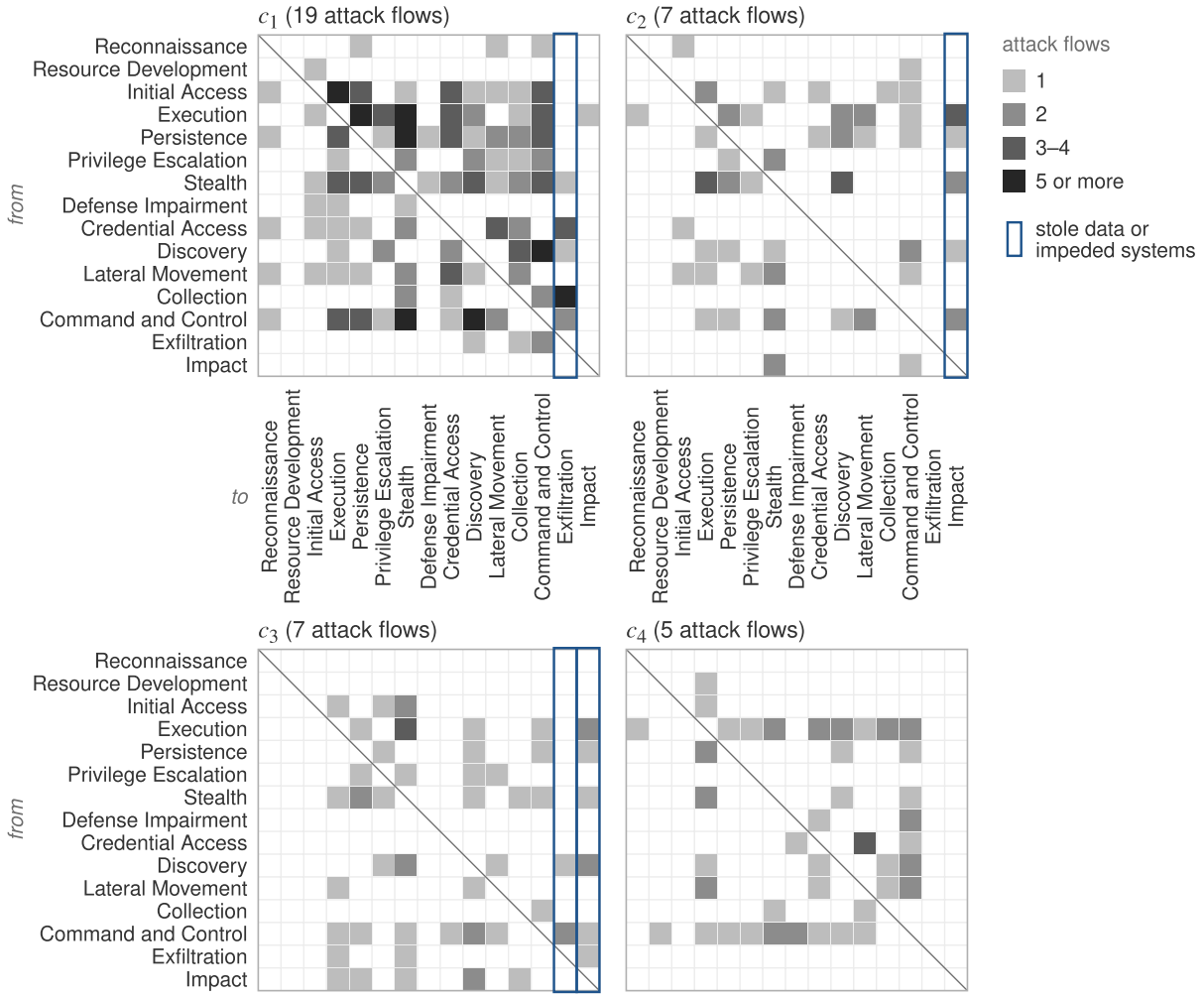


Figure C.2: Edges of the four attack profiles. Each cell is an edge from the tactic of its row to the tactic of its column. Darker cells are drawn by more of the attack profile's attack flows. A cell above the diagonal runs forward in ATT&CK's order of tactics, and one below it back to an earlier tactic. Exfiltration is outlined where the attackers stole data, and Impact where they impeded systems (Table 4.1).

C.4 Tactic durations and attack action times

An attack action can take a different time for each attacker (Section 6.5, T5); Table C.2 gives both times. Six tactics take MTDSim's time for the same kind of work, even where they run another attack action. Each takes the exploit's 4.5 s or the three scans' 35 s together. We set the other nine by judgement (Section 4.4.2).

Table C.2: Each tactic in ATT&CK's order with the attack action it runs (Figure 4.6), that attack action's time in MTDSim [15] and the tactic's mean duration. The baseline attacker takes MTDSim's time, and the APT attacker model takes the mean duration instead. An exploit takes 0 to 9 s by the vulnerability. The exploit's time is its mean.

Tactic	Attack action	MTDSim's time (s)	Mean duration (s)
Reconnaissance	scan host	5	35
Resource Development	*unmapped*		0
Initial Access	exploit	4.5	4.5
Execution	exploit	4.5	22.5
Persistence	*unmapped*		45
Privilege Escalation	exploit	4.5	4.5
Stealth	*unmapped*		45
Defense Impairment	*unmapped*		22.5
Credential Access	brute force	20	4.5
Discovery	scan port	25	35
Lateral Movement	enumerate host	5	4.5
Collection	*unmapped*		36
Command and Control	scan neighbours	5	45
Exfiltration	*unmapped*		36
Impact	*unmapped*		36

C.5 Failure matrix rules and distance factor

Each factor of the failure matrix (Section 4.4.3) is the value of the first rule of Table C.3 that applies to the move, times a distance factor. The distance factor keeps the rule value for a move within a lifecycle stage or to an adjacent one, quarters it for a move two stages away, and sets it to zero for a move three away.

Table C.3: The rules of the failure matrix (Section 4.4.3). After a tactic's attack action fails, the first rule from A down that applies to a candidate next tactic gives its value. In ATT&CK's order, a tactic after Initial Access needs a foothold (Section 3.1.1). *Stage* is the next tactic's lifecycle stage against the failed tactic's.

Rule	Failed tactic	Next tactic	Stage	Value	Reason
A	Initial Access	after Initial Access	any	0.02	No foothold was gained, so these moves are almost out of reach.
B	Reconnaissance	Initial Access	any	0.4	Nothing was found, so breaking in is harder but still possible.
C	Reconnaissance	after Initial Access	any	0.05	With no foothold, a move deep into the network is implausible.
D	after Initial Access	up to Initial Access	earlier	0.25	Falling right back to before the foothold is an uncommon response.
E	any	Execution	earlier	0.35	Running code again on a held foothold continues the attack.
F	any	any	earlier	0.9	Going back one stage is the common response to a failure.
G	any	any	same	0.7	Another tactic of the stage is an alternative route.
H	after Initial Access	any	later	0.35	Advancing straight after a failure is possible but uncommon.
I	up to Initial Access	any	later	0.3	The step the next one depends on did not succeed.

C.6 Sources of the APT attacker model's values

The values we set by judgement (Section 6.5, T5) come first in Table C.4.

Table C.4: The source of every value the APT attacker model needs and the section that gives its reason.

Source	Value	Reason in
Our judgement	Mean durations of nine tactics	Section 4.4.2
	Tactic-to-action mapping	Section 4.4.1
	Failure-matrix values	Table C.3
	Weights of the pre-intrusion overlay	Section 4.3
	One attack flow per APT group	Section 4.3
	Vulnerability memory's factor of three	Section 4.4.4
Attack Flow corpus	Base weights of the Petri nets	Section 4.3
	Membership of each attack profile	Section 4.2
MTDSim	Mean durations of six tactics, from MTDSim's scans and exploit	Section 4.4.2
	Confusion penalty	Section 2.2.3
Literature	Order of the lifecycle stages	Section 3.1.1
	Exponential delay	Section 4.4.2

APPENDIX D

Supplementary material for the MTD evaluation

Section D.5.1 backs Chapter 6, and the rest of this appendix backs Chapter 5.

D.1 MTDShield's settings

MTDShield in Chapter 5 is the agent Tay scored highest in his hyperparameter evaluation [14, Sect. 5.2]. It runs as released, with his weights and input code [15]. Chapter 5 runs the random deployment strategy over the same four MTD mechanisms beside it, so the difference between the two is what the agent's policy adds. Table D.1 compares its training with the runs of Chapter 5.

Table D.1: MTDShield's settings in the release's training script [15] and as run in Chapter 5 (Table 5.1). In both, it chooses among complete topology shuffle, IP shuffle, OS diversity and service diversity, or nothing. It is told the attacker's current attack action at every decision. A deployment is forced after 2 000 s without one.

Setting	Training	Chapter 5
Hosts	100, a new network each episode	50
Time limit	5 000 s per episode	15 000 s per run
Deployment interval	200 s	50, 100, 200, 500, 1 000 and 2 000 s
Attacker	the baseline attacker	the baseline attacker and the APT attacker model

Two changes let MTDShield run: Tay's input construction is restored as an option beside the simulator's, and a decision to do nothing waits for the next deployment interval instead of stopping the clock. As released, its evaluation code reads four of its 11 inputs before the run starts, so they are zero at every decision, as in Tay's own evaluations. With the inputs read live, as in its training code, its NCR reduction against the baseline attacker does not change measurably; against the APT attacker model, the training code fails in most runs. The agent reads an unmapped tactic of the APT attacker model as nothing detected. At 200 s, the agent chose service diversity in 76 % of its decisions against the baseline attacker and 70 % against the APT attacker model, and nothing in 23 % and 29 % (100 runs each).

D.2 Values of attack actions blocked and time lost

Table D.2 gives the values Figure 5.2 plots.

Table D.2: Attack actions blocked and time lost per MTD deployment at a deployment interval of 2 000 s, each with its 95 % confidence interval (Figure 5.2). The APT attacker model is averaged over c_1 to c_4. Time lost is over the MTD deployments made while the same seed's run with no MTD is still going. These are at least 78 % of each cell's MTD deployments. A printed 1.000 is exact.

MTD mechanism	APT attacker model		Baseline attacker	
	Attack actions blocked per MTD deployment	Time lost per MTD deployment (s)	Attack actions blocked per MTD deployment	Time lost per MTD deployment (s)
host layer				
IP shuffle	0.351 (0.345, 0.357)	324 (314, 335)	1.000 (1.000, 1.000)	−26.7 (−39.8, −13.8)
complete topology shuffle	0.393 (0.387, 0.398)	242 (230, 253)	1.000 (1.000, 1.000)	−13.9 (−29.4, 1.9)
host topology shuffle	0.391 (0.385, 0.397)	246 (235, 257)	1.000 (1.000, 1.000)	−14.7 (−29.9, −0.2)
service layer				
port shuffle	0.150 (0.146, 0.154)	26 (18, 34)	0.951 (0.946, 0.957)	−23.6 (−35.0, −11.7)
OS diversity	0.151 (0.147, 0.155)	−8 (−17, 1)	0.951 (0.946, 0.957)	−58.2 (−71.6, −44.6)
service diversity	0.150 (0.145, 0.154)	60 (51, 69)	0.951 (0.946, 0.956)	492.5 (468.6, 515.6)
credentials				
user shuffle	0.009 (0.008, 0.010)	−26 (−34, −18)	0.068 (0.061, 0.075)	−22.7 (−36.6, −9.0)

D.3 Values of the MTD ranking

Table D.3 gives the values Figure 5.3 plots, and Table D.4 gives rank 1 of Table 5.3 at every deployment interval. Its intervals resample runs, but runs on one seed share a network, so they are not independent. Resampling seeds instead, with the APT attacker model's four attack profiles averaged at each seed, keeps the host layer and the service layer apart at every deployment interval against both attackers. They are closest for the baseline attacker at 500 s, where the service layer leads by 0.11 (0.09, 0.12).

Figure D.1 and Table D.5 give the same comparison on ASP reduction, NCR reduction (Equation 4.12) with ASP in place of NCR:

$$\text{ASP reduction} = 1 - \frac{\text{ASP under MTD}}{\text{ASP with no MTD}}. \tag{D.1}$$

ASP reduction is 1 when no run under MTD compromises a target host. At 200 s, ASP reduction shows the order of NCR reduction. Against the APT attacker model, whose ASP with no MTD is 0.09, it cannot separate the six that reach 0.99 or more: every host-layer MTD mechanism and the random, random over MTDShield's four, and alternative deployment strategies. Chapter 5 therefore ranks on NCR reduction.

Table D.3: NCR reduction for each MTD mechanism and deployment strategy against each attacker at every deployment interval (Figure 5.3). The APT attacker model is averaged over c_1 to c_4. An italic row is the layer's mean the figure plots. Grey: the 95 % interval includes zero. Random, MTDShield's four: the random deployment strategy over the four MTD mechanisms MTDShield chooses from (Appendix D.1).

	APT attacker model						Baseline attacker					
Deployment interval (s)	50	100	200	500	1 000	2 000	50	100	200	500	1 000	2 000
host layer, mean	0.98	0.96	0.96	0.79	0.52	0.24	0.72	0.52	0.49	0.16	0.06	−0.01
IP shuffle	0.99	0.97	0.96	0.81	0.58	0.33	0.88	0.70	0.68	0.39	0.30	0.19
complete topology shuffle	0.97	0.95	0.95	0.77	0.48	0.19	0.58	0.42	0.42	0.09	−0.03	−0.09
host topology shuffle	0.99	0.97	0.95	0.77	0.48	0.20	0.70	0.44	0.38	0.01	−0.08	−0.11
service layer, mean	0.35	0.35	0.21	0.09	0.05	0.02	0.97	0.97	0.84	0.27	0.19	0.11
port shuffle	0.36	0.36	0.20	0.07	0.04	0.02	0.99	0.99	0.91	0.10	0.05	0.03
OS diversity	0.18	0.19	0.07	0.00	0.00	−0.02	0.92	0.93	0.68	0.04	−0.01	−0.01
service diversity	0.50	0.50	0.36	0.20	0.11	0.06	0.99	0.99	0.92	0.67	0.54	0.30
credentials												
user shuffle	−0.13	−0.11	−0.10	−0.06	−0.06	−0.04	0.45	0.07	−0.05	−0.06	−0.05	−0.03
deployment strategies												
random	0.95	0.90	0.75	0.43	0.22	0.08	0.67	0.52	0.33	0.06	−0.01	−0.01
MTDShield	0.42	0.39	0.28	0.15	0.09	0.04	0.84	0.83	0.68	0.40	0.19	0.05
random, MTDShield's four	0.95	0.92	0.80	0.51	0.28	0.13	0.78	0.68	0.56	0.19	0.09	0.04
alternative	0.94	0.89	0.72	0.42	0.19	0.07	0.65	0.58	0.38	0.05	−0.02	−0.02

Table D.4: Rank 1 by NCR reduction (Table 5.3) against each attacker at every deployment interval. The APT attacker model is averaged over c_1 to c_4.

Deployment interval (s)	APT attacker model	Baseline attacker
50	IP shuffle, host topology shuffle and complete topology shuffle	service diversity and port shuffle
100	IP shuffle, host topology shuffle and complete topology shuffle	service diversity and port shuffle
200	IP shuffle, host topology shuffle and complete topology shuffle	service diversity and port shuffle
500	IP shuffle	service diversity
1 000	IP shuffle	service diversity
2 000	IP shuffle	service diversity

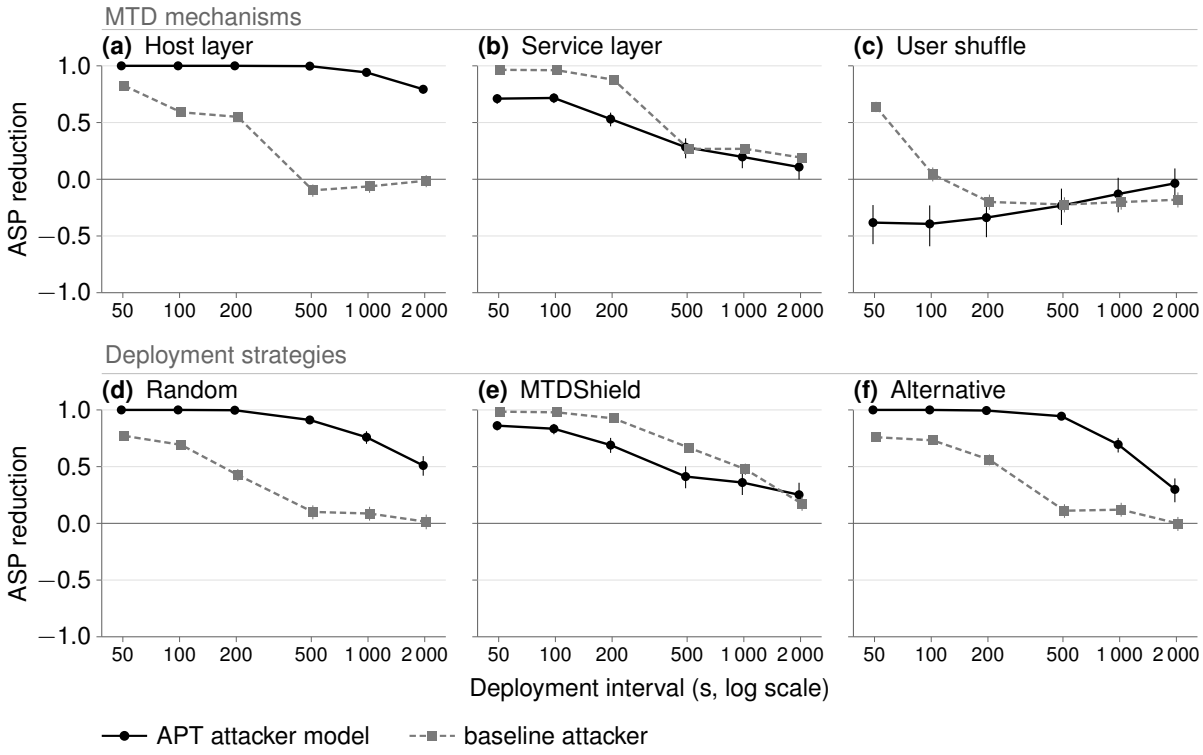


Figure D.1: ASP reduction against the deployment interval, drawn as Figure 5.3 draws NCR reduction. The APT attacker model is averaged over c_1 to c_4. A point at 1 may be a reduction above 0.99. Table D.5 prints it as > 0.99.

Table D.5: ASP reduction (Figure D.1), laid out as Table D.3. Grey: the 95 % interval includes zero. A printed 1.00 is exactly 1; a value above 0.99 but short of 1 prints as > 0.99.

	APT attacker model						Baseline attacker					
Deployment interval (s)	50	100	200	500	1 000	2 000	50	100	200	500	1 000	2 000
host layer, mean	1.00	1.00	1.00	>0.99	0.94	0.79	0.82	0.59	0.55	−0.10	−0.06	−0.01
IP shuffle	1.00	1.00	1.00	0.99	0.85	0.61	0.97	0.78	0.76	−0.09	−0.11	−0.10
complete topology shuffle	1.00	1.00	1.00	1.00	0.98	0.92	0.64	0.39	0.39	−0.23	−0.16	−0.05
host topology shuffle	1.00	1.00	1.00	>0.99	0.99	0.85	0.86	0.61	0.50	0.02	0.08	0.12
service layer, mean	0.71	0.72	0.53	0.28	0.20	0.11	0.96	0.96	0.88	0.27	0.27	0.19
port shuffle	0.73	0.78	0.52	0.20	0.16	0.07	1.00	1.00	0.97	−0.02	0.00	0.00
OS diversity	0.46	0.44	0.24	0.03	0.06	0.01	0.89	0.88	0.66	−0.11	−0.06	−0.05
service diversity	0.94	0.93	0.83	0.62	0.38	0.25	1.00	1.00	>0.99	0.93	0.87	0.62
credentials												
user shuffle	−0.38	−0.39	−0.34	−0.23	−0.13	−0.04	0.64	0.04	−0.20	−0.22	−0.20	−0.18
deployment strategies												
random	1.00	1.00	>0.99	0.91	0.76	0.51	0.77	0.69	0.43	0.10	0.09	0.01
MTDShield	0.86	0.83	0.69	0.41	0.36	0.25	0.98	0.98	0.93	0.67	0.48	0.17
random, MTDShield's four	1.00	1.00	1.00	0.96	0.83	0.52	0.92	0.84	0.74	0.15	0.18	0.06
alternative	1.00	1.00	0.99	0.94	0.70	0.30	0.76	0.73	0.56	0.11	0.12	0.00

D.4 Sensitivity analysis of the tactic durations

Most tactic durations are our judgement (Section 4.4.2), so this sensitivity analysis tests whether the results depend on them. Each of the four mean durations of Table D.6 was moved to both ends of its band, one at a time, with the other three held at their declared values; moving one value at a time leaves their interactions untested [74]. Only the 45 s mean duration of Persistence, Stealth and Command and Control changes NCR much (Table D.6).

Table D.6: NCR at the low end, the declared value and the high end of each mean duration's band (Table 4.2). The APT attacker model is averaged over c_1 to c_4 with 400 runs per cell. Execution and Defense Impairment have half the 45 s mean duration, so moving the 45 s moves theirs too. *Change*: high end minus low end. Bold: the largest. Every NCR's 95 % interval is within ±0.014.

	No MTD				Random, 200 s			
Mean duration (band), s	low end	declared	high end	change	low end	declared	high end	change
4.5 (2.25 to 9)	0.165	0.163	0.159	−0.006	0.043	0.041	0.038	−0.005
35 (17.5 to 70)	0.172	0.163	0.145	−0.027	0.053	0.041	0.029	−0.024
36 (18 to 72)	0.170	0.163	0.148	−0.022	0.047	0.041	0.034	−0.013
45 (11.25 to 180)	0.267	0.163	0.059	**−0.208**	0.150	0.041	0.006	**−0.144**

We therefore repeated the 200 s ranking of Section 5.3.2 at both ends of the 45 s band, on the same 1 000 seeds, by a test fixed before the run (Table D.7). The comparison holds at both ends. The host layer still ranks above the service layer, the APT attacker model's rank 1 still shares no MTD mechanism with the baseline attacker's, and the two rankings stay uncorrelated.

Table D.7: The rank of each MTD mechanism and deployment strategy by NCR reduction at a deployment interval of 200 s, with the APT attacker model's 45 s mean duration at its declared value and at the low and high ends of its band. The baseline attacker's ranks are Table 5.3's, and the APT attacker model is averaged over c_1 to c_4. Bold: rank 1. ρ: Spearman's rank correlation with the baseline attacker's NCR reductions. Every 95 % interval of ρ lies between −0.27 and 0.05.

		APT attacker model			
MTD	Group	declared (45 s)	low end (11.25 s)	high end (180 s)	Baseline attacker
IP shuffle	host layer	**1**	**1**	**1**	2
complete topology shuffle	host layer	**1**	2	**1**	3
host topology shuffle	host layer	**1**	2	**1**	3
random	strategy	2	3	2	4
alternative	strategy	3	3	2	3
service diversity	service layer	4	4	3	**1**
MTDShield	strategy	5	5	4	2
port shuffle	service layer	6	6	4	**1**
OS diversity	service layer	7	7	5	2
user shuffle	credentials	8	8	5	5
ρ with the baseline attacker		0.01	0.03	−0.03	

D.5 Ablation study tables

Each table gives in full an ablation study of Section 5.4: the attack profiles at every MTD mechanism, deployment strategy and deployment interval (Table D.8), the attack profiles against random partitions of the attack flows (Table D.9), and the vulnerability memory at every number of services per operating system (Table D.10). Every d is on NCR (Section 5.1), on the same 1 000 seeds, with the APT attacker model averaged over c_1 to c_4.

Table D.8: d on NCR of the attack profiles minus the attack graph. Bold: not negligible. Negligible: service diversity at 1 000 and 2 000 s and MTDShield at every interval but 200 s. Every other plain d is inconclusive.

	Deployment interval (s)					
MTD	50	100	200	500	1 000	2 000
host layer						
IP shuffle	**−0.32**	**−0.41**	**−0.47**	**−0.70**	**−0.76**	**−0.56**
complete topology shuffle	**−0.35**	**−0.46**	**−0.44**	**−0.58**	**−0.54**	−0.25
host topology shuffle	**−0.30**	**−0.37**	**−0.43**	**−0.55**	**−0.45**	**−0.34**
service layer						
port shuffle	−0.22	−0.23	−0.21	−0.23	−0.26	−0.25
OS diversity	−0.26	−0.26	**−0.26**	−0.26	**−0.28**	−0.21
service diversity	−0.18	−0.19	−0.16	−0.14	−0.11	−0.12
credentials						
user shuffle	−0.20	−0.20	−0.24	**−0.27**	−0.24	−0.25
deployment strategies						
random	**−0.49**	**−0.47**	**−0.54**	**−0.36**	**−0.35**	−0.25
MTDShield	−0.09	−0.11	−0.14	−0.08	−0.13	−0.11
alternative	**−0.51**	**−0.57**	**−0.47**	**−0.33**	−0.25	−0.18

D.5.1 Ablation of the tactic durations

Section 6.1 explains the APT attacker model's differences from the baseline attacker with no MTD by its tactic durations and the order of its tactics. This ablation tests the tactic durations; it was chosen after the results of Chapter 5 were seen, and Section 6.5 bounds its use (T7). With no MTD and only its tactic durations changed, the APT attacker model runs again in two steps (Table D.11). First its unmapped tactics take no time. Then each mapped tactic also takes the baseline attacker's time for the attack action it runs. Command and Control takes 5 s for scan neighbours in place of 45 s, and Credential Access 20 s for brute force in place of 4.5 s. The two steps raise the APT attacker model's ASP, averaged over c_1 to c_4, from 0.09 to 0.45 against the baseline attacker's 0.66; the share of its gap in ASP they close is 0.63 (0.61, 0.65). An attack action fails when it runs before its preconditions are met (Section 4.4). The first step also runs under IP shuffle and service diversity at 200 s (Table D.12). Under service diversity, the APT attacker model then spends 45 % of its time in scan neighbours, the attack action of Command and Control. Under IP shuffle, 79 % of its attack actions fail for missing preconditions.

Table D.9: d on NCR of the attack profiles minus each of 10 random partitions of the attack flows into four groups of the attack profiles' sizes. Rows are in falling order of d with no MTD. Bold: not negligible, in the random partition's favour. Negligible: partitions 3, 8, 9 and 10 under IP shuffle at 200 s. Inconclusive: partition 9 under OS diversity at 200 s. Every other d is not negligible (Section 5.1).

		IP shuffle		OS diversity	
Random partition	no MTD	200 s	2 000 s	200 s	2 000 s
4	+1.47	+0.92	+1.82	+1.49	+1.52
1	+0.98	+0.48	+1.21	+0.88	+1.00
2	+0.88	+0.42	+1.25	+0.87	+0.96
3	+0.66	−0.01	+0.76	+0.67	+0.75
7	+0.62	+0.44	+0.88	+0.61	+0.67
6	+0.57	**−1.03**	+0.40	+0.56	+0.64
5	+0.45	**−0.48**	+0.40	+0.42	+0.49
10	+0.40	−0.05	+0.39	+0.37	+0.46
8	+0.33	−0.09	+0.51	+0.27	+0.38
9	+0.25	+0.06	+0.28	+0.16	+0.30
Not negligible (of 10)	10	6	10	9	10

Table D.10: The APT attacker model with and without the vulnerability memory at every number of services per operating system. d on NCR is with minus without. Parentheses give its 95 % interval. An exploit the host's operating system rules out is not counted.

	Exploits that succeed		NCR		
Services per operating system	with	without	with	without	d
no MTD					
1	0.92	0.69	0.151	0.146	+0.07 (+0.05, +0.10)
2	0.87	0.69	0.162	0.157	+0.09 (+0.06, +0.11)
3	0.84	0.69	0.168	0.161	+0.11 (+0.08, +0.13)
5	0.80	0.70	0.170	0.167	+0.05 (+0.02, +0.07)
10	0.76	0.70	0.173	0.169	+0.06 (+0.04, +0.09)
20	0.73	0.69	0.171	0.169	+0.04 (+0.02, +0.06)
service diversity, 200 s					
1	0.94	0.71	0.099	0.094	+0.11 (+0.09, +0.14)
2	0.91	0.70	0.104	0.099	+0.12 (+0.10, +0.15)
3	0.88	0.70	0.106	0.102	+0.10 (+0.07, +0.12)
5	0.84	0.71	0.108	0.104	+0.10 (+0.08, +0.13)
10	0.80	0.71	0.110	0.107	+0.08 (+0.05, +0.10)
20	0.76	0.71	0.109	0.106	+0.07 (+0.05, +0.09)
OS diversity, 200 s					
1	0.93	0.70	0.140	0.134	+0.09 (+0.06, +0.11)
2	0.89	0.69	0.149	0.142	+0.12 (+0.10, +0.15)
3	0.86	0.70	0.156	0.149	+0.13 (+0.10, +0.15)
5	0.82	0.70	0.159	0.153	+0.11 (+0.09, +0.14)
10	0.78	0.70	0.162	0.157	+0.08 (+0.05, +0.11)
20	0.74	0.70	0.159	0.156	+0.05 (+0.02, +0.07)

Table D.11: The APT attacker model with its tactic durations ablated, against the baseline attacker, with no MTD. The same 1 000 seeds in every row. Bold: past the baseline attacker's value with the 95 % intervals apart (a higher ASP, NCR or attack rate, or a lower attack confidentiality). Blank: not applicable, as the baseline attacker runs no tactics.

Attacker	ASP	NCR	Attack rate (per minute)	Attack confidentiality (%)	Attack actions failed (%)
Baseline attacker	0.66 (0.63, 0.69)	0.497 ± 0.012	1.709 ± 0.015	82.2 (81.8, 82.6)	
Tactic durations as declared					
c_1	0.12 (0.10, 0.15)	0.149 ± 0.004	1.087 ± 0.005	92.4 (92.2, 92.5)	14.5 ± 0.2
c_2	0.14 (0.12, 0.16)	0.209 ± 0.005	0.888 ± 0.006	96.0 (95.8, 96.1)	26.9 ± 0.4
c_3	0.06 (0.04, 0.07)	0.172 ± 0.006	0.680 ± 0.008	98.1 (98.0, 98.2)	29.8 ± 0.9
c_4	0.04 (0.03, 0.06)	0.154 ± 0.005	1.228 ± 0.009	89.4 (89.3, 89.6)	28.3 ± 0.5
Unmapped tactics at 0 s					
c_1	0.40 (0.37, 0.43)	0.261 ± 0.006	$\mathbf{2.146} \pm 0.011$	**71.5** (71.3, 71.8)	15.1 ± 0.3
c_2	0.44 (0.41, 0.47)	0.354 ± 0.008	$\mathbf{1.814} \pm 0.012$	81.9 (81.7, 82.1)	27.1 ± 0.4
c_3	0.08 (0.06, 0.10)	0.202 ± 0.008	1.256 ± 0.029	85.3 (84.9, 85.6)	44.8 ± 1.3
c_4	0.10 (0.08, 0.12)	0.220 ± 0.007	$\mathbf{1.831} \pm 0.014$	**78.0** (77.8, 78.3)	29.1 ± 0.5
Unmapped tactics at 0 s, mapped tactics at the baseline attacker's times					
c_1	0.63 (0.60, 0.66)	0.348 ± 0.011	$\mathbf{4.593} \pm 0.042$	**32.1** (31.7, 32.6)	18.9 ± 0.6
c_2	0.71 (0.68, 0.73)	0.447 ± 0.013	$\mathbf{4.052} \pm 0.029$	**37.7** (37.4, 38.1)	27.9 ± 0.5
c_3	0.08 (0.06, 0.10)	0.202 ± 0.009	$\mathbf{2.183} \pm 0.071$	**56.0** (54.8, 57.2)	53.3 ± 1.5
c_4	0.37 (0.34, 0.40)	0.384 ± 0.010	$\mathbf{4.063} \pm 0.032$	**36.9** (36.6, 37.3)	31.4 ± 0.5

Table D.12: The APT attacker model with its unmapped tactics at 0 s, against the baseline attacker, under MTD at a deployment interval of 200 s. The same 1 000 seeds in every row. The APT attacker model is averaged over c_1 to c_4. ±: the 95 % interval.

	NCR reduction		Attack actions blocked per MTD deployment	
Attacker	IP shuffle	service diversity	IP shuffle	service diversity
Baseline attacker	0.685 ± 0.013	0.918 ± 0.006	1	0.989 ± 0.001
APT attacker model				
tactic durations as declared	0.959 ± 0.003	0.364 ± 0.011	0.107 ± 0.002	0.168 ± 0.002
unmapped tactics at 0 s	0.848 ± 0.005	0.294 ± 0.012	0.270 ± 0.005	0.294 ± 0.004

Bibliography

[1] Mandiant, "M-Trends 2026." Special Report, Mandiant / Google Cloud, Mar. 2026. [Online]. Available: https://cloud.google.com/blog/topics/threat-intelligence/m-trends-2026. Accessed 5 July 2026.

[2] Cybersecurity and Infrastructure Security Agency, "PRC state-sponsored actors compromise and maintain persistent access to U.S. critical infrastructure." Joint Cybersecurity Advisory AA24-038A, Feb. 2024. Authored by CISA, NSA, and FBI, with DOE, EPA, TSA, ASD's ACSC, CCCS, NCSC-UK, and NCSC-NZ. First published 7 February 2024. [Online]. Available: https://www.cisa.gov/news-events/cybersecurity-advisories/aa24-038a. Accessed 31 August 2026.

[3] A. Alshamrani, S. Myneni, A. Chowdhary, and D. Huang, "A survey on advanced persistent threats: Techniques, solutions, challenges, and research opportunities," *IEEE Communications Surveys & Tutorials*, vol. 21, no. 2, pp. 1851–1877, 2019.

[4] Networking and Information Technology Research and Development (NITRD) Program, "National cyber leap year summit 2009 co-chairs' report," Technical Report, Networking and Information Technology Research and Development (NITRD) Program, Washington, DC, USA, 2009. [Online]. Available: https://www.nitrd.gov/nitrdgroups/images/b/bd/National_Cyber_Leap_Year_Summit_2009_CoChairs_Report.pdf.

[5] J.-H. Cho, D. P. Sharma, H. Alavizadeh, S. Yoon, N. Ben-Asher, T. J. Moore, D. S. Kim, H. Lim, and F. F. Nelson, "Toward proactive, adaptive defense: A survey on moving target defense," *IEEE Communications Surveys & Tutorials*, vol. 22, no. 1, pp. 709–745, 2020.

[6] A. Brown, T.-W. Lee, and J. B. Hong, "Evaluating moving target defenses against realistic attack scenarios," in *2023 IEEE/ACM 4th International Workshop on Engineering and Cybersecurity of Critical Systems (EnCyCriS)*, (Melbourne, Australia), pp. 1–8, 2023.

[7] R. Zhuang, S. Zhang, S. A. DeLoach, X. Ou, and A. Singhal, "Simulation-based approaches to studying effectiveness of moving-target network defense," in *Proceedings of the National Symposium on Moving Target Research*, 2012.

[8] Ł. Jalowski, M. Zmuda, M. Rawski, and P. Rekosz, "Rethinking the security assurances of MTD: A gap analysis for network defense," *Future Internet*, vol. 18, no. 2, 2026. Art. no. 89. [Online]. Available: https://www.mdpi.com/1999-5903/18/2/89.

[9] J. B. Hong, S. Y. Enoch, D. S. Kim, A. Nhlabatsi, N. Fetais, and K. M. Khan, "Dynamic security metrics for measuring the effectiveness of moving target defense techniques," *Computers & Security*, vol. 79, pp. 33–52, 2018.

[10] Center for Threat-Informed Defense, "Attack Flow." MITRE, 2026. Version 3.2.0. [Online]. Available: https://center-for-threat-informed-defense.github.io/attack-flow/. Accessed 21 May 2026.

[11] J. B. Hong and D. S. Kim, "Assessing the effectiveness of moving target defenses using security models," *IEEE Transactions on Dependable and Secure Computing*, vol. 13, no. 2, pp. 163–177, 2016.

[12] W. Zhang, "Evaluating multiple moving target defence in the time domain," Master's thesis, The University of Western Australia, Perth, Australia, May 2023.

[13] J. B. Hong and D. S. Kim, "HARMs: Hierarchical attack representation models for network security analysis," in *Proceedings of the 10th Australian Information Security Management Conference*, (Perth, Australia), pp. 74–81, Edith Cowan University, 2012. [Online]. Available: `https://ro.ecu.edu.au/ism/146/`.

[14] J. K. Tay, "Using artificial intelligence to automate the deployment of moving target defence operations," Master's thesis, The University of Western Australia, Perth, Australia, Oct. 2024.

[15] J. K. Tay, "MTDSim with MTDShield: Trained agents and evaluation code." GitHub repository, 2024. Agents from commit f13ed49 (23 September 2024); evaluation notebook at commit e6c13c6 (3 December 2024). [Online]. Available: `https://github.com/bsubs/MTDSim`.

[16] E. M. Hutchins, M. J. Cloppert, and R. M. Amin, "Intelligence-driven computer network defense informed by analysis of adversary campaigns and intrusion kill chains." Lockheed Martin Corporation white paper, 2011. [Online]. Available: `https://www.lockheedmartin.com/content/dam/lockheed-martin/rms/documents/cyber/LM-White-Paper-Intel-Driven-Defense.pdf`. Accessed 16 October 2026.

[17] W. H. Ho, "Using artificial intelligence to automate the deployment of MTD operation," Master's thesis, The University of Western Australia, Perth, Australia, Oct. 2024.

[18] Joint Task Force Transformation Initiative, "Managing information security risk: Organization, mission, and information system view," NIST Special Publication 800-39, National Institute of Standards and Technology, Gaithersburg, MD, USA, Mar. 2011.

[19] Mandiant, "APT1: Exposing one of China's cyber espionage units." Mandiant, Alexandria, VA, 2013. [Online]. Available: `https://services.google.com/fh/files/misc/mandiant-apt1-report.pdf`. Accessed 16 October 2026.

[20] N. I. Che Mat, N. Jamil, Y. Yusoff, and M. L. Mat Kiah, "A systematic literature review on advanced persistent threat behaviors and its detection strategy," *Journal of Cybersecurity*, vol. 10, no. 1, 2024. Art. no. tyad023.

[21] B. E. Strom, A. Applebaum, D. P. Miller, K. C. Nickels, A. G. Pennington, and C. B. Thomas, "MITRE ATT&CK: Design and philosophy," Technical Report MP180360R1, The MITRE Corporation, McLean, VA, 2018. Originally published July 2018; revised March 2020. [Online]. Available: `https://attack.mitre.org/docs/ATTACK_Design_and_Philosophy_March_2020.pdf`.

[22] Á. L. R. Ferraz, S. Barbieri, M. E. de Souza, and L. A. Pereira Júnior, "The procedural semantics gap in structured CTI: A measurement-driven STIX analysis for APT emulation." arXiv preprint arXiv:2512.12078 [cs.CR], 2026. Version 3, 6 May 2026. [Online]. Available: `https://arxiv.org/abs/2512.12078`.

[23] M. Büchel, T. Paladini, S. Longari, M. Carminati, S. Zanero, H. Binyamini, G. Engelberg, D. Klein, G. Guizzardi, M. Caselli, A. Continella, M. van Steen, A. Peter, and T. van Ede, "SoK: Automated TTP extraction from CTI reports – are we there yet?," in *34th USENIX Security Symposium (USENIX Security 25)*, (Seattle, WA), pp. 4621–4641, USENIX Association, Aug. 2025. [Online]. Available: `https://www.usenix.org/conference/usenixsecurity25/presentation/buechel`.

[24] The MITRE Corporation, "MITRE ATT&CK." Knowledge base, 2026. Enterprise domain, v19.1. [Online]. Available: `https://attack.mitre.org`. Accessed 2 September 2026.

[25] B. Al-Sada, A. Sadighian, and G. Oligeri, "MITRE ATT&CK: State of the art and way forward," *ACM Computing Surveys*, vol. 57, no. 1, 2024. Art. no. 12.

[26] M. Rodríguez, G. Betarte, and D. Calegari, "A process mining-based method for attacker profiling using the MITRE ATT&CK taxonomy," *Journal of Internet Services and Applications*, vol. 15, pp. 212–232, Aug. 2024. [Online]. Available: `https://journals-sol.sbc.org.br/index.php/jisa/article/view/3902`.

[27] M. R. Rahman, B. Wroblewski, Q. Matthews, B. Morgan, T. Menzies, and L. Williams, "Mining temporal attack patterns from cyberthreat intelligence reports," *Knowledge and Information Systems*, vol. 67, no. 10, pp. 8941–8981, 2025.

[28] Y. Zhang, T. Du, Y. Ma, X. Wang, Y. Xie, G. Yang, Y. Lu, and E.-C. Chang, "AttacKG+: Boosting attack graph construction with large language models," *Computers & Security*, vol. 150, 2025. Art. no. 104220.

[29] D. Evans, A. Nguyen-Tuong, and J. Knight, "Effectiveness of moving target defenses," in *Moving Target Defense: Creating Asymmetric Uncertainty for Cyber Threats* (S. Jajodia, A. K. Ghosh, V. Swarup, C. Wang, and X. S. Wang, eds.), vol. 54 of *Advances in Information Security*, ch. 2, pp. 29–48, Springer, 2011.

[30] M. Kim, J.-H. Cho, H. Lim, T. Moghaddam, T. J. Moore, F. F. Nelson, and D. D. Kim, "MTD in depth: Multi-phased moving target defense techniques against cyber-attacks based on cyber kill chain," *Future Generation Computer Systems*, vol. 181, 2026. Art. no. 108419.

[31] M. T. Masud, M. Keshk, N. Moustafa, B. Turnbull, and W. Susilo, "Vulnerability defence using hybrid moving target defence in Internet of Things systems," *Computers & Security*, vol. 153, 2025. Art. no. 104380.

[32] M. A. McQueen, W. F. Boyer, M. A. Flynn, and G. A. Beitel, "Time-to-compromise model for cyber risk reduction estimation," in *Quality of Protection: Security Measurements and Metrics* (D. Gollmann, F. Massacci, and A. Yautsiukhin, eds.), Advances in Information Security, pp. 49–64, Springer, 2006.

[33] P. K. Manadhata and J. M. Wing, "An attack surface metric," *IEEE Transactions on Software Engineering*, vol. 37, no. 3, pp. 371–386, 2011.

[34] H. Alavizadeh, S. Aref, D. S. Kim, and J. Jang-Jaccard, "Evaluating the security and economic effects of moving target defense techniques on the cloud," *IEEE Transactions on Emerging Topics in Computing*, vol. 10, no. 4, pp. 1772–1788, 2022.

[35] M. Cremonini and P. Martini, "Evaluating information security investments from attackers perspective: The return-on-attack (ROA)," in *Fourth Workshop on the Economics of Information Security (WEIS 2005)*, 2005.

[36] J. Mendonça, J.-H. Cho, T. J. Moore, F. F. Nelson, H. Lim, and D. D. Kim, "Performance impact analysis of services under a time-based moving target defense mechanism," *Journal of Defense Modeling and Simulation: Applications, Methodology, Technology*, vol. 20, no. 1, pp. 41–56, 2023.

[37] The MITRE Corporation, "Updates – April 2026." MITRE ATT&CK website, Apr. 2026. ATT&CK v19. Released 28 April 2026. [Online]. Available: `https://attack.mitre.org/resources/updates/updates-april-2026/`. Accessed 17 August 2026.

[38] J.-H. Cho and N. Ben-Asher, "Cyber defense in breadth: Modeling and analysis of integrated defense systems," *Journal of Defense Modeling and Simulation: Applications, Methodology, Technology*, vol. 15, no. 2, pp. 147–160, 2018.

[39] G. Cai, B. Wang, Y. Luo, and W. Hu, "A model for evaluating and comparing moving target defense techniques based on generalized stochastic Petri net," in *Advanced Computer Architecture* (J. Wu and L. Li, eds.), (Singapore), pp. 184–197, Springer Singapore, 2016.

[40] M. Ajmone Marsan, G. Conte, and G. Balbo, "A class of generalized stochastic Petri nets for the performance evaluation of multiprocessor systems," *ACM Transactions on Computer Systems*, vol. 2, no. 2, pp. 93–122, 1984.

[41] M. Ajmone Marsan, G. Balbo, G. Conte, S. Donatelli, and G. Franceschinis, *Modelling with Generalized Stochastic Petri Nets*. Chichester: John Wiley & Sons, 1995.

[42] H. Holm, "A large-scale study of the time required to compromise a computer system," *IEEE Transactions on Dependable and Secure Computing*, vol. 11, no. 1, pp. 2–15, 2014.

[43] B. B. Madan, K. Goševa-Popstojanova, K. Vaidyanathan, and K. S. Trivedi, "A method for modeling and quantifying the security attributes of intrusion tolerant systems," *Performance Evaluation*, vol. 56, no. 1–4, pp. 167–186, 2004.

[44] J. A. Bland, M. D. Petty, T. S. Whitaker, K. P. Maxwell, and W. A. Cantrell, "Machine learning cyberattack and defense strategies," *Computers & Security*, vol. 92, 2020. Art. no. 101738.

[45] Z. Zhan, M. Xu, and S. Xu, "Characterizing honeypot-captured cyber attacks: Statistical framework and case study," *IEEE Transactions on Information Forensics and Security*, vol. 8, no. 11, pp. 1775–1789, 2013.

[46] K. Zaffarano, J. Taylor, and S. Hamilton, "A quantitative framework for moving target defense effectiveness evaluation," in *Proceedings of the Second ACM Workshop on Moving Target Defense (MTD '15)*, (Denver, CO, USA), pp. 3–10, ACM, 2015.

[47] P. Royston and M. K. B. Parmar, "Restricted mean survival time: An alternative to the hazard ratio for the design and analysis of randomized trials with a time-to-event outcome," *BMC Medical Research Methodology*, vol. 13, no. 1, 2013. Art. no. 152.

[48] M. Pendleton, R. Garcia-Lebron, J.-H. Cho, and S. Xu, "A survey on systems security metrics," *ACM Computing Surveys*, vol. 49, no. 4, 2016. Art. no. 62.

[49] J. Jung, V. Paxson, A. W. Berger, and H. Balakrishnan, "Fast portscan detection using sequential hypothesis testing," in *Proceedings of the IEEE Symposium on Security and Privacy*, pp. 211–225, 2004.

[50] A. Arcuri and L. Briand, "A hitchhiker's guide to statistical tests for assessing randomized algorithms in software engineering," *Software Testing, Verification and Reliability*, vol. 24, no. 3, pp. 219–250, 2014.

[51] C. J. Clopper and E. S. Pearson, "The use of confidence or fiducial limits illustrated in the case of the binomial," *Biometrika*, vol. 26, no. 4, pp. 404–413, 1934.

[52] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. New York: Chapman and Hall/CRC, 1994.

[53] J. Cohen, *Statistical Power Analysis for the Behavioral Sciences*. Hillsdale, NJ: Lawrence Erlbaum Associates, 2nd ed., 1988.

[54] C. Tantithamthavorn, S. McIntosh, A. E. Hassan, and K. Matsumoto, "An empirical comparison of model validation techniques for defect prediction models," *IEEE Transactions on Software Engineering*, vol. 43, no. 1, pp. 1–18, 2017.

[55] C. Spearman, "The proof and measurement of association between two things," *The American Journal of Psychology*, vol. 15, no. 1, pp. 72–101, 1904.

[56] D. Lakens, A. M. Scheel, and P. M. Isager, "Equivalence testing for psychological research: A tutorial," *Advances in Methods and Practices in Psychological Science*, vol. 1, no. 2, pp. 259–269, 2018.

[57] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in Software Engineering*. Berlin, Heidelberg: Springer, 2012.

[58] R. G. Sargent, "Verification and validation of simulation models," in *Proceedings of the 2011 Winter Simulation Conference* (S. Jain, R. R. Creasey, J. Himmelspach, K. P. White, and M. Fu, eds.), pp. 183–198, 2011.

[59] Cybersecurity and Infrastructure Security Agency, "Threat actors chaining unpatched VMware vulnerabilities for full system control." Cybersecurity Advisory AA22-138B, CISA, 2022. First published 18 May 2022; last revised 2 June 2022. [Online]. Available: `https://www.cisa.gov/news-events/cybersecurity-advisories/aa22-138b`. Accessed 20 August 2026.

[60] Y. Chen, C. Zheng, W. Hu, and Z. Xu, "Mac malware steals cryptocurrency exchanges' cookies." Unit 42, Palo Alto Networks, 2019. Published 31 January 2019. [Online]. Available: https://unit42.paloaltonetworks.com/mac-malware-steals-cryptocurrency-exchanges-cookies/. Accessed 20 August 2026.

[61] T. Reed, "Mac malware intercepts encrypted web traffic for ad injection." Malwarebytes Labs, 2018. Published 24 October 2018. [Online]. Available: https://www.malwarebytes.com/blog/news/2018/10/mac-malware-intercepts-encrypted-web-traffic-for-ad-injection. Accessed 20 August 2026.

[62] A. Dahan, "Operation Cobalt Kitty: A large-scale APT in Asia carried out by the OceanLotus group." Cybereason, 2017. Published 24 May 2017. [Online]. Available: http://web.archive.org/web/20260803042637/https://www.cybereason.com/blog/operation-cobalt-kitty-apt. Accessed 20 August 2026.

[63] J. Fruhlinger, "Equifax data breach FAQ: What happened, who was affected, what was the impact?." CSO Online, 2020. Published 12 February 2020. [Online]. Available: https://www.csoonline.com/article/3444488/equifax-data-breach-faq-what-happened-who-was-affected-what-was-the-impact.html. Accessed 20 August 2026.

[64] A. Mundo, T. Roccia, J. Saavedra-Morales, and C. Beek, "Shamoon returns to wipe systems in Middle East, Europe." McAfee Labs, 2018. Published 14 December 2018. [Online]. Available: https://www.mcafee.com/blogs/other-blogs/mcafee-labs/shamoon-returns-to-wipe-systems-in-middle-east-europe/. Accessed 20 August 2026.

[65] M. Meltzer, R. J. Mora, S. Koessel, S. Adair, and T. Lancaster, "Active exploitation of two zero-day vulnerabilities in Ivanti Connect Secure VPN." Volexity, 2024. Published 10 January 2024. [Online]. Available: https://www.volexity.com/blog/2024/01/10/active-exploitation-of-two-zero-day-vulnerabilities-in-ivanti-connect-secure-vpn/. Accessed 20 August 2026.

[66] A. Elsad, "Threat assessment: Black Basta ransomware." Unit 42, Palo Alto Networks, 2022. Published 25 August 2022. [Online]. Available: https://unit42.paloaltonetworks.com/threat-assessment-black-basta-ransomware/. Accessed 20 August 2026.

[67] L. Crumpton and C. Clancy, "Technical deep dive: Understanding the anatomy of a cyber intrusion." MITRE Engenuity, 2024. Published 3 May 2024. [Online]. Available: https://medium.com/mitre-engenuity/technical-deep-dive-understanding-the-anatomy-of-a-cyber-intrusion-080bddc679f3. Accessed 20 August 2026.

[68] A. Malhotra and V. Ventura, "Iranian APT MuddyWater targets Turkish users via malicious PDFs, executables." Cisco Talos Intelligence, 2022. Published 31 January 2022. [Online]. Available: https://blog.talosintelligence.com/iranian-apt-muddywater-targets-turkey/. Accessed 20 August 2026.

[69] Unit 42, "Active exploitation of Microsoft SharePoint vulnerabilities: Threat brief." Unit 42, Palo Alto Networks, 2025. Published 31 July 2025, updated 12 August 2025. [Online]. Available: https://unit42.paloaltonetworks.com/microsoft-share

point-cve-2025-49704-cve-2025-49706-cve-2025-53770/. Accessed 20 August 2026.

[70] The DFIR Report, "BumbleBee: Round two." The DFIR Report, 2022. Published 26 September 2022. [Online]. Available: https://thedfirreport.com/2022/09/26/bumblebee-round-two/. Accessed 20 August 2026.

[71] Uber Technologies, "Security update." Uber Newsroom, 2022. Published 16 September 2022. [Online]. Available: https://www.uber.com/newsroom/security-update/. Accessed 20 August 2026.

[72] The DFIR Report, "SEO poisoning — a Gootloader story." The DFIR Report, 2022. Published 9 May 2022. [Online]. Available: https://thedfirreport.com/2022/05/09/seo-poisoning-a-gootloader-story/. Accessed 20 August 2026.

[73] The DFIR Report, "From zero to domain admin." The DFIR Report, 2021. Published 1 November 2021. [Online]. Available: https://thedfirreport.com/2021/11/01/from-zero-to-domain-admin/. Accessed 20 August 2026.

[74] A. Saltelli, K. Aleksankina, W. Becker, P. Fennell, F. Ferretti, N. Holst, S. Li, and Q. Wu, "Why so many published sensitivity analyses are false: A systematic review of sensitivity analysis practices," *Environmental Modelling & Software*, vol. 114, pp. 29–39, 2019.